جامعة الدلتا التكنولوجية

DELTA TECHNOLOGICAL UNIVERSITY

Linux Essentials



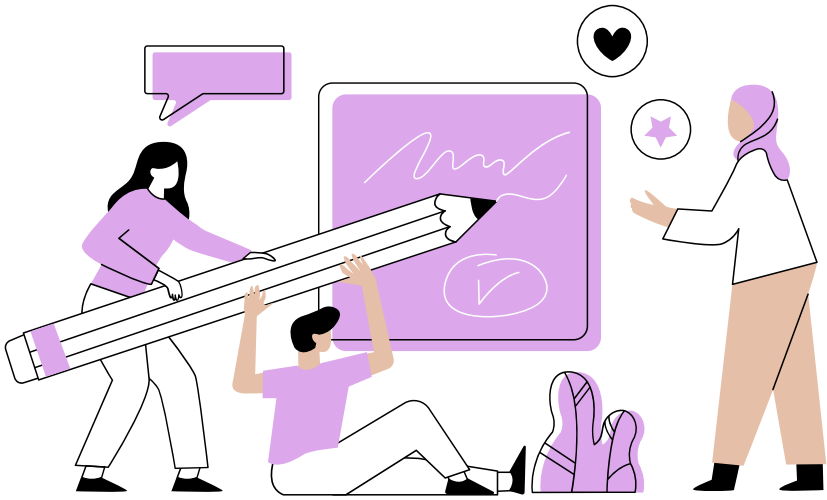
Collected & Organized by

Dr. Nesma Mahmoud

Delta Technological University

2026-2027

TABLE OF CONTENTS



01 Introduction to Linux

02 Linux Basic Commands

03 Users & Groups

04 Managing Files

05 Ownerships & Permissions

06 Programs & Processes

07 Linux File System

08 Managing Hardware

09 Managing Network

1. INTRODUCTION TO LINUX

The fact that you're reading this book means you want to learn about the Linux operating system (OS). To begin this journey, you must first understand what Linux is and what an OS is. This chapter is therefore devoted to these basic issues. I describe what an OS is, how users interact with an OS, how Linux compares to other OSs with which you may be familiar, and how specific Linux implementations vary. Understanding these issues will help you make your way as you learn about Linux and switch between Linux-based systems and other computers.

- **What is an OS?**
- **Investigating user interfaces**
- **Where does Linux fit in the OS world?**
- **What is a distribution?**

What Is an OS?

An OS provides all the most fundamental features of a computer, at least from a software point of view. An OS enables you to use the computer's hardware devices, it defines the user interface standards, and it provides basic tools that begin to make the computer useful. Ultimately, many of these features trace their way back to the OS's *kernel*, which is described in more detail next. Other OS features are owed to additional programs that run atop the kernel, as described later in this chapter.

What Is a Kernel?

An OS kernel is a software component that's responsible for managing various low-level features of the computer, including:

- Interfacing with hardware devices (network adapters, hard disks, and so on)

- Allocating memory to individual programs
- Allocating CPU time to individual programs
- Enabling programs to interact with each other

When you use a program (say, a Web browser), it relies on the kernel for many of its basic functions. The Web browser can only communicate with the outside world by using network functions provided by the kernel. The kernel allocates memory and CPU time to the Web browser, without which it couldn't run. The Web browser may rely on plug-ins to display multimedia content; such programs are launched and interact with the Web browser through kernel services. Similar comments apply to any program you run on a computer, although the details vary from one OS to another and from one program to another.

In sum, the kernel is the software "glue" that holds the computer together. Without a kernel, a modern computer can do very little.

Many programs run on multiple kernels, but most need OS-specific tweaks. Programmers create *binaries*—the program files for a particular processor and kernel—for each OS.

Kernels are not interchangeable; the Linux kernel is different from the Mac OS X kernel or the Windows kernel. Each of these kernels uses a different internal design and provides different software interfaces for programs to use. Thus, each OS is built from the kernel up and uses its own set of programs that further define each OS's features.

Linux uses a kernel called *Linux*—in fact, technically speaking, the word *Linux* refers *only* to the kernel. Other features that you might associate with Linux are provided by non-kernel programs, most of which are available on other platforms, as described shortly, in "What Else Identifies an OS."

A student named Linus Torvalds created the Linux kernel in 1991. Linux has evolved considerably since that time. Today, it runs on a wide variety of CPUs and other hardware. The easiest way to learn about Linux is to use it on a desktop or laptop PC, so that's the type of configuration that's emphasized in this book. The Linux kernel, however, runs on everything from tiny cell phones to powerful supercomputers.

What Else Identifies an OS?

The kernel is at the core of any OS, but it's a component that most users don't directly manipulate. Instead, most users interact with a number of other software components, many of which are closely associated with particular OSs. Such programs include the following:

Certification Objective

Command-line shells Years ago, users interacted with computers exclusively by typing commands in a program (known as a *shell*) that accepted such

commands. The commands would rename files, launch programs, and so on. Although many computer users today don't use text-mode shells, they're still important for intermediate and advanced Linux users, so I describe them in more detail in Chapter 6, "Getting to Know the Command Line," and subsequent chapters rely heavily on your ability to use a text-mode shell. Many different shells are available, and which shells are available and popular differ from one OS to another. In Linux, a shell known as the Bourne Again Shell (`bash` or Bash) is popular.

Graphical user interfaces A graphical user interface (GUI) is an improvement on a text-mode shell, at least from the perspective of a beginning user. Instead of using typed commands, GUIs rely on icons, menus, and a mouse pointer. Windows and Mac OS both have their own OS-specific GUIs. Linux relies on a GUI known as the X Window System, or X for short. X is a very basic GUI, so Linux also uses *desktop environment* program suites, such as the GNU Object Model Environment (GNOME) or the K Desktop Environment (KDE), to provide a more complete user experience. It's the differences between a Linux desktop environment and the GUIs in Windows or OS X that will probably strike you most when you first begin using Linux.

Certification Objective

Utility programs Modern OSs invariably ship with a wide variety of simple utility programs—calculators, calendars, text editors, disk maintenance tools, and so on. These programs differ from one OS to another. Indeed, even the names and methods of launching these programs can differ between OSs. Fortunately, you can usually find the programs you want by perusing menus in the main desktop environment.

Libraries Unless you're a programmer, you're unlikely to need to work with libraries directly; nonetheless, I include them in this list because they provide critical services to programs. Libraries are collections of programming functions that can be used by a variety of programs. For instance, in Linux most programs rely on a library called `libc`. Other libraries provide features associated with the GUI or that help programs parse options passed to them on the command line. Many libraries exist for Linux, which helps enrich the Linux software landscape.

Productivity programs Major productivity programs—Web browsers, word processors, graphics editors, and so on—are the usual reason for using a computer. Although such programs are often technically separate from the OS, they are sometimes associated with certain OSs. Even when a program is available on many OSs, it may have a different "feel" on each OS because of the different GUIs and other OS-specific features.

You can search for Linux equivalents to popular Mac OS X or Windows programs on Web sites such as `http://www.linuxrsp.ru/win-lin-soft/table-eng` or `http://www.linuxalt.com`.

In addition to software that runs on an OS, several other features can distinguish between OSs, such as the details of user accounts, rules for naming disk files, and technical details of how the computer starts up. These features are all controlled by software that's part of the OS, of course—sometimes by the kernel and sometimes by non-kernel software.

Investigating User Interfaces

Earlier, I noted the distinction between text-mode and graphical user interfaces. Although most end users favor GUIs because of their ease of use, Linux retains a strong text-mode tradition. Chapter 6 describes Linux's text-mode tools in more detail, and Chapter 4, "Using Common Linux Programs," covers basic principles of Linux GUI operations. It's important that you have some grounding in the basic principles of both text-mode and graphical user interfaces now, since user interface issues crop up from time to time in intervening chapters.

Using a Text-Mode User Interface

Certification Objective

In the past, and even sometimes today, Linux computers booted in text mode. Once the system had completely booted, the screen would display a simple text-mode login prompt, which might resemble this:

```
Debian GNU/Linux 6.0 essentials tty1

essentials login:
```

The details of such a login prompt vary from one system to another. This example includes several pieces of information:

- The OS name and version—Debian GNU/Linux 6.0
- The computer's name—`essentials`
- The name of the hardware device being used for the login—`tty1`
- The login prompt itself—`login:`

To log in to such a system, you must type your username at the `login:` prompt. The system then prompts you for a password, which you must also type. If you entered a valid username and password, the computer is likely to display a login message, followed by a shell prompt:

```
rodsmith@essentials:~$
```

To try a text-mode login, you must first install Linux on a computer. Neither the Linux Essentials exam nor this book covers Linux installation; consult your distribution's documentation to learn more.

If you see a GUI login prompt, you can obtain a text-mode prompt by pressing Ctrl+Alt+F1 or Ctrl+Alt+F2. To return to the GUI login prompt, press Alt+F1 or Alt+F7.

In this book, I omit most of the prompt from example commands when they appear on their own lines. I keep the dollar sign ($) prompt, though, for ordinary user commands. Some commands must be entered as `root`, which is the Linux administrative user. I change the prompt to a hash mark (#) for such commands, since most Linux distributions make a similar change to their prompts for the `root` user.

The details of this shell prompt vary from one installation to another, but you can type text-mode commands at the shell prompt. For instance, you could type **`ls`** (short for *list*) to see a list of files in the current directory. The most basic commands are shortened by removing vowels, and sometimes consonants, in order to minimize the amount of typing required to execute a command. This has the unfortunate effect of making many commands rather obscure.

Some commands display no information, but most produce some type of output. For instance, the `ls` command produces a list of files:

```
$ ls
106792c01.doc  f0101.tif
```

This example shows two files in the current directory: `106792c01.doc` and `f0101.tif`. You can use additional commands to manipulate these files, such as `cp` to copy them or `rm` to remove (delete) them. Chapter 6 ("Getting to Know the Command Line") and Chapter 7 ("Managing Files") describe some common file manipulation commands.

Some text-mode programs take over the display in order to provide constant updates or to enable you to interact with data in a flexible manner. Figure 1.1, for instance, shows the `nano` text editor, which is described in more detail in Chapter 11, "Editing Files." Once `nano` is working, you can use your keyboard's arrow keys to move the cursor around, add text by typing, and so on.

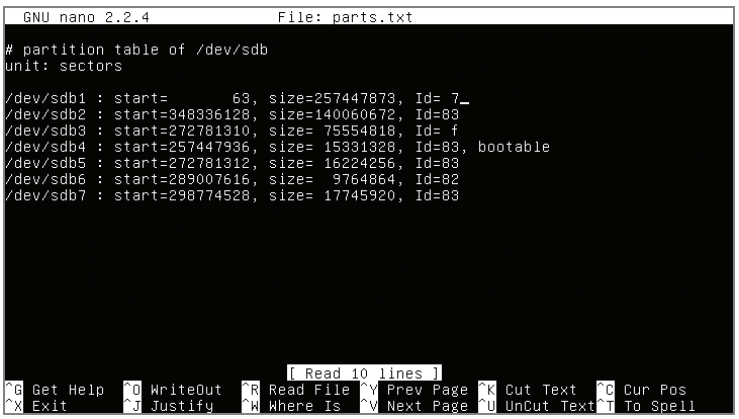


FIGURE 1.1 Some text-mode programs take over the entire display.

Chapter 13, "Understanding Users and Groups," describes Linux accounts, including the root account, in more detail.

Even if you use a graphical login, you can use a text-mode shell inside a window, known as a *terminal*. Common Linux GUIs provide the ability to launch a terminal program, which provides a shell prompt and the means to run text-mode programs.

Using a Graphical User Interface

Certification Objective

Most users are more comfortable with GUIs than with text-mode commands. Thus, many modern Linux systems start up in GUI mode by default, presenting a login screen similar to the one shown in Figure 1.2. You can select your username from a list or type it, followed by typing your password, to log in.

Some Linux GUI login screens don't prompt you for a password until after you've entered a valid username.

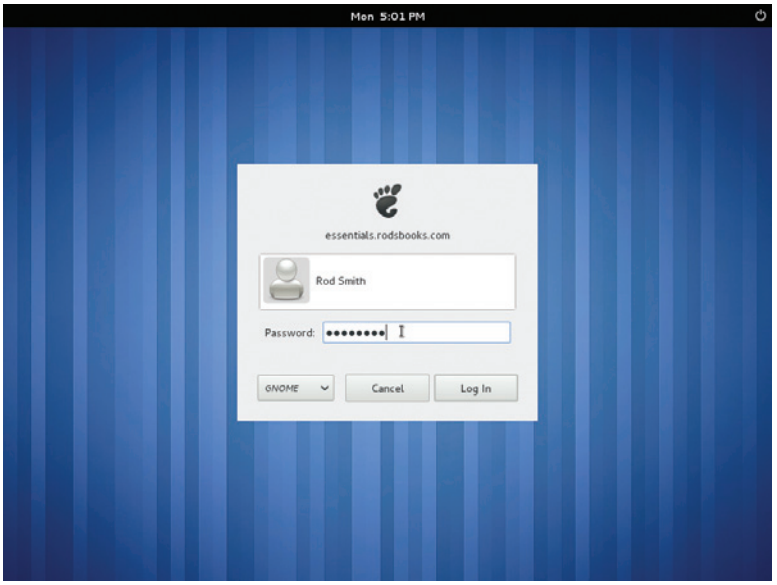


FIGURE 1.2 Graphical login screens on Linux are similar to those for Windows or Mac OS X.

Unlike Windows and Mac OS X, Linux provides a number of desktop environments. Which one you use depends on the specific variety of Linux you're using, what software options you selected at installation time, and your own personal preferences. Common choices include GNOME, KDE, Xfce, and Unity. Many other options are available as well. In Figure 1.2, you can see a selection option for the desktop environment in the lower-left corner of the central dialog box. It reads GNOME in Figure 1.2, meaning that if the item is left unchanged, the computer will launch GNOME when the user logs in.

Linux desktop environments can look quite different from one another, but they all provide similar functionality. Figure 1.3 shows the default KDE on an openSUSE 12.1 installation, with a couple of programs running. Chapter 4 describes common desktop environments and their features in more detail, but for now, you should know that they all provide features such as:

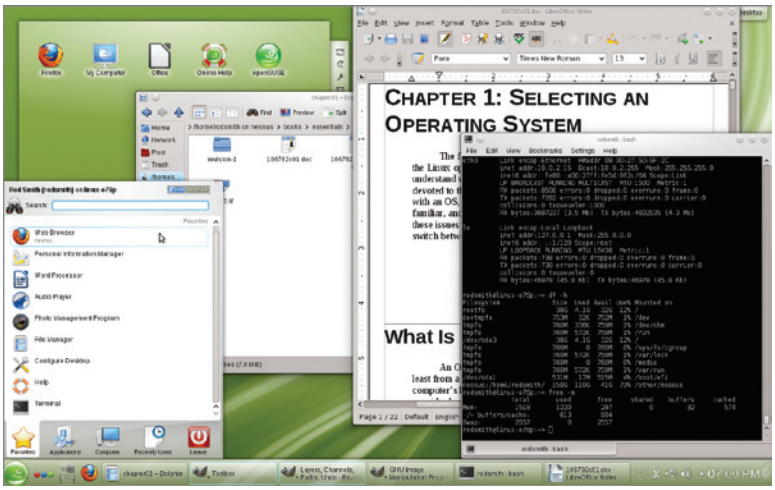


FIGURE 1.3 Linux desktop environments provide the types of GUI controls that most users expect.

Program launchers You can launch programs by selecting them from menus or lists. Typically, one or more menus exist along the top, bottom, or side of the screen. In Figure 1.3, you can click the openSUSE gecko icon in the bottom-left corner of the screen to produce the menu that appears in that figure.

File managers Linux provides GUI file managers similar to those in Windows or Mac OS X. A window for one of these is open in the center of Figure 1.3.

Window controls You can move windows by clicking and dragging their title bars, resize them by clicking and dragging their edges, and so on.

Multiple desktops Most Linux desktop environments enable you to keep multiple virtual desktops active, each with its own set of programs. This feature is very handy to keep the screen uncluttered while you run many programs simultaneously. Typically, an icon in one of the menus enables you to switch between virtual desktops.

Logout options You can log out of your Linux session, which enables you to shut down the computer or let another user log in.

Logging out is very important in public computing environments. If you fail to log out, a stranger might come along and use your account for malicious purposes.

You may need to install extra desktop environments to use them. This topic is not covered in this book.

As you learn more about Linux, you'll discover that its GUI environments are quite flexible. If you find you don't like the environment that's the default for your distribution, you can change it. Although they all provide similar features, some people have strong preferences about desktop environments. Linux gives you a choice in the matter that's not available in Windows or Mac OS X, so feel free to try multiple desktop environments.

Where Does Linux Fit in the OS World?

As described later, in "What Is a Distribution?," Linux can be considered a family of OSs. Thus, you can compare one Linux version to another one.

This chapter's title implies a comparison, and as this book is about Linux, the comparison must be to non-Linux OSs. Thus, I compare Linux to three other OSs or OS families: Unix, Mac OS X, and Microsoft Windows.

Comparing Linux to Unix

Certification Objective

If you were to attempt to draw a "family tree" of OSs, you would end up scratching your head a lot. This is because OS designers often mimic each other's features, and sometimes even incorporate each other's code into their OSs' workings. The result can be a tangled mess of similarities between OSs, with causes ranging from coincidence to code "borrowing." Attempting to map these influences can be difficult. In the case of Linux and Unix, though, a broad statement is possible: Linux is modeled after Unix.

Open source software is software that you can not only run, but modify and redistribute yourself. Chapter 2, "Investigating Linux's Principles and Philosophy," covers the philosophy and legal issues concerning open source software.

Unix was created in 1969 at AT&T's Bell Labs. Unix's history is complex and involves multiple *forks* (that is, splitting of the code into two or more independent projects) and even entirely separate code rewrites. Modern Linux systems are, by and large, the product of open source projects that clone Unix programs, or of original open source code projects for Unix generally. These projects include:

The Linux kernel Linus Torvalds created the Linux kernel as a hobby programming project in 1991, but it soon grew to be much more than that. The Linux kernel was designed to be compatible with other Unix kernels, in the sense that it used the same software interfaces in source code. This made using open source programs for other Unix versions with the Linux kernel easy.

The GNU project The GNU's Not Unix (GNU) project is an effort by the Free Software Foundation (FSF) to develop open source replacements for all the core elements of a Unix OS. In 1991, the FSF had already released the most important such tools, with the notable exception of the kernel. (The GNU HURD kernel is now available but is not as popular as the Linux kernel.) Alternatives to the GNU tools include proprietary commercial tools and open source tools developed for the BSD Unix variants. The tools used on a Unix-like OS can influence its overall

"flavor," but all of these tool sets are similar enough to give any Unix variety a similar feel compared to a non-Unix OS.

Xorg-X11 The X Window System is the GUI environment for most Unix OSs. Most Linux distributions today use the Xorg-X11 variety of X. As with the basic text-mode tools provided by the GNU project, choice of an X server can affect some features of a Unix-like OS, such as the types of fonts it supports.

Desktop environments GNOME, KDE, Xfce, and other popular open source desktop environments have largely displaced commercial desktop environments even on commercial versions of Unix. Thus, you won't find big differences between Linux and Unix in this area.

Server programs Historically, Unix and Linux have been popular as server OSs—organizations use them to run Web servers, e-mail servers, file servers, and so on. Linux runs the same popular server programs as do commercial Unix versions and the open source BSDs.

User productivity programs In this realm, as in server programs, Linux runs the same software as do other Unix-like OSs. In a few cases, Linux runs more programs, or runs them better. This is mostly because of Linux's popularity and the vast array of hardware drivers that Linux offers. If a program needs advanced video card support, for example, it's more likely to find that support on Linux than on a less popular Unix-like OS.

On the whole, Linux can be thought of as a member of the family of Unix-like OSs. Although Linux is technically *not* a Unix OS, it's similar enough that the differences are unimportant compared to the differences between this family as a whole and other OSs, such as Windows. Because of its popularity, Linux offers better hardware support, at least on commodity PC hardware. Some Unix varieties offer specific features that Linux lacks, though. For instance, the Zettabyte File System (ZFS), available on Solaris, FreeBSD, and some other OSs, provides advanced filesystem features that aren't yet fully implemented in Linux.

GNU is an example of a recursive acronym—an acronym whose expansion includes the acronym itself. This is an example of geek humor.

Mac OS X, described shortly, is a commercial Unix that eschews both X and the desktop environments that run on it in favor of Apple's own GUI.

A ZFS add-on for Linux is available, but it's not fully integrated into the OS. A Linux filesystem known as Btrfs offers many ZFS features, but Btrfs isn't yet complete.

Code Types

Human beings write programs in a form known as *source code*. Although source code can seem arcane to the uninitiated, it's crystal clear compared to the form a program must take for a computer to run it: *binary code*. A program known as a *compiler* translates source code to binary code. (Alternatively, some programming languages rely on an *interpreter*, which converts source code to binary code "on the fly," eliminating the need to compile source code.)

(Continues)

CODE TYPES *(Continued)*

The term *open source* refers to the availability of source code, which is generally withheld from the public in the case of commercial programs and OSs. A programmer with access to a program's source code can fix bugs, add features, and otherwise alter how the program operates.

Comparing Linux to Mac OS X

Certification Objective

Mac OS X is a commercial Unix-based OS that borrows heavily from the BSDs and discards the usual Unix GUI (namely X) in favor of its own user interface. This makes OS X both very similar to Linux and quite different from it.

You can open an OS X Terminal window and type many of the same commands described in this book to achieve similar ends. If a command described in this book isn't present, you may be able to install it in one way or another. OS X ships with some popular Unix server programs, so you can configure it to work much like Linux or another Unix-like OS as a network server computer.

The X in X server is a letter X, but the X in OS X is a Roman numeral (10), denoting the tenth version of Mac OS.

OS X differs from Linux in its user interface, though. The OS X user interface is known as *Cocoa* from a programming perspective, or *Aqua* from a user's point of view. It includes elements that are roughly equivalent to both X and a desktop environment in Linux. Because Cocoa isn't compatible with X from a programming perspective, applications developed for OS X can't be run directly on Linux (or on other Unix-like OSs), and porting them (that is, modifying the source code and recompiling them) for Linux is a non-trivial undertaking. Thus, native OS X applications seldom make the transition to Linux.

OS X includes an implementation of X that runs under Aqua. This makes the transfer of GUI Linux and Unix programs to OS X relatively straightforward. The resulting programs don't entirely conform to the Aqua user interface, though. They may have buttons, menus, and other features that look out of place compared to the usual appearance of OS X equivalents.

Apple makes OS X available for its own computers. Its license terms forbid installation on non-Apple hardware, and even aside from licensing issues, installing OS X on non-Apple hardware is a non-trivial undertaking. A variant of OS X, known as iOS, runs on Apple's iPad and iPhone devices, and is equally non-portable to other devices. Thus, OS X is largely limited to Apple hardware. Linux, by contrast, runs on a wide variety of hardware, including most PCs. You can even install Linux on Macintosh computers.

Comparing Linux to Windows

Certification Objective

Most desktop and laptop computers today run Microsoft Windows. Thus, if you're considering running Linux, the most likely comparison is to Windows. Broadly speaking, Linux and Windows have similar capabilities; however, there are significant differences in details. These include the following:

Licensing Linux is an open source OS whereas Windows is a proprietary commercial OS. Chapter 2 covers open source issues in greater detail, but for now you should know that open source software gives you greater control over your computer than does proprietary software—at least in theory. In practice, you may need a great deal of expertise to take advantage of open source's benefits. Proprietary software may be preferable if you work for an organization that's only comfortable with the idea of software that's sold in a more traditional way. (Some Linux variants, though, are sold in a similar way, along with service contracts.)

Costs Many Linux varieties are available free of charge, and so are appealing if you're trying to cut costs. On the other hand, the expertise needed to install and maintain a Linux installation is likely to be greater, and therefore more expensive, than the expertise needed to install and maintain a Windows installation. Different studies on the issue of total cost of ownership of Linux vs. Windows have gone both ways, but most tend to favor Linux.

Hardware compatibility Most hardware components require OS support, usually in the form of drivers. Most hardware manufacturers provide Windows drivers for their devices, or work with Microsoft to ensure that Windows includes appropriate drivers. Although some manufacturers provide Linux drivers, too, for the most part the Linux community as a whole must supply the necessary drivers. This means that Linux drivers may take a few weeks or even months to appear after a device becomes available. On the other hand, Linux developers tend to maintain drivers for old hardware for much longer than manufacturers continue to support their own old hardware. Thus, a modern Linux may run better than a recent version of Windows on old hardware. Linux also tends to be less resource-intensive, so you can be productive on older hardware when using Linux.

Software availability Some popular desktop applications, such as Microsoft Office, are available on Windows but not on Linux. Although Linux alternatives, such as OpenOffice.org or LibreOffice, are available, they haven't caught on in the public's mind. In other realms, the situation is reversed. Popular server programs, such as the Apache Web server, were developed first for Linux or Unix.

Although many such servers are available for Windows, they run more efficiently on Linux. If you have a specific program you must run, you may want to research its availability and practicality on any platforms you're considering.



Microsoft is making major changes to its user interface with Windows 8. The new user interface, Metro, works the same on everything from cell phones to desktop computers.

User interfaces Like Mac OS X, Windows uses its own unique user interface. This fact contributes to poor inter-OS portability. (Tools exist to help bridge the gap, though; X Window System implementations for Windows are available, as are tools for running Windows programs in Linux.) Some users prefer the Windows user interface to any Linux desktop environment, but others prefer a Linux desktop environment.

Configurability Linux is a much more configurable OS than is Windows. Although both OSs provide means to run specific programs at startup, change user interface themes, and so on, Linux's open source nature means you can tweak any detail you want. Furthermore, you can pick any Linux variant you like to get a head start on setting up the system as you see fit.

Security Advocates of each OS claim it's more secure than the other. They can do this because they focus on different security issues. Many of the threats to Windows come from viruses, which by and large target Windows and its huge installed user base. Viruses are essentially a non-issue for Linux; in Linux, security threats come mostly from break-ins involving misconfigured servers or untrustworthy local users.

For over a decade, Windows has dominated the desktop arena. In both homes and offices, users have become familiar with Windows and are used to popular Windows applications, such as Microsoft Office. Although Linux *can* be used in such environments, it's a less popular choice for a variety of reasons—its unfamiliarity, the fact that Windows comes pre-installed on most PCs, and the lack of any compelling Linux-only applications for most users.

Unix generally, and Linux in particular, on the other hand, have come to dominate the server market. Linux powers the Web servers, email servers, file servers, and so on that make up the Internet and that many businesses rely on to provide local network services. Thus, most people use Linux daily even if they don't realize it.

In most cases, it's possible to use either Linux or Windows on a computer and have it do an acceptable job. Sometimes, though, specific needs dictate use of one OS or another. You might need to run a particular exotic program, for instance, or your hardware might be too old for a modern Windows or too new for Linux. In other cases, your own or your users' familiarity with one OS or the other may favor its use.

What Is a Distribution?

Up until now, I've described Linux as if it were a single OS, but this isn't really the case. Many different Linux *distributions* are available, each consisting of a Linux kernel along with a set of utilities and configuration files. The result is a complete OS, and two Linux distributions can differ from each other as much as either differs from OS X or even Windows. I therefore describe in more detail what a distribution is, what distributions are popular, and the ways in which distribution maintainers keep their offerings up to date.

Creating a Complete Linux-Based OS

Certification Objective

I've already described some of what makes up a Linux OS, but some details need reiteration or elaboration:

A Linux kernel A Linux kernel is at the core of any Linux OS, of course. I've written this item as *a* Linux kernel because the Linux kernel is constantly evolving. Two distributions are likely to use slightly different kernels. Distribution maintainers also often *patch* kernels—that is, they make small changes to fix bugs or add features.

Core Unix tools Tools such as the GNU tool set, the X Window System, and the utilities used to manage disks are critical to the normal functioning of a Linux system. Most Linux distributions include more or less the same set of such tools, but as with the kernel, they can vary in versions and patches.

Supplemental software Additional software, such as major server programs, desktop environments, and productivity tools, ships with most Linux distributions. As with core Unix software, most Linux distributions provide similar options for such software. Distributions sometimes provide their own "branding," though, particularly in desktop environment graphics.

Startup scripts Much of a Linux distribution's "personality" comes from the way it manages its startup process. Linux uses scripts and utilities to launch the dozens of programs that link the computer to a network, present a login prompt, and so on. These scripts and utilities vary between distributions, which means that they have different features and may be configured in different ways.

An installer Software must be installed to be used, and most Linux distributions provide unique installation software to help you manage this important task. Thus, two distributions may install in very different ways, giving you different options for key features such as disk layouts and initial user account creation.

The UNetbootin tool (`http://unetbootin.sourceforge.net`) can copy the files from a Linux installation disc image file to a USB flash drive.

Typically, Linux distributions are available for download from their Web sites. You can usually download a CD-R or DVD image file that you can then burn to an optical disc. When you boot the resulting disc, the installer runs and you can install the OS. You can sometimes download an image that can be copied to a USB flash drive if your computer lacks an optical drive.

Some Linux installers come complete with all the software you're likely to install. Others come with only minimal software and expect you to have a working Internet connection so that the installer can download additional software. If your computer isn't connected to the Internet, be sure to get the right type of installer.

A Summary of Common Linux Distributions

Certification Objective

Depending on how you count, there are about a dozen major Linux distributions for desktop, laptop, and small server computers, and hundreds more that serve specialized purposes. Table 1.1 summarizes the features of the most important distributions.

TABLE 1.1 Features of major Linux distributions

Distribution	Availability	Package format	Release cycle	Administrator skill requirements
Arch	Free	pacman	Rolling	Expert
CentOS	Free	RPM	approximately 2-year	Intermediate
Debian	Free	Debian	2-year	Intermediate to Expert
Fedora	Free	RPM	approximately 6-month	Intermediate
Gentoo	Free	ebuild	Rolling	Expert
Mandriva	Free	RPM	1-year	Intermediate
openSUSE	Free	RPM	8-month	Intermediate
Red Hat Enterprise	Commercial	RPM	approximately 2-year	Intermediate

(Continues)

TABLE 1.1 *(Continued)*

Distribution	Availability	Package format	Release cycle	Administrator skill requirements
Slackware	Free	tarballs	Irregular	Expert
SUSE Enterprise	Commercial	RPM	2–3 years	Intermediate
Ubuntu	Free	Debian	6-month	Novice to Intermediate

These features require explanation:

Availability Most Linux distributions are entirely open source or free software; however, some include proprietary components and are sold for money, typically with a support contract. Red Hat Enterprise Linux (RHEL) and SUSE Enterprise Linux are the two most prominent examples of this type of distribution. Both have completely free cousins. For RHEL, CentOS is a near-clone that omits the proprietary components, and Fedora is an open version that serves as a testbed for technologies that may eventually be included in CentOS. For SUSE Enterprise, openSUSE is a free alternative.

Package format Most Linux distributions distribute software in *packages*, which are collections of many files in one. Package software maintains a local database of installed files, making upgrades and uninstallations easy. The RPM Package Manager (RPM) system is the most popular one in the Linux world, but Debian packages are very common, too. Other packaging systems work fine but are distribution-specific. Slackware is unusual in that it uses *tarballs* for its packages. These are package files created by the standard `tar` utility, which is used for backing up computers and for distributing source code, among other things. The tarballs that Slackware uses for its packages contain Slackware-specific information to help with package management. Gentoo is unusual because its package system is based on compiling most software from source code. This is time-consuming but enables experienced administrators to tweak compilation options to optimize the packages for their own hardware and software environments.

Tarballs are similar to the zip files that are common on Windows. Chapter 10, "Searching, Extracting, and Archiving Data," describes how to create and use tarballs.

Release cycle I describe release cycles in more detail shortly, in "Understanding Release Cycles." As a general rule, distributions with short release cycles aim to provide the latest software possible, whereas those with longer release cycles strive

to provide the most stable environments possible. Some try to have it both ways; for instance, Ubuntu releases long-term support (LTS) versions in April of even-numbered years. Its other releases aim to provide the latest software.

Don't be scared off by the "intermediate" classification of most distributions. This book's purpose is to help you manage the essential features of such distributions.

Administrator skill requirements The final column in Table 1.1 provides my personal estimation of the skill level required to administer a distribution. As you can see, I've described most Linux distributions as requiring "intermediate" skill to administer. Some, however, provide less in the way of user-friendly GUI administrative tools, and so require more skill. Ubuntu aims to be particularly easy to use and administer.

Most Linux distributions are available for at least two platforms—that is, CPU types: *x*86 (also known as IA32, i386, and several variants) and *x*86-64 (also known as AMD64, EM64T, and *x*64). Until about 2007, *x*86 computers were the most common variety, but more recently, *x*86-64 computers have become the standard. If you have an *x*86-64 computer, you can run either an *x*86 or an *x*86-64 distribution on it, although the latter provides a small speed improvement. More exotic platforms, such as PowerPC, Alpha, and SPARC, are available. Such platforms are mostly restricted to servers and to specialized devices (described shortly).

In addition to the mainstream PC distributions, several others are available that serve more specialized purposes. Some of these run on regular PCs, but others run on their own specialized hardware:

Android is best known as a cell phone OS, but it can be used on other devices. Some e-book readers, for instance, run Android.

Android Many cell phones today use a Linux-based OS known as *Android*. Its user interface is similar to that of other smart phones, but underneath lies a Linux kernel and a significant amount of the same Linux infrastructure you'll find on a PC. Such phones don't use X or typical desktop applications, though; instead, they run specialized applications for cell phones.

Network appliances Many broadband routers, print servers, and other devices you plug into a local network to perform specialized tasks run Linux. You can sometimes replace the standard OS with a customized one if you want to add features to the device. Tomato (`http://www.polarcloud.com/tomato`) and OpenWrt (`https://openwrt.org`) are two examples of such customized Linux distributions. Don't install such software on a whim, though; if done improperly, or on the wrong device, they can render the device useless!

I recommend you download Parted Magic, or a similar tool, to have on hand in case you run into problems with your main Linux installation.

TiVo This popular digital video recorder (DVR) uses a Linux kernel and a significant number of standard support programs, along with proprietary drivers and DVR software. Although many people who use them don't realize it, they are Linux-based computers under the surface.

Parted Magic This distribution, based at `http://partedmagic.com`, is a Linux distribution for PCs that's intended for emergency recovery operations. It runs

from a single CD-R and you can use it to access a Linux or Windows hard disk if the main installation won't boot.

Android, Linux-based network appliances, and TiVo are examples of *embedded systems* that use Linux. Such devices typically require little or no administrative work from users, at least not in the way such tasks are described in this book. Instead, these devices have fixed basic configurations and guided setup tools to help inexperienced users set critical basic options, such as network settings and your time zone.

Certification Objective

Understanding Release Cycles

Certification Objective

Table 1.1 summarized the release cycles employed by a number of common Linux distributions. The values cited in that table are the time between releases. For instance, new versions of Ubuntu come out every six months, like clockwork. Most other distributions' release schedules provide some "wiggle room"; if a release date slides a month, that may be acceptable.

After its release, a distribution is typically supported until sometime *after* the next version's release—typically a few months to a year or more. During this support period, the distribution's maintainers provide software updates to fix bugs and security problems. Once the support period has passed, you can continue to use a distribution, but you're on your own—if you need updated software, you'll have to compile it from source code yourself or hope that you can find a compatible binary package from some other source. As a practical matter, therefore, it's generally a good idea to upgrade to the latest version before the support period ends. This fact makes distributions with longer release cycles appealing to businesses, since a longer time between installations minimizes disruptions and costs associated with upgrades.

Two of the distributions in Table 1.1 (Arch and Gentoo) have *rolling* release cycles. Such distributions have no version numbers in the usual sense; instead, upgrades occur in an ongoing manner. Using such a distribution makes it unnecessary to ever do a full upgrade, with all the hassles that creates; however, you'll occasionally have to do a disruptive upgrade of one particular subsystem, such as a major upgrade in your desktop environment.

Certification Objective

Prior to the release of a new version, most distributions make pre-release versions available. *Alpha software* is extremely new and very likely to contain serious bugs, while *beta software* is more stable but nonetheless more likely to contain bugs than is the final release software. As a general rule, you should avoid using such software unless you want to contribute to the development effort by reporting bugs or unless you're desperate to have a new feature.

You can frequently select a product or technology on purely pragmatic grounds—what OS works well for a given task, which software suite is the least expensive, and so on. Sometimes, though, understanding the principles and philosophy that underlie a technology can be useful, and might even guide your choice. This is true of some Linux users; the open source model of Linux, which I introduced in Chapter 1, "Selecting an Operating System," has implications that can affect how Linux works. Furthermore, some people in the Linux world can become quite passionate about these principles. Whether or not you agree with these people, understanding their point of view can help you appreciate the Linux culture that you'll find in the workplace, online, at conferences, and so on.

This chapter covers these issues, beginning with information on Linux's origins and development over time up to the present. I then describe open source principles and how they can affect the way an open source OS works in the real world. Finally, I describe some of the roles in which Linux can work—as an embedded OS, as a desktop or laptop OS, and as a server OS.

- **Linux through the ages**
- **Using open source software**
- **Understanding OS roles**

Linux through the Ages

Although Linux's birth date of 1991 is recent by most historical standards, in the computer world 20 years is an eternity. Nonetheless, the software and culture in the early 1990s, and even before then, has had quite a legacy on today's software world. After all, what we use today is built atop the foundation created in the past. Thus, looking at how Linux originated will help you to understand Linux as it exists today.

Understanding Linux's Origins

Computers today can be classified in much the same way as was done in 1991, although some details have changed. A notable addition is embedded computers, as in cell phones.

In 1991, as today, computers were classified by their sizes and capabilities. Computers could belong to any of a handful of categories, ranging from desktop personal computers (PCs) to supercomputers. The PC marketplace of 1991 was dominated by *x*86-based computers that are the direct ancestors of today's PCs; however, other types of PCs were available, such as Macintoshes. Such computers generally used different CPUs and ran their own custom OSs.

In 1991, most PCs ran Microsoft's Disk Operating System (MS-DOS, PC-DOS, or DOS). DOS was extremely limited by today's standards; it was a single-tasking OS that didn't take full advantage of the memory or CPUs available at the time. The versions of Microsoft Windows available in 1991 ran on top of DOS. Although Windows helped work around some of DOS's limitations, it didn't fundamentally fix any of them. These early versions of Windows employed *cooperative multitasking*, for instance, in which programs could voluntarily give up CPU time to other processes. The DOS kernel could not wrest control from a program that hogged CPU time.

Unix was not the only multi-user, multitasking OS in 1991. Others, such as Virtual Memory System (VMS), were available. Unix is most relevant to Linux's history, though.

Above the PC level, Unix was a common OS in 1991. Compared to DOS and the Windows of that time, Unix was a sophisticated OS. Unix supported multiple accounts and provided true *preemptive multitasking*, in which the kernel could schedule CPU time for programs, even if the programs didn't voluntarily give up control. These features were practical necessities for many servers and for multi-user computers such as minicomputers and mainframes.

As time has progressed, the capabilities of each class of computer have grown. By most measures, today's PCs have the power of the minicomputers or even the mainframes of 1991. The OSs used on the PCs of 1991 don't scale well to more powerful hardware, and today's PCs are now powerful enough to run the more sophisticated OSs of 1991. For this reason, DOS and its small-computer contemporaries have been largely abandoned in favor of Unix and other alternatives.

Today's versions of Windows are not derived from DOS. Instead, they use a new kernel that shares many design features with VMS.

In 1991, Linus Torvalds was a student at the University of Helsinki, studying computer science. He was interested in learning about both Unix and the capabilities of the new *x*86 computer he'd just purchased. Torvalds began the program that would become the Linux kernel as a low-level terminal emulator—a program to connect to his university's larger computers. As his program grew, he began adding features that turned his terminal program into something that could be better described as an OS kernel. Eventually, he began writing with the goal of creating a Unix-compatible kernel—that is, a kernel that could run the wide range of Unix software that was available at the time.

Unix's history, in turn, stretched back two more decades, to its origin at AT&T in 1969. Because AT&T was at that time a telephone monopoly in the United

States, it was legally forbidden from selling software. Therefore, when its employees created Unix, AT&T basically gave the OS away. Universities were particularly enthusiastic about adopting Unix, and some began modifying it, since AT&T made the source code available. Thus, Unix had a two-decade history of open software development. Most Unix programs were distributed as source code, since Unix ran on a wide variety of hardware platforms—binary programs made for one machine would seldom run on a different machine.

Early on, Linux began to tap into this reservoir of available software. As noted in Chapter 1, early Linux developers were particularly keen on the GNU's Not Unix (GNU) project's software, so Linux quickly accumulated a collection of GNU utilities. Much of this software had been written with workstations and more powerful computers in mind, but because computer hardware kept improving, it ran fine on the *x*86 PCs of the early 1990s.

Linux quickly acquired a devoted following of developers who saw its potential to bring workstation-class software to the PC. These people worked to improve the Linux kernel, to make the necessary changes in existing Unix programs so that they would work on Linux, and to write Linux-specific support programs. By the mid-1990s, several Linux distributions existed, including some that survive today. (Slackware was released in 1993, and Red Hat in 1995, for example.)

The 386BSD OS was a competing Unix-like OS in the early 1990s. Today it has forked into several related OSs: FreeBSD, NetBSD, OpenBSD, Dragonfly BSD, and PC-BSD.

The Microkernel Debate

Linux is an example of a monolithic kernel, which is a kernel that does everything a kernel is supposed to do in one big process. In 1991, a competing kernel design, known as a microkernel, was all the rage. Microkernels are much smaller than monolithic kernels; they move as many tasks as they can into non-kernel processes and then manage the communications between processes.

Soon after Linux's release, Linus Torvalds engaged in a public debate with Andrew Tanenbaum, the creator of the Minix OS that Torvalds used as an early development platform for Linux. Minix uses a microkernel design, and Tanenbaum considered Linux's monolithic design to be backward.

As a practical matter for an end user, either design works. Linux and the BSD-derived kernels use monolithic designs, whereas modern versions of Windows, the GNU HURD, and Minix are examples of microkernels. Some people still get worked up over this distinction, though.

Seeing Today's Linux World

By the mid-1990s, the most important features of Linux as it exists today had been established. Changes since then have included:

Improvements in the kernel The Linux kernel has seen massive changes since 1991, when it lacked many of the features we rely on today. Improvements include the addition of networking features, innumerable hardware drivers, support for power management features, and support for many non-*x*86 CPUs.

Improvements in support tools Just as work has progressed on the Linux kernel, improvements have also been made to the support programs on which it relies—the compilers, shells, GUIs, and so on.

Creation of new support tools New support tools have emerged over the years. These range from simple and small utilities to big desktop environments. In fact, some of these tools, such as modern desktop environments, are far more obvious to the end user than is the kernel itself.

Creation of new distributions As noted earlier, Slackware dates to 1993 and Red Hat (the predecessor to Red Hat Enterprise Linux, CentOS, and Fedora) originated in 1995. Other distributions have emerged in the intervening years, and some have been quite important. The Android OS used on smart phones and tablets, for instance, is becoming influential in the early 2010s.

Linux's roots remain very much in the open source software of the 1980s and 1990s. Although a typical desktop or embedded OS user is likely to perceive the OS through the lens of the GUI, much of what happens under the surface happens because of the Linux kernel and open source tools, many of which have existed for decades.

Using Open Source Software

The philosophies that underlie much software development for Linux are different from those that drive most software development for Windows. These differing philosophies affect how you obtain the software, what you can do with it, and how it changes over time. Thus, I describe these principles. I also describe how Linux functions as a sort of "magnet," integrating software from many sources in one place.

Understanding Basic Open Source Principles

Broadly speaking, software can be described as coming in several different forms, each with different expectations about payment, redistribution, and users' rights.

The number of categories varies depending on the depth of analysis and the prejudices of the person doing the categorization, but as a starting point, four categories will do:

Commercial software Individuals or companies develop commercial software with the intent to sell it for a profit. Developers generally keep the source code for commercial source software secret, which means that users can't normally make changes to the software except to alter configuration settings the software supports. In the past, commercial software was sold in stores or by mail order, but today it's often sold via downloads from the Internet. Redistributing commercial software is generally illegal. Microsoft Windows and Microsoft Office are both common examples of commercial software.

Shareware software From a legal perspective, *shareware* software is similar to commercial software in that it's copyrighted and the author asks for payment. The difference is that shareware is distributed on the Internet or in other ways and "sold" on an honor system—if you use the software beyond a trial period, you're expected to pay the author. Shareware was common in 1991 and is still available today, but it's much rarer.

Freeware Freeware, like shareware, is available for free. Unlike shareware authors, though, the authors of freeware don't ask for payment. Sometimes, freeware is a stripped-down version of a more complete shareware or commercial program. Other times, the authors make it available for free to promote another product. Examples include Windows drivers for many hardware devices or the Adobe Reader program for reading Portable Document Format (PDF) files. As with commercial and shareware programs, freeware generally comes without source code.

***Freeware* should not be confused with *free software*, which is closely related to open source software. Chapter 3, "Understanding Software Licensing," describes free software in more detail.**

Open source software Open source software is defined by a set of ten principles, available at `http://www.opensource.org/docs/osd`. The most important of these principles are the right of the user to redistribute the program, the availability of source code, and the right of the user to make and distribute changed versions of the program. These principles mean that users can alter open source programs to suit their own needs, even in ways or for purposes the original author doesn't support.

Certification Objective

Variants within each of these categories exist, as well as hybrids that don't quite fit in any category. For instance, the Open Source Initiative maintains a list of licenses it has approved as fulfilling its criteria (`http://www.opensource.org/licenses`); however, developers sometimes release software using obscure licenses or using licenses that impose conditions that run afoul of one of the more obscure Open Source Initiative rules. Such software is technically not open source, but it might be closer to open source than to another category.

Chapter 3 covers specific open source licenses in greater detail.

The basic idea behind open source software is that software developed in a transparent manner is likely to be superior to software developed in a closed manner. This superiority (and arguments against it) comes in several ways:

Better code Exposing source code to the community at large means that it can be reviewed, judged, and improved upon by any interested party. Otherwise obscure bugs might be found and squashed when they might linger and cause problems in a closed-source product. On the other hand, the validity of this claim is not well supported by research, and smaller projects might not gain much in the way of interest from other programmers, so they might not benefit from outside code review.

This principle is sometimes referred to as "Linus's Law," which was stated by Eric S. Raymond in "The Cathedral and the Bazaar": "Given enough eyeballs, all bugs are shallow."

More flexibility By providing users with the source code, an open source project gives users the ability to customize the software for their own needs. If users submit changes back to the maintainer, or release them as a new branch of the project, then everybody can benefit from such changes. Of course, critics would argue that this flexibility is only a benefit to those with the necessary skill and time to make such changes, or to those with the money to hire somebody to do it.

Lower cost Although the open source definition does not forbid sale of software, the redistribution requirements mean that open source software ends up being available free of charge. On the other hand, if you want support you may need to purchase a support contract, which can reduce or eliminate the cost benefits.

Lack of vendor lock-in The developers of some proprietary products, and particularly very popular ones, can make it difficult for competing products by using proprietary file formats or standards and by not supporting more open standards. Open source tools are less subject to such problems, since they can be modified to support open standards even if they initially don't do so. As a practical matter, though, even proprietary file formats and protocols are usually reverse-engineered, so vendor lock-in usually ends up being a temporary problem rather than a permanent one.

Of course, within the Linux community the general consensus is that each of these factors is a real point in favor of Linux, and of open source software generally; the downsides noted are generally regarded as minor compared to the advantages. In the end, you'll need to make up your own mind on these matters after using different types of software.

Linux as a Software Integrator

Since soon after Unix was created, the OS fragmented into a set of loosely affiliated OSs. These OSs were incompatible on the binary level but more or less compatible on the source code level. This is still true today. You can take the same program and compile it for FreeBSD, OS X, and Linux, and it will work the same on all three platforms—but the compiled binaries made for one platform won't work on the others.

There are exceptions to this rule, though. Some programs rely on features that are available on just some Unix-like OSs. Others have quirks that make it impossible to compile them on some OSs. If a program falls into disuse, it may become unusable on newer OSs because it relies on compiler or OS features that have changed. Such problems tend to be ironed out over time, but they do crop up periodically.

Because of Linux's popularity, most open source Unix programs compile and work fine on Linux. Commercial programs for Linux also exist, although most of these are obscure or specialized. In any event, Linux has become an OS that most open source Unix programs must support. This effect is so strong that many projects now target Linux as the primary platform.

Understanding OS Roles

Computers fill many roles in the world, and as computers have become more common and less expensive, those roles have multiplied. Linux can serve as the OS for most of these roles, each of which draws on its own subset of support utilities. Some of these roles also require tweaking the kernel itself. I briefly describe three of these roles: embedded computers, desktop and laptop computers, and server computers.

Understanding Embedded Computers

Certification Objective

As noted in Chapter 1, embedded computers are specialized devices that fulfill a specific purpose. Examples include:

Cell phones Modern cell phones use computers with OSs that range from simple to complex. Linux powers some of these cell phones, usually in the form of Android.

Apple, Microsoft, and other vendors provide their own OSs for cell phones.

e-book readers These devices, like cell phones, are specialized computers and so use an OS to power them. For many current e-book readers, that OS is Linux—either a custom Linux version or Android.

DVRs Digital video recorders (DVRs), which record TV shows for later viewing, are computers with specialized software. Some of these, including the popular TiVo models, run Linux.

The MythTV package (`http://www.mythtv.org`) can turn an ordinary PC into a Linux-based DVR, although you'll need a TV tuner and other specific hardware to make it work.

Car computers Automobiles have included computers for years. These have mostly been tucked out of the way to monitor and control the engine; however, modern cars increasingly come with computers that users more readily identify as being computers. They manage global positioning system (GPS) navigation systems, control the radio, and even provide Internet access.

Appliances Televisions, refrigerators, and other appliances are increasingly using computers to monitor energy use and for other purposes.

You might also think of tablet computers as falling in this category as well, although they can more closely resemble desktop or laptop computers. The distinction is mainly one of how much control the user has over the OS; embedded devices are designed to be used, but not maintained, by end users. The system administration tasks described in this book are done at the factory or using much simpler and more specialized user interfaces.

Understanding Desktop and Laptop Computers

Certification Objective

Linux began life on a desktop computer, and although Linux doesn't come close to dominating that market, desktop computers are a good way to begin learning about Linux. Laptop computers are similar to desktop computers from a system administration perspective; both types of computers are often used by a small number of people for productivity tasks, such as word processing, Web browsing, and managing digital photos. For brevity, I'll use the term *desktop* to refer to both types of computer from here on.

Desktop computers are similar to another class of computer, known as *workstations*. Workstations tend to be more powerful and specialized, and they often run Unix or Linux.

Linux software for such tasks is widely available and is quite good, although some people prefer commercial counterparts, such as Microsoft Office or Adobe Photoshop, that aren't available for Linux. This preference for a few specific commercial products is part of why Microsoft Windows continues to dominate the desktop market. Some people have speculated that the open source development model doesn't lend itself to the creation of popular GUI applications because software developers tend to be too technically oriented to fully appreciate the needs of less technically capable users. Without an explicit way to require developers to fulfill these needs, which for-profit companies create, open source software projects lag behind their commercial counterparts in usability. This view is not

universally held, though, and at worst, open source projects lag behind their commercial counterparts just a bit.

Specific software that's required on most Linux-based desktop computers includes:

- The X Window System GUI (X for short)
- A popular desktop environment, such as GNOME, KDE, Xfce, or Unity
- A Web browser, such as Mozilla Firefox
- An email client, such as Mozilla Thunderbird or Evolution
- A graphics editor, such as the GIMP
- An office suite, such as OpenOffice.org or the similar LibreOffice

Additional requirements vary depending on the user's needs. For instance, one user might need multimedia editing tools, whereas another might need scientific data analysis software.

Linux distributions such as Fedora and Ubuntu typically install these popular desktop tools by default, or as a group by selecting a single install-time option. These distributions are also designed for relatively easy maintenance, so that users with only modest skill can install the OS and keep it running over time.

Understanding Server Computers

Certification Objective

Server computers can be almost identical to desktop computers in terms of their hardware, although servers sometimes require bigger hard disks or better network connections, depending on how they're used. Many popular network server programs were written for Unix or Linux first, making these platforms the best choice for running them. Examples include:

- Web servers, such as Apache
- Email servers, such as sendmail and Postfix
- Databases, such as MySQL
- File servers, such as the Network File System (NFS) or Samba
- Print servers, such as the Common Unix Printing System (CUPS) or Samba
- Domain Name System (DNS) servers, such as the Berkeley Internet Name Domain (BIND)
- Dynamic Host Configuration Protocol (DHCP) servers, such as the Internet Software Consortium's (ISC's) `dhcpd`

- Time servers, such as the Network Time Protocol (NTP)
- Remote login servers, such as Secure Shell (SSH) or Virtual Network Computing (VNC)

Remote login servers enable users to run desktop-style programs on a computer remotely. Therefore, they're sometimes found even on desktop systems.

In a large organization, each of these services may have a distinct associated server computer. It's possible, though, for one computer to run many of these server programs simultaneously.

Most of these servers do not require a GUI, so server computers can do without X, desktop environments, or the typical desktop programs you'll find on a desktop computer. One of Linux's advantages over Windows is that you can run the computer without these elements, and even uninstall them completely. Doing so means that the GUI won't be needlessly consuming system resources such as RAM. Furthermore, if an item such as X isn't running, any security bugs it might harbor become unimportant. Some distributions, such as Debian, Arch, and Gentoo, eschew GUI configuration utilities. This makes these distributions unfriendly to new users, but the reliance on text-mode configuration tools is not a problem to experienced administrators of server computers.

The people who maintain large server computers are generally technically quite proficient and can often contribute directly to the open source server projects they use. This close association between users and programmers can help keep server projects on the cutting edge of what's required in the real world.

Note that the distinction between desktop and server computers is not absolute; a computer can run a mixture of both types of software. For instance, you might configure desktop computers in an office environment to run file server software. This configuration enables users to more easily share their work with others in the office. In a home or small office setting, running other servers on desktop computers can obviate the need to buy specialized hardware to fulfill those roles.

The Essentials and Beyond

Linux's development history is tied to that of Unix and to open source development generally. Open source software is provided with source code and with the right to modify and redistribute the source code. This guarantees your ability to use the software even in ways the original author did not anticipate or support, provided you have the knowledge and time to alter it, or the resources to hire somebody else to do so. These open source principles have led to a great deal of popular software, particularly in the server arena; however, open source developers have been less able to capture the general public's excitement with applications designed for desktop computers.

(Continues)

2. Linux Basic Commands

IN THIS CHAPTER

- Understanding the Linux shell
- Using the shell from consoles or terminals
- Using commands
- Using command history and tab completion
- Connecting and expanding commands
- Understanding variables and aliases
- Making shell settings permanent
- Using man pages and other documentation

Before icons and windows took over computer screens, you typed commands to interact with most computers. On UNIX systems, from which Linux was derived, the program used to interpret and manage commands was referred to as the *shell*.

No matter which Linux distribution you are using, you can always count on the fact that the shell is still available to you, and Ubuntu is no exception. It provides a way to create executable script files, run programs, work with filesystems, compile computer code, and manage the computer. Although the shell is less intuitive than common graphical user interfaces (GUIs), most Linux experts consider the shell to be much more powerful than GUIs. Shells have been around a long time, and many advanced features that aren't available from the desktop can be accessed by running shell commands.

The Linux shell illustrated in this chapter is called the *Bash shell*, which stands for Bourne Again Shell. The name is derived from the fact that Bash is compatible with one of the earliest UNIX shells: the Bourne shell (named after its creator, Stephen Bourne, and represented by the `sh` command).

Although Bash is included with Ubuntu and considered a standard, other shells are available, including the C shell (`csh`), which is popular among BSD UNIX users, and the Korn shell (`ksh`), which is popular among UNIX System V users. Ubuntu uses the dash shell by default at boot time, which is designed to perform faster than the Bash shell. Linux also has a `tcsh` shell (an improved C shell) and an ash shell (another Bourne shell look-alike).

Ubuntu has more than one shell available for your use. This chapter, however, focuses primarily on the Bash shell. That is because Ubuntu uses the Bash shell by default when you open a Terminal window.

The following are a few major reasons to learn how to use the shell:

- *You will learn to get around any Linux or other UNIX-like system*. For example, with strong shell skills, you can be equally productive in just about any Linux server, home router, or even a Mac computer. You can even log in and run commands on your Android phone. They're all running Linux or similar systems on the inside.
- *Special shell features enable you to gather data input and direct data output between commands and Linux filesystems*. To save on typing, you can find, edit, and repeat commands from your shell history. Many power users hardly touch a graphical interface, doing most of their work from a shell.
- *You can gather commands into a file using programming constructs such as conditional tests, loops, and case statements to quickly perform complex operations, which would be difficult to retype over and over.* Programs consisting of commands that are stored and run from a file are referred to as *shell scripts*. Many Linux system administrators use shell scripts to automate tasks such as backing up data, monitoring log files, or checking system health.

The shell is a command language interpreter. If you have used Microsoft operating systems, you'll see that using a shell in Linux is similar to, but generally much more powerful than, the PowerShell interpreter used to run commands. You can happily use Linux from a graphical desktop interface, but as you grow into Linux you will surely need to use the shell at some point to track down a problem or administer some features.

How to use the shell isn't obvious at first, but with the right help you can quickly learn many of the most important shell features. This chapter is your guide to working with the Linux system commands, processes, and filesystem from the shell. It describes the shell environment and helps you tailor it to your needs.

About Shells and Terminal Windows

There are several ways to get to a shell interface in Linux. Three of the most common are the shell prompt, Terminal window, and virtual console, which you learn more about in the following sections.

To start, boot up your Linux system. On your screen, you should see either a graphical login screen or a plain-text login prompt similar to the following:

```
Ubuntu 18.04.3 LTS ubuntu tty1
ubuntu login:
```

In either case, you should log in with a regular user account. If you have a plain-text login prompt, continue to the next section, "Using the shell prompt." If you log in through a graphical screen, go to the section "Using a Terminal window" to see how to access a shell

from the desktop. In either case, you can access more shells as described in the section "Using virtual consoles," which appears shortly in this chapter.

Using the shell prompt

If your Linux system has no graphical user interface (or one that isn't working at the moment), you will most likely see a shell prompt after you log in. Typing commands from the shell will probably be your primary means of using the Linux system.

The default prompt for a regular user is simply a dollar sign:

```
$
```

The default prompt for the root user is a pound sign (also called a *number sign* or a *hash tag*):

```
#
```

In most Linux systems, the `$` and `#` prompts are preceded by your username, system name, and current directory name. For example, a login prompt for the user named `jake` on a computer named `pine` with `/usr/share/` as the current working directory would appear as follows:

```
[jake@pine share]$
```

You can change the prompt to display any characters you like and even read in pieces of information about your system. For example, you can use the current working directory, the date, the local computer name, or any string of characters as your prompt. To configure your prompt, see the section "Setting your prompt" later in this chapter.

Although a tremendous number of features are available with the shell, it's easy to begin by just entering a few commands. Try some of the commands shown in the remaining sections to become familiar with your current shell environment.

In the examples that follow, the dollar ($) and pound (#) symbols indicate a prompt. A `$` indicates that the command can be run by any user, but a `#` typically means that you should run the command as the root user; that is, many administrative tools require root permission to be able to run them. The prompt is followed by the command that you type (and then press Enter). The lines that follow show the output resulting from the command.

Note

Although we use `#` to indicate that a command should be run as the root user, you do not need to log in as the root user to run a command as root. In fact, the most common way to run a command as a root user is to use the `sudo` command. See Chapter 8, "Learning System Administration," for further information about the `sudo` command.

Using a Terminal window

With the desktop GUI running, you can open a terminal emulator program (sometimes referred to as a Terminal window) to start a shell. Ubuntu makes it easy for you to get to a shell from the GUI. Here are some common ways to launch a Terminal window from the desktop:

Right-click the desktop In the context menu that appears, you should see Open Terminal or something similar. Select it to start a Terminal window.

Select Terminal from the Applications screen You can search for "terminal" within the GNOME 3 Applications page.

Press Ctrl+Alt+T Ubuntu systems will respond to the Ctrl+Alt+T combination by opening a new terminal shell.

Press Alt+F2 Alt+F2 will open a command prompt into which you can type `gnome-terminal`.

The GNOME Terminal supports many features beyond the basic shell. For example, you can cut and paste text to or from a GNOME Terminal window, change fonts, set a title, choose colors or images to use as background, and set how much text to save when text scrolls off the screen.

To try some GNOME Terminal features, open a new terminal and then follow this procedure:

1. Select Edit ➪ Preferences.
2. On the General tab or current profile (depending on your version of GNOME), check the "Custom font" box.
3. With the Text tab selected and Custom font selected, try a different font and size. The new font appears in the Terminal window.
4. Return to the Preferences menu and unselect the "Custom font" box. This will restore the original font.
5. On the Colors tab, clear the "Use colors from system theme" check box. From here, you can try some different font and background colors.
6. Re-select the "Use colors from system theme" box to go back to the default colors.
7. Go to your Profile window. There are other features with which you may want to experiment, such as setting how much scrolled data is kept.
8. Close the Profile window when you are finished. You are now ready to use your Terminal window.

If you are using Linux from a graphical desktop, you will probably most often access the shell from a Terminal window.

Using virtual consoles

Most Linux systems that include a desktop interface start multiple virtual consoles running on the computer. Virtual consoles are a way to have multiple shell sessions open at once in addition to the graphical interface you are using.

You can switch between virtual consoles by holding the Ctrl and Alt keys and pressing a function key between F2 and F6. For example, pressing Ctrl+Alt+F2 will open a new virtual console session. You can return to your initial GUI session by pressing Ctrl+Alt+F1.

Try it right now. Hold down the Ctrl+Alt keys and press F3 (invoking the F keys might require additional actions on some laptop keyboards). You should see a plain-text login prompt. Log in using your username and password. Try a few commands. When you are finished, type **`exit`** to exit the shell and then press Ctrl+Alt+F1 to return to your graphical desktop interface. You can go back and forth between these consoles as much as you like.

Choosing Your Shell

In most Linux systems, your default shell is the Bash shell. To find out your default login shell, enter the following commands:

```
$ whoami
chris
$ grep chris /etc/passwd
chris:x:1000:1000:Chris,,,:/home/chris:/bin/bash
```

Notice that the command-line examples shown here and throughout the book show the command followed by output from that command. When the command completes, you are presented with the command prompt again.

The `whoami` command shows your username, and the `grep` command (replacing `chris` with your username) shows the definition of your user account in the `/etc/passwd` file. The last field in that entry shows that the Bash shell (`/bin/bash`) is your default shell (the one that starts up when you log in or open a Terminal window).

It's possible, although not likely, that you might have a different default shell set. To try a different shell, simply type the name of that shell (like **`dash`** and others, assuming that they're installed). You can try a few commands in that shell and type **`exit`** when you are finished to return to the Bash shell.

You might choose to use different shells for the following reasons:

- You are used to using UNIX System V systems (often `ksh` by default) or Sun Microsystems and other Berkeley UNIX-based distributions (frequently `csh` by default), and you are more comfortable using default shells from those environments.
- You want to run shell scripts that were created for a particular shell environment, and you need to run the shell for which they were made so that you can test or use those scripts from your current shell.

- You simply prefer features in one shell over those in another. For example, some prefer `ksh` over Bash because they don't like the way aliases are used with Bash.

Although most Linux users have a preference for one shell or another, when you know how to use one shell, you can quickly learn any of the others by occasionally referring to the shell's man page (for example, type **`man Bash`**). The man pages (described later in the section "Getting Information about Commands") provide documentation for commands, file formats, and other components in Linux. Most people use Bash just because they don't have a particular reason for using a different shell.

Bash includes features originally developed for `sh` and `ksh` shells in early UNIX systems, as well as some `csh` features. Expect Bash to be the default login shell in most Linux systems that you are using, with the exception of some specialized Linux systems (such as some that run on embedded devices) that may require a smaller shell that needs less memory and requires fewer features. Most of the examples in this chapter are based on the Bash shell.

Tip

The Bash shell is worth knowing not only because it is the default in most installations, but because it is the one you will use with most Linux certification exams.

Running Commands

The simplest way to run a command is just to type the name of the command from a shell. From your desktop, open a Terminal window. Then enter the following command:

```
$ date
Thu Jun 29 08:14:53 EDT 2019
```

Entering the `date` command, with no options or arguments, causes the current day, month, date, time, time zone, and year to be displayed as just shown.

Here are a few other commands you can try:

```
$ pwd
/home/chris
$ hostname
mydesktop
$ ls
Desktop    Downloads  Pictures  Templates
Documents  Music      Public    Videos
```

The `pwd` command shows your current working directory. Entering `hostname` shows your computer's hostname. The `ls` command lists the files and directories in your current directory. Although many commands can be run by just entering command names, it's more common to type other characters after the command to modify its behavior. The characters and words that you can type after a command are called *options* and *arguments*.

Understanding command syntax

Most commands have one or more *options* that you can add to change the command's behavior. Options typically consist of a single letter preceded by a hyphen. However, you can group single-letter options together or precede each with a hyphen to use more than one option at a time. For example, the following two uses of options for the `ls` command are the same:

```
$ ls -l -a -t
$ ls -lat
```

In both cases, the `ls` command is run with the `-l` (long listing), `-a` (show hidden dot files), and `-t` options (list by age).

Some commands include options that are represented by a whole word. To tell a command to use a whole word as an option, you typically precede it with a double hyphen (`--`). For example, to use the help option on many commands, you enter `--help` on the command line. Without the double hyphen, the letters `h`, `e`, `l`, and `p` would be interpreted as separate options. There are some commands that don't follow the double hyphen convention, using a single hyphen before a word, but most commands use double hyphens for word options.

NOTE

You can use the `--help` option with most commands to see the options and arguments that they support. For example, try typing **`hostname --help`**.

Many commands also accept arguments after certain options are entered or at the end of the entire command line. An *argument* is an extra piece of information, such as a filename, directory, username, device, or other item, that tells the command what to act on. For example, `cat /etc/passwd` displays the contents of the `/etc/passwd` file on your screen. In this case, `/etc/passwd` is the argument. Usually, you can have as many arguments as you want on the command line, limited only by the total number of characters allowed on a command line. Sometimes, an argument is associated with an option. In that case, the argument must immediately follow the option. With single-letter options, the argument typically follows after a space. For full-word options, the argument often follows an equal sign (=). Here are some examples:

```
$ ls --hide=Desktop
Documents  Music     Public    Videos
Downloads  Pictures  Templates
```

In the previous example, the `--hide` option tells the `ls` command not to display the file or directory named `Desktop` when listing the contents of the directory. Notice that the equal sign immediately follows the option (no space) and then the argument (again, no space).

Here's an example of a single-letter option that is followed by an argument:

```
$ tar -cvf backup.tar /home/chris
```

In the `tar` example just shown, the options say to create (`c`) a file (`f`) named `backup.tar` that includes all of the contents of the `/home/chris` directory and its subdirectories and show verbose messages as the backup is created (`v`). Because `backup.tar` is an argument to the `f` option, `backup.tar` must immediately follow the option. As it turns out, you don't actually need the hyphen to introduce your options (`cvf`) for the `tar` command, although it doesn't do any harm.

Here are a few commands that you can try out. See how they behave differently with different options:

```
$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
$ ls -a
.               Desktop     .gnome2_private  .lesshst       Public
..              Documents   .gnote           .local         Templates
.bash_history   Downloads   .gnupg           .mozilla       Videos
.bash_logout    .emacs      .gstreamer-0.10  Music          .xsession-errors
.bash_profile   .esd_auth   .gtk-bookmarks   Pictures       .zshrc
.bashrc         .fsync.log  .gvfs            Pictures
$ uname
Linux
$ uname -a
Linux workstation 5.3.0-28-generic #30~18.04.1-Ubuntu SMP Fri Jan 17
06:14:09 UTC 2020
x86_64 x86_64 x86_64 GNU/Linux
$ date
Thu Jun 29 08:14:53 EDT 2019
$ date +'%d/%m/%y'
06/29/19
$ date +'%A, %B %d, %Y'
Thursday, June 29, 2019
```

The `ls` command, by itself, shows all regular files and directories in the current directory. By adding the `-a`, you can also see the hidden files in the directory (those beginning with a dot). The `uname` command shows the type of system you are running (Linux). When you add `-a`, you also can see the hostname, kernel release, and kernel version.

The `date` command has some special types of options. By itself, `date` simply prints the current day, date, and time as shown in the preceding code. But the `date` command supports a special + format option, which lets you display the date in different formats. Enter **`date --help`** to see different format indicators you can use.

Try the `id` and `who` commands to get a feel for your current Linux environment, as described in the following paragraphs.

When you log in to a Linux system, Linux views you as having a particular identity, which includes your username, group name, user ID, and group ID. Linux also keeps track of your login session: It knows when you logged in, how long you have been idle, and where you logged in from.

To find out information about your identity, use the `id` command as follows:

```
$ id
uid=1000(chris) gid=1000(chris) groups=1005(sales), 7(lp)
```

In this example, the username is `chris`, which is represented by the numeric user ID (uid) `1000`. The primary group for chris also is called `chris`, which has a group ID (gid) of `1000`. It is normal for Ubuntu users to have the same primary group name as their username. The user `chris` also belongs to other groups called `sales` (gid `1005`) and `lp` (gid `7`). These names and numbers represent the permissions that `chris` has to access computer resources.

You can see information about your current login session by using the `who` command. In the following example, the `-u` option says to add information about idle time and the process ID and `-H` asks that a header be printed:

```
$ who -uH
NAME       LINE     TIME            IDLE      PID         COMMENT
chris      tty1     2020-02-17      18:53     00:38         10782
```

The output from this `who` command shows that the user `chris` is logged in on `tty1` (which is the first virtual console on the monitor connected to the computer) and his login session began at 18:53 on February 17. The `IDLE` time shows how long the shell has been open without any command being typed. `PID` shows the process ID of the user's login shell. `COMMENT` would show the name of the remote computer from which the user had logged in, if that user had logged in from another computer on the network, or the name of the local X display if that user were using a Terminal window (such as :0.0).

Locating commands

Now that you have typed a few commands, you may wonder where those commands are located and how the shell finds the commands you type. To find commands you type, the shell looks in what is referred to as your *path*. For commands that are not in your path, you can type the complete identity of the location of the command.

If you know the directory that contains the command that you want to run, one way to run it is to type the full, or absolute, path to that command. For example, you run the `date` command from the `/bin` directory by entering the following:

```
$ /bin/date
```

Of course, this can be inconvenient, especially if the command resides in a directory with a long pathname. The better way is to have commands stored in well-known directories and then add those directories to your shell's `PATH` environment variable. The path consists of a list of directories that are checked sequentially for the commands you enter. To see your current path, enter the following:

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin:/sbin:↩
/home/chris/bin
```

The results show a common default path for a regular Linux user. Directories in the path list are separated by colons. Most user commands that come with Linux are stored in the /bin, /usr/bin, or /usr/local/bin directory. The /sbin and /usr/sbin directories contain administrative commands (some Linux systems don't put those directories in regular users' paths). The last directory shown is the bin directory in the user's home directory (/home/chris/bin).

Tip

If you want to add your own commands or shell scripts, place them in the bin directory in your home directory (such as /home/chris/bin for the user named chris). This directory is automatically added to your path in some Linux systems, although you may need to create that directory or add it to your PATH on other Linux systems. So, as long as you add the command to your bin with execute permission, you can begin using it by simply typing the command name at your shell prompt. To make commands available to all users, add them to /usr/local/bin.

Unlike some other operating systems, Linux does not, by default, check the current directory for an executable before searching the path. It immediately begins searching the path, and executables in the current directory are run only if they are in the PATH variable or you give their absolute (such as /home/chris/scriptx.sh) or relative (for example, ./scriptx.sh) location.

The path directory order is important. Directories are checked from left to right. So, in this example, if there is a command called foo located in both the /usr/bin and /bin directories, the one in /usr/bin is executed. To have the other foo command run, you either type the full path to the command or change your PATH variable. (Changing your PATH and adding directories to it are described later in this chapter.)

Not all of the commands you run are located in directories in your PATH variable. Some commands are built into the shell. Other commands can be overridden by creating aliases that define any commands and options that you want the command to run. There are also ways of defining a function that consists of a stored series of commands. Here is the order in which the shell checks for the commands you type:

1. **Aliases**. These are names set by the alias command that represent a particular command and a set of options. Type **alias** to see what aliases are set. Often, aliases enable you to define a short name for a long, complicated command. (I describe how to create your own aliases later in this chapter.)
2. **Shell reserved word**. These are words reserved by the shell for special use. Many of these are words that you would use in programming-type functions, such as do, while, case, and else. (I cover some of these reserved words in Chapter 7, "Writing Simple Shell Scripts.")
3. **Function**. This is a set of commands that are executed together within the current shell.
4. **Built-in command**. This is a command built into the shell. As a result, there is no representation of the command in the filesystem. Some of the most common commands that you will use are shell built-in commands, such as cd (to change directories), echo (to output text to the screen), exit (to exit from a shell), fg (to

bring a command running in the background to the foreground), `history` (to see a list of commands that were previously run), `pwd` (to list the present working directory), `set` (to set shell options), and `type` (to show the location of a command).

5. **Filesystem command**. This command is stored in and executed from the computer's filesystem. (These are the commands that are indicated by the value of the `PATH` variable.)

To determine the location of a particular command, you can use the `type` command. (If you are using a shell other than Bash, use the `which` command instead.) For example, to find out where the Bash shell command is located, enter the following:

```
$ type bash
bash is /bin/bash
```

Try these few words with the `type` command to see other locations of commands: `which`, `case`, and `return`. If a command resides in several locations, you can add the `-a` option to have all of the known locations of the command printed. For example, the command `type -a ls` should show an aliased and filesystem location for the `ls` command.

Tip

Sometimes, you run a command and receive an error message that the command was not found or that permission to run the command was denied. If the command was not found, check that you spelled the command correctly and that it is located in your `PATH` variable. If permission to run the command was denied, the command may be in the `PATH` variable but may not be executable. Also remember that case is important, so typing CAT or Cat will not find the `cat` command.

If a command is not in your `PATH` variable, you can use the `locate` command to try to find it. Using `locate`, you can search any part of the system that is accessible to you. (Some files are only accessible to the root user.) For example, if you wanted to find the location of the `chage` command, you could enter the following:

```
$ locate chage
/snap/core/8268/usr/bin/chage
/snap/core/8268/usr/share/bash-completion/completions/chage
/snap/core/8592/usr/bin/chage
/snap/core/8592/usr/share/bash-completion/completions/chage
/snap/core18/1650/usr/bin/chage
/snap/core18/1668/usr/bin/chage
/usr/bin/chage
/usr/share/bash-completion/completions/chage
/usr/share/man/de/man1/chage.1.gz
/usr/share/man/fr/man1/chage.1.gz
/usr/share/man/it/man1/chage.1.gz
/usr/share/man/ja/man1/chage.1.gz
/usr/share/man/man1/chage.1.gz
/usr/share/man/pl/man1/chage.1.gz
```

Continues

Continued

```
/usr/share/man/ru/man1/chage.1.gz
/usr/share/man/sv/man1/chage.1.gz
/usr/share/man/tr/man1/chage.1.gz
/usr/share/man/zh_CN/man1/chage.1.gz
/var/lib/app-info/icons/ubuntu-bionic-universe/64x64/patchage_
patchage.png
```

Notice that `locate` not only found the `chage` command, it also found a variety of man pages associated with `chage` for different languages. The `locate` command looks all over your filesystem, not just in directories that contain commands. (If `locate` does not find files recently added to your system, run `updatedb` as root to update the locate database.)

In the coming chapters, you'll learn to use these and additional commands. But for now, I want you to become more familiar with how the shell itself works. So next I discuss features for recalling commands, completing commands, using variables, and creating aliases.

Recalling Commands Using Command History

Being able to repeat a command you ran earlier in a shell session can be convenient. Recalling a long and complex command line that you mistyped can save you some trouble. Fortunately, some shell features enable you to recall previous command lines, edit those lines, or complete a partially typed command line.

The *shell history* is a list of the commands that you have entered before. Using the `history` command in a Bash shell, you can view your previous commands. Then using various shell features, you can recall individual command lines from that list and change them however you please.

The rest of this section describes how to do command-line editing, how to complete parts of command lines, and how to recall and work with the history list.

Command-line editing

If you type something wrong on a command line, the Bash shell ensures that you don't have to delete the entire line and start over. Likewise, you can recall a previous command line and change the elements to make a new command.

By default, the Bash shell uses command-line editing that is based on the emacs text editor. (Type **`man emacs`** to read about it, if you care to do so.) If you are familiar with emacs, you probably already know most of the keystrokes described here.

TIP

If you prefer the `vi` command for editing shell command lines, you can easily make that happen. Add the following line to the `.bashrc` file in your home directory:

```
set -o vi
```

The next time you open a shell, you can use `vi` commands to edit your command lines.

To do the editing, you can use a combination of control keys, Meta keys, and arrow keys. For example, Ctrl+F means to hold down the Ctrl key, and type **f**. Alt+F means to hold down the Alt key, and type **f**. (Instead of the Alt key, your keyboard may use a Meta key or the Esc key. On a Windows keyboard, you can use the Windows key.)

To try out a bit of command-line editing, enter the following:

```
$ ls /usr/bin | sort -f | less
```

This command lists the contents of the `/usr/bin` directory, sorts the contents in alphabetical order (regardless of case), and pipes the output to `less`. The `less` command displays the first page of output, after which you can go through the rest of the output a line (press Enter) or a page (press spacebar) at a time. Simply press **q** when you are finished. Now, suppose that you want to change `/usr/bin` to `/bin`. You can use the following steps to change the command:

1. **Press the up arrow (↑) key**. This displays the most recent command from your shell history.
2. **Press Ctrl+A**. This moves the cursor to the beginning of the command line.
3. **Press Ctrl+F or the right arrow (→) key**. Repeat this command a few times to position the cursor under the first slash (/).
4. **Press Ctrl+D**. Type this command four times to delete `/usr` from the line.
5. **Press Enter**. This executes the command line.

As you edit a command line, at any point you can type regular characters to add those characters to the command line. The characters appear at the location of your text cursor. You can use right (→) and left (←) arrows to move the cursor from one end to the other on the command line. You can also press the up (↑) and down (↓) arrow keys to step through previous commands in the history list to select a command line for editing. (See the section "Command-line recall" for details on how to recall commands from the history list.) You can use many keystrokes to edit your command lines. Table 3.1 lists the keystrokes that you can use to move around the command line.

TABLE 3.1 Keystrokes for Navigating Command Lines

keystroke	Full Name	Meaning
Ctrl+F	Character forward	Go forward one character.
Ctrl+B	Character backward	Go backward one character.
Alt+F	Word forward	Go forward one word.
Alt+B	Word backward	Go backward one word.
Ctrl+A	Beginning of line	Go to the beginning of the current line.
Ctrl+E	End of line	Go to the end of the line.
Ctrl+L	Clear screen	Clear screen and leave line at the top of the screen.

The keystrokes in Table 3.2 can be used to edit command lines.

TABLE 3.2 Keystrokes for Editing Command Lines

Keystroke	Full Name	Meaning
Ctrl+D	Delete current	Delete the current character.
Back-space	Delete previous	Delete the previous character.
Ctrl+T	Transpose character	Switch positions of current and previous characters.
Alt+T	Transpose words	Switch positions of current and previous words.
Alt+U	Uppercase word	Change the current word to uppercase.
Alt+L	Lowercase word	Change the current word to lowercase.
Alt+C	Capitalize word	Change the current word to an initial capital.
Ctrl+V	Insert special character	Add a special character. For example, to add a Tab character, press Ctrl+V+Tab.

Use the keystrokes in Table 3.3 to cut and paste text on a command line.

TABLE 3.3 Keystrokes for Cutting and Pasting Text from within Command Lines

Keystroke	Full Name	Meaning
Ctrl+K	Cut end of line	Cut text to the end of the line.
Ctrl+U	Cut beginning of line	Cut text to the beginning of the line.
Ctrl+W	Cut previous word	Cut the word located behind the cursor.
Alt+D	Cut next word	Cut the word following the cursor.
Ctrl+Y	Paste recent text	Paste most recently cut text.
Alt+Y	Paste earlier text	Rotate back to previously cut text and paste it.
Ctrl+C	Delete whole line	Delete the entire line.

Command-line completion

To save you a few keystrokes, the Bash shell offers several different ways of completing partially typed values. To attempt to complete a value, type the first few characters and press Tab. Here are some of the values you can type partially from a Bash shell:

Command, alias, or function If the text you type begins with regular characters, the shell tries to complete the text with a command, alias, or function name.

Variable If the text you type begins with a dollar sign ($), the shell completes the text with a variable from the current shell.

Username If the text you type begins with a tilde (~), the shell completes the text with a username. As a result, *~username* indicates the home directory of the named user.

Hostname If the text you type begins with the at symbol (@), the shell completes the text with a hostname taken from the /etc/hosts file.

Tip

To add hostnames from an additional file, you can set the HOSTFILE variable to the name of that file. The file must be in the same format as /etc/hosts.

Here are a few examples of command completion. (When you see *<Tab>*, it means to press the Tab key on your keyboard.) Enter the following:

```
$ echo $OS<Tab>
$ cd ~ro<Tab>
$ userm<Tab>
```

The first example causes $OS to expand to the $OSTYPE variable. In the next example, ~ro expands to the root user's home directory (~root/). Next, userm expands to the usermod command.

Pressing Tab twice offers some wonderful possibilities. Sometimes, several possible completions for the string of characters you have entered are available. In those cases, you can check the possible ways that text can be expanded by pressing Tab twice at the point where you want to do completion.

The following shows the result you would get if you checked for possible completions on $P:

```
$ echo $P<Tab><Tab>
$PATH $PPID $PS1 $PS2 $PS4 $PWD
$ echo $P
```

In this case, there are six possible variables that begin with $P. After possibilities are displayed, the original command line returns, ready for you to complete it as you choose. For example, if you typed another P and hit Tab again, the command line would be completed with $PPID (the only unique possibility).

Command-line recall

After you type a command, the entire command line is saved in your shell's history list. The list is stored in the current shell until you exit the shell. After that, it is written to a history file, from which any command can be recalled to be run again in your next session. After a command is recalled, you can modify the command line, as described earlier.

To view your history list, use the `history` command. Enter the command without options or followed by a number to list that many of the most recent commands. For example:

```
$ history 8
 382 date
 383 ls /usr/bin | sort -a | more
 384 man sort
 385 cd /usr/local/bin
 386 man more

 387 passwd chris
 388 history 7
```

A number precedes each command line in the list. You can recall one of those commands using an exclamation point (!). Keep in mind that when an exclamation point is used, the command runs blind without presenting an opportunity to confirm the command you're referencing. There are several ways to run a command immediately from this list, including the following:

!*n* *Run command number*. Replace the *n* with the number of the command line and that line is run. For example, here's how to repeat the `date` command shown as command number 382 in the preceding history listing:

```
$ !382
date
Fri Jun 29 15:47:57 EDT 2019
```

!!—!! *Run previous command*. Runs the previous command line. Here's how you would immediately run that same `date` command:

```
$ !!
date
Fri Jun 29 15:53:27 EDT 2019
```

!?*string*—? *Run command containing string*. This runs the most recent command that contains a particular string of characters. For example, you can run the `date` command again by just searching for part of that command line as follows:

```
$ !?dat?
date
Fri Jun 29 16:04:18 EDT 2019
```

Instead of just running a `history` command line immediately, you can recall a particular line and edit it. You can use the following keys or key combinations to do that, as shown in Table 3.4.

Another way to work with your history list is to use the `fc` command. Type **`fc`** followed by a history line number, and that command line is opened in your default text editor. Make the changes that you want. When you exit the editor, the command runs. You can also give a range of line numbers (for example, `fc 100 105`). All of the commands open in your text editor and then run one after the other when you exit the editor.

TABLE 3.4 Keystrokes for Using Command History

Key(s)	Function Name	Description
Arrow keys (↑ and ↓)	Step	Press the up and down arrow keys to step through each command line in your history list to arrive at the one you want. (Ctrl+P and Ctrl+N do the same functions, respectively.)
Ctrl+R	Reverse incremental search	After you press these keys, you enter a search string to do a reverse search. As you type the string, a matching command line appears that you can run or edit.
Ctrl+S	Forward incremental search	This is the same as the preceding function but for forward search. (It may not work in all instances.)
Alt+P	Reverse search	After you press these keys, you enter a string to do a reverse search. Type a string and press Enter to see the most recent command line that includes that string.
Alt+N	Forward search	This is the same as the preceding function but for forward search. (It may not work in all instances.)

After you close your shell, the history list is stored in the `.bash_history` file in your home directory. Up to 1,000 history commands are stored for you by default.

Note

Some people disable the history feature for the root user by setting the `HISTFILE` shell variable to `/dev/null` or simply leaving `HISTSIZE` blank. This prevents information about the root user's activities from potentially being exploited. If you are an administrative user with root privileges, you may want to consider emptying your file upon exiting as well for the same reasons. Also, because shell history is stored permanently when the shell exits properly, you can prevent storing a shell's history by killing a shell. For example, to kill a shell with process ID 1234, type `kill -9 1234` from any shell.

Connecting and Expanding Commands

A truly powerful feature of the shell is the capability to redirect the input and output of commands to and from other commands and files. To allow commands to be strung together, the shell uses metacharacters. A *metacharacter* is a typed character that has special meaning to the shell for connecting commands or requesting expansion.

Metacharacters include the pipe character (|), ampersand (&), semicolon (;), right parenthesis ( )), left parenthesis (( ), less than sign (<), and greater than sign (>). The next sections describe how to use metacharacters on the command line to change how commands behave.

Piping between commands

The pipe (|) metacharacter connects the output from one command to the input of another command. This lets you have one command work on some data and then have the next command deal with the results. Here is an example of a command line that includes pipes:

```
$ cat /etc/passwd | sort | less
```

This command lists the contents of the `/etc/passwd` file and pipes the output to the `sort` command. The `sort` command takes the usernames that begin each line of the `/etc/passwd` file, sorts them alphabetically, and pipes the output to the `less` command (to page through the output).

Pipes are an excellent illustration of how UNIX, the predecessor of Linux, was created as an operating system made up of building blocks. A standard practice in UNIX was to connect utilities in different ways to get different jobs done. For example, before the days of graphical word processors, users created plain-text files that included macros to indicate formatting. To see how the document really appeared, they would use a command such as the following:

```
$ gunzip < /usr/share/man/man1/grep.1.gz | nroff -c -man | less
```

In this example, the contents of the `grep` man page (`grep.1.gz`) are directed to the `gunzip` command to be unzipped. The output from `gunzip` is piped to the `nroff` command to format the man page using the manual macro (`-man`). The output is piped to the `less` command to be displayed. Because the file being displayed is in plain text, you could have substituted any number of options to work with the text before displaying it. You could sort the contents, change or delete some of the content, or bring in text from other documents. The key is that, instead of all of those features being in one program, you get results from piping and redirecting input and output between multiple commands.

Sequential commands

Sometimes, you may want a sequence of commands to run, with one command completing before the next command begins. You can do this by typing several commands on the same command line and separating them with semicolons (;):

```
$ date ; troff -me verylargedocument | lp ; date
```

In this example, I was formatting a huge document and wanted to know how long it would take. The first command (`date`) showed the date and time before the formatting started. The `troff` command formatted the document and then piped the output to the printer. When the formatting was finished, the date and time were printed again (so I knew how long the `troff` command took to complete).

Another useful command to add to the end of a long command line is `mail`. You could add the following to the end of a command line:

```
; mail -s "Finished the long command" chris@example.com
```

Then, for example, a mail message is sent to the user you choose after the command completes.

Background commands

Some commands can take a while to complete. Sometimes, you may not want to tie up your shell waiting for a command to finish. In those cases, you can have the commands run in the background by using the ampersand (&).

Text formatting commands (such as `nroff` and `troff`, described earlier) are examples of commands that can be run in the background to format a large document. You also might want to create your own shell scripts that run in the background to check continuously for certain events to occur, such as the hard disk filling up or particular users logging in.

The following is an example of a command being run in the background:

```
$ troff -me verylargedocument | lp &
```

Don't close the shell until the process is completed or that kills the process. Other ways to manage background and foreground processes are described in Chapter 6, "Managing Running Processes."

Expanding commands

With command substitution, you can have the output of a command interpreted by the shell instead of by the command itself. In this way, you can have the standard output of a command become an argument for another command. The two forms of command substitution are `$(command)` and `` `command` `` (backticks, not single quotes).

The command in this case can include options, metacharacters, and arguments. The following is an example of using command substitution:

```
$ nano $(find /home | grep xyzzy)
```

In this example, the command substitution is done before the `nano` command is run. First, the `find` command starts at the `/home` directory and prints out all of the files and directories below that point in the filesystem. The output is piped to the `grep` command, which filters out all files except for those that include the string `xyzzy` in the filename. Finally, the `nano` command opens all filenames for editing (one at a time) that include `xyzzy`.

This particular example is useful if you want to edit a file for which you know the name but not the location. As long as the string is uncommon, you can find and open every instance of a filename existing beneath a point you choose in the filesystem. (In other words, don't use `grep` from the root filesystem or you'll match and try to edit several thousand files.)

Expanding arithmetic expressions

Sometimes, you want to pass arithmetic results to a command using *`$[expression]`*. The following is an example:

```
$ echo "I am $[2019 - 1957] years old."
I am 62 years old.
```

The shell interprets the arithmetic expression first [`2019 - 1957`] and then passes that information to the `echo` command. The `echo` command displays the text with the results of the arithmetic (`58`) inserted.

Here's an example of another way to do this:

```
$ echo "There are $(ls | wc -w) files in this directory."
There are 14 files in this directory.
```

This lists the contents of the current directory (`ls`) and runs the word count command to count the number of files found (`wc -w`). The resulting number (14, in this case) is echoed back with the rest of the sentence shown.

Expanding variables

Variables that store information within the shell can be expanded using the dollar sign (`$`) metacharacter. When you expand an environment variable on a command line, the value of the variable is printed instead of the variable name itself, as follows:

```
$ ls -l $BASH
-rwxr-xr-x. 1 root root 1219248 Oct 12 17:59 /usr/bin/bash
```

Using `$BASH` as an argument to `ls -l` causes a long listing of the Bash command to be printed.

Using Shell Variables

The shell itself stores information that may be useful to the user's shell session in what are called *variables*. Examples of variables include `$SHELL` (which identifies the shell you are using), `$PS1` (which defines your shell prompt), and `$MAIL` (which identifies the location of your user's mailbox).

You can see all variables set for your current shell by typing the `set` command. A subset of your local variables is referred to as *environment variables*. Environment variables are variables that are exported to any new shells opened from the current shell. Type **`env`** to see environment variables.

You can type **`echo $`*`VALUE`***, where ***`VALUE`*** is replaced by the name of a particular environment variable you want to list. And because there are always multiple ways to do anything in Linux, you can also type **`declare`** to get a list of the current environment variables and their values along with a list of shell functions.

Besides those that you set yourself, system files set variables that store things such as locations of configuration files, mailboxes, and path directories. They can also store values for your shell prompts, the size of your history list, and type of operating system. You can refer to the value of any of those variables by preceding it with a dollar sign (`$`) and placing it anywhere on a command line. For example:

```
$ echo $USER
chris
```

This command prints the value of the `USER` variable, which holds your username (`chris`). Substitute any other value for `USER` to print its value instead.

When you start a shell (by logging in via a virtual console or opening a Terminal window), many environment variables are already set. Table 3.5 shows some variables that are either set when you use a Bash shell or that can be set by you to use with different features.

TABLE 3.5 Common Shell Environment Variables

Variable	Description
BASH	This contains the full pathname of the `bash` command. This is usually `/bin/bash`.
BASH_VERSION	This is a number representing the current version of the `bash` command.
EUID	This is the effective user ID number of the current user. It is assigned when the shell starts, based on the user's entry in the `/etc/passwd` file.
FCEDIT	If set, this variable indicates the text editor used by the `fc` command to edit `history` commands. If this variable isn't set, the default browser is used.
HISTFILE	This is the location of your history file. It is typically located at `$HOME/.bash_history`.
HISTFILESIZE	This is the number of history entries that can be stored. After this number is reached, the oldest commands are discarded. The default value is 1,000.
HISTCMD	This returns the number of the current command in the `history` list.
HOME	This is your home directory. It is your current working directory each time you log in or type the `cd` command with any options.
HOSTTYPE	This is a value that describes the computer architecture on which the Linux system is running. For most modern PCs, the value is `x86_64`.
MAIL	This is the location of your mailbox file. The file is typically your username in the `/var/spool/mail` directory.
OLDPWD	This is the directory that was the working directory before you changed to the current working directory.
OSTYPE	This name identifies the current operating system. For Ubuntu, the `OSTYPE` value is either `linux` or `linux-gnu`, depending on the type of shell you are using. (Bash can run on other operating systems as well.)
PATH	This is the colon-separated list of directories used to find commands that you type. The default value for regular users varies for different distributions but typically includes the following: `/bin:/usr/bin:/usr/local/bin:/usr/bin/X11:/usr/X11R6/bin:~/bin`. You need to type the full path or a relative path to a command that you want to run which is not in your `PATH`. For the root user, the value also includes `/sbin`, `/usr/sbin`, and `/usr/local/sbin`.
PPID	This is the process ID of the command that started the current shell (for example, the Terminal window containing the shell).
PROMPT_COMMAND	This can be set to a command name that is run each time before your shell prompt is displayed. Setting `PROMPT_COMMAND=date` lists the current date/time before the prompt appears.

Continues

TABLE 3.5 *(continued)*

Variable	Description
PS1	This sets the value of your shell prompt. There are many items that you can read into your prompt (date, time, username, hostname, and so on). Sometimes a command requires additional prompts, which you can set with the variables PS2, PS3, and so on.
PWD	This is the directory that is assigned as your current directory. This value changes each time you change directories using the cd command.
RANDOM	Accessing this variable causes a random number to be generated. The number is between 0 and 99999.
SECONDS	This is the number of seconds since the time the shell was started.
SHLVL	This is the number of shell levels associated with the current shell session. When you log in to the shell, the SHLVL is 1. Each time you start a new Bash command (by, for example, using su to become a new user, or by simply typing bash), this number is incremented.
TMOUT	This can be set to a number representing the number of seconds the shell can be idle without receiving input. After the number of seconds is reached, the shell exits. This security feature makes it less likely for unattended shells to be accessed by unauthorized people. (This must be set in the login shell for it actually to cause the shell to log out the user.)

Creating and using aliases

Using the `alias` command, you can effectively create a shortcut to any command and options that you want to run later. You can add and list aliases with the `alias` command. Consider the following examples of using `alias` from a Bash shell:

```
$ alias p='pwd ; ls –CF'
$ alias rm='rm -i'
```

In the first example, the letter `p` is assigned to run the command `pwd` and then to run `ls -CF` to print the current working directory and list its contents in column form. The second example runs the `rm` command with the `-i` option each time you type `rm`. (This is an alias that is often set automatically for the root user. Instead of just removing files, you are prompted for each individual file removal. This prevents you from automatically removing all of the files in a directory by mistakenly typing something such as `rm *`.)

While you are in the shell, you can check which aliases are set by typing the `alias` command. If you want to remove an alias, use **`unalias`**. (Remember that if the `alias` is set in a configuration file, it will be set again when you open another shell.)

Exiting the shell

To exit the shell when you are finished, type **exit** or press Ctrl+D. If you go to the shell from a Terminal window and you are using the original shell from that window, exiting causes the Terminal window to close. If you are at a virtual console, the shell exits and returns you to a login prompt.

If you have multiple shells open from the same shell session, exiting a shell simply returns you to the shell that launched the current shell. For example, the `su` command opens a shell as a new user. Exiting from that shell simply returns you to the original shell.

Creating Your Shell Environment

You can tune your shell to help you work more efficiently. You can set aliases to create shortcuts to your favorite command lines and environment variables to store bits of information. By adding those settings to shell configuration files, you can have the settings available every time you open a shell.

Configuring your shell

Several configuration files support how your shell behaves. Some of the files are executed for every user and every shell, whereas others are specific to the user who creates the configuration file. Table 3.6 shows the files that are of interest to anyone using the Bash shell in Linux. (Notice the use of ~ in the filenames to indicate that the file is located in each user's home directory.)

TABLE 3.6 Bash Configuration Files

File	Description
`/etc/profile`	This sets up user environment information for every user. It is executed when you first log in. This file provides values for your path in addition to setting environment variables for such things as the location of your mailbox and the size of your history files. Finally, `/etc/profile` gathers shell settings from configuration files in the `/etc/profile.d` directory.
`/etc/bash.bashrc`	This executes for every user who runs the Bash shell each time a Bash shell is opened. It sets the default prompt and may add one or more aliases. Values in this file can be overridden by information in each user's `~/.bashrc` file.
`~/.profile`	This is used by each user to enter information that is specific to his or her use of the shell. It is executed only once—when the user logs in. By default, it sets a few environment variables and executes the user's `.bashrc` file. This is a good place to add environment variables because, once set, they are inherited by future shells. This file will be overruled if a `~/.bash _ profile` file exists.
`~/.bashrc`	This contains the information that is specific to your Bash shells. It is read when you log in and also each time you open a new Bash shell. This is the best location to add aliases so that your shell picks them up.
`~/.bash _ logout`	This executes each time you log out (exit the last Bash shell).

To change the /etc/profile or /etc/bashrc files, you must be the root user. It is better to create a /etc/profile.d/custom.sh file to add system-wide settings instead of editing those files directly, however. Users can change the information in the $HOME/.bash _ profile, $HOME/.bashrc, and $HOME/.bash_logout files in their own home directories.

Until you learn to use the vi editor, described in Chapter 5, "Working with Text Files," you can use a simple editor called nano to edit plain-text files. For example, enter the following to edit and add stuff to your $HOME/.bashrc file:

```
$ nano $HOME/.bashrc
```

With the file open in nano, move the cursor down to the bottom of the file (using the down arrow key). Type the line you want (for example, you could type **alias d='date +%D'**). To save the file, press Ctrl+O (the letter *O*); to quit, press Ctrl+X. The next time you log in or open a new shell, you can use the new alias (in this case, just type **d**). To make the new information you just added to the file available from the current shell right away, type the following:

```
$ source $HOME/.bashrc
$ d
06/29/19
```

The following sections provide ideas about items to add to your shell configuration files. In most cases, you add these values to the .bashrc file in your home directory. However, if you administer a system, you may want to set some of these values as defaults for all your Ubuntu system's users.

Setting your prompt

Your prompt consists of a set of characters that appear each time the shell is ready to accept a command. The PS1 environment variable sets what the prompt contains and is what you will interact with most of the time. If your shell requires additional input, it uses the values of PS2, PS3, and PS4.

When your Ubuntu system is installed, your prompt is set to include the following information: your username, your hostname, and the base name of your current working directory. That information is followed by a dollar sign (for regular users) or a pound sign (for the root user). The following is an example of that prompt:

```
chris@workstation:~/myfiles$
```

If you change directories, the myfiles name would change to the name of the new directory. Likewise, if you were to log in as a different user or to a different host, that information would change.

You can use several special characters (indicated by adding a backslash to a variety of letters) to include different information in your prompt. Special characters can be used to output your terminal number, the date, and the time as well as other pieces of information. Table 3.7 provides some examples (you can find more on the Bash man page).

Tip

If you are setting your prompt temporarily by typing at the shell, you should put the value of `PS1` in quotes. For example, you could type `export PS1="[\t \w]\$ "` to see a prompt that looks like this:

```
[20:26:32 /var/spool]$.
```

TABLE 3.7 Characters to Add Information to *bash* Prompt

Special Character	Description
\!	This shows the current command history number. This includes all previous commands stored for your username.
\#	This shows the command number of the current command. This includes only the commands for the active shell.
\$	This shows the user prompt ($) or root prompt (#), depending on which type of user you are.
\W	This shows only the current working directory base name. For example, if the current working directory was /var/spool/mail, this value simply appears as mail.
\[	This precedes a sequence of nonprinting characters. This can be used to add a terminal control sequence into the prompt for such things as changing colors, adding blink effects, or making characters bold. (Your terminal determines the exact sequences available.)
\]	This follows a sequence of nonprinting characters.
\\	This shows a backslash.
\d	This displays the day name, month, and day number of the current date, for example, Sat Jan 23.
\h	This shows the hostname of the computer running the shell.
\n	This causes a new line to occur.
\nnn	This shows the character that relates to the octal number replacing *nnn*.
\s	This displays the current shell name. For the Bash shell, the value would be bash.
\t	This prints the current time in hours, minutes, and seconds, for example, 10:14:39.
\u	This prints your current username.
\w	This displays the full path to the current working directory.

To make a change to your prompt permanent, add the value of `PS1` to your `.bashrc` file in your home directory (assuming that you are using the Bash shell). There may already be a `PS1` value in that file, which you can modify. Refer to the Bash Prompt HOWTO (`www.tldp.org/HOWTO/Bash-Prompt-HOWTO`) for information on changing colors, commands, and other features of your Bash shell prompt.

Adding environment variables

You might want to consider adding a few environment variables to your `.bashrc` file. These can help make working with the shell more efficient and effective:

TMOUT This sets how long the shell can be inactive before Bash automatically exits. The value is the number of seconds for which the shell has not received input. This can be a nice security feature, in case you leave your desk while you are still logged in to Linux. To prevent being logged off while you are working, you may want to set the value to something like `TMOUT=1800` (to allow 30 minutes of idle time). You can use any terminal session to close the current shell after a set number of seconds, for example, `TMOUT=30`.

PATH As described earlier, the `PATH` variable sets the directories that are searched for the commands that you use. If you often use directories of commands that are not in your path, you can permanently add them. To do this, add a `PATH` variable to your `.bashrc` file. For example, to add a directory called `/getstuff/bin`, add the following:

```
PATH=$PATH:/getstuff/bin ; export PATH
```

This example first reads all of the current path directories into the new PATH (`$PATH`), adds the `/getstuff/bin` directory, and then exports the new `PATH`.

CAUTION

Some people add the current directory to their PATH by adding a directory identified simply as a dot (`.`), as follows:

```
PATH=.:$PATH ; export PATH
```

This enables you to run commands in your current directory before evaluating any other command in the path (which people may be used to if they have used DOS). However, the security risk with this procedure is that you could be in a directory that contains a command that you don't intend to run from that directory. For example, a malicious person could put an `ls` command in a directory that, instead of listing the content of your directory, does something devious. Because of this, the practice of adding the dot to your path is highly discouraged.

Custom environment variables You can create your own environment variables to provide shortcuts in your work. Choose any name that is not being used and assign a useful value to it. For example, if you do lots of work with files in the `/work/time/files/info/memos` directory, you could set the following variable:

```
M=/work/time/files/info/memos ; export M
```

You could make that your current directory by typing **`cd $M`**. You could run a program from that directory called `hotdog` by typing **`$M/hotdog`**. You could edit a file from there called `bun` by typing **`vi $M/bun`**.

Getting Information about Commands

When you first start using the shell, it can be intimidating. All that you see is a prompt. How do you know which commands are available, which options they use, or how to use advanced features? Fortunately, lots of help is available. Here are some places that you can look to supplement what you learn in this chapter:

- **Check the PATH**. Type **`echo $PATH`**. You see a list of the directories containing commands that are immediately accessible to you. Listing the contents of those directories displays most standard Linux commands. For example:

```
$ ls /bin
bash            fuser       networkctl      static-sh
brltty          fusermount  nisdomainname   stty
bunzip2         getfacl     ntfs-3g         su
busybox         grep        ntfs-3g.probe   sync
bzcat           gunzip      ntfscat         systemctl
bzcmp           gzexe       ntfscluster     systemd
bzdiff          gzip        ntfscmp         systemd-ask-password
bzegrep         hciconfig   ntfsfallocate   systemd-escape
bzexe           hostname    ntfsfix         systemd-hwdb
bzfgrep         ip          ntfsinfo        systemd-inhibit
bzgrep          journalctl  ntfsls          systemd-machine-id-setup
bzip2           kbd_mode    ntfsmove        systemd-notify
bzip2recover    kill        ntfsrecover     systemd-sysusers
bzless          kmod        ntfssecaudit    systemd-tmpfiles
bzmore          less        ntfstruncate  systemd-tty-ask-password-agent
cat             lessecho    ntfsusermap     tar
chacl           lessfile    ntfswipe        tempfile
chgrp           lesskey     open            touch
chmod           lesspipe    openvt          true
chown           ln          pidof           udevadm
chvt            loadkeys    ping            ulockmgr_server
cp              login       ping4           umount
cpio            loginctl    ping6           uname
dash            lowntfs-3g  plymouth        uncompress
date            ls          ps              unicode_start
dd              lsblk       pwd             vdir
df              lsmod       rbash           wdctl
dir             mkdir       readlink        which
dmesg           mknod       red             whiptail
dnsdomainname   mktemp      rm              ypdomainname
domainname      more        rmdir           zcat
dumpkeys        mount       rnano           zcmp
echo            mountpoint  run-parts       zdiff
ed              mt          sed             zegrep
efibootdump     mt-gnu      setfacl         zfgrep
efibootmgr      mv          setfont         zforce
```

```
egrep           nano        setupcon        zgrep
false           nc          sh              zless
fgconsole       nc.openbsd  sh.distrib      zmore
fgrep           netcat      sleep           znew
findmnt         netstat     ss
```

- **Use the `help` command**. Some commands are built into the shell, so they do not appear in a directory. The `help` command lists those commands and shows options available with each of them. (Enter **`help | less`** to page through the list.) For help with a particular built-in command, enter **`help` *`command`***, replacing ***`command`*** with the name that interests you. The `help` command works with the Bash shell only.
- **Use `--help` with the command**. Many commands include a `--help` option that you can use to get information about how the command is used. For example, if you enter **`date --help | less`**, the output shows not only options, but also time formats that you can use with the `date` command. Other commands simply use a `–h` option, like `fdisk -h`.
- **Use the `info` command**. The `info` command is another tool for displaying information about commands from the shell. The `info` command can move among a hierarchy of nodes to find information about commands and other items. Not all commands have information available in the info database, but sometimes more information can be found there than on a man page.
- **Use the `man` command**. To learn more about a particular command, enter **`man` *`command`***. (Replace ***`command`*** with the command name you want.) A description of the command and its options appears on the screen.

Man pages are the most common means of getting information about commands as well as other basic components of a Linux system. Each man page falls into one of the categories listed in Table 3.8. As a regular user, you will be most interested in the man pages in section 1. As a system administrator, you will also be interested in sections 5 and 8, and occasionally section 4. Programmers will be interested in section 2 and 3 man pages.

Options to the `man` command enable you to search the man page database or display man pages on the screen. Here are some examples of man commands and options:

```
$ man -k passwd
...
passwd               (1)  - update user's authentication tokens
passwd               (5)  - password file
$ man passwd
$ man 5 passwd
```

Using the `-k` option, you can search the name and summary sections of all man pages installed on the system. There are about a dozen man pages that include "`passwd`" in the name or description of a command.

TABLE 3.8 Manual Page Sections

Section Number	Section Name	Description
1	User Commands	Commands that can be run from the shell by a regular user (typically no administrative privilege is needed)
2	System Calls	Programming functions used within an application to make calls to the kernel
3	C Library Functions	Programming functions that provide interfaces to specific programming libraries (such as those for certain graphical interfaces or other libraries that operate in user space)
4	Devices and Special Files	Filesystem nodes that represent hardware devices (such as terminals or CD drives) or software devices (such as random number generators)
5	File Formats and Conventions	Types of files (such as a graphics or word processing file) or specific configuration files (such as the `passwd` or `group` file)
6	Games	Games available on the system
7	Miscellaneous	Overviews of topics such as protocols, filesystems, character set standards, and so on
8	System Administration Tools and Daemons	Commands that require root or other administrative privileges to use

Note

If `man -k` displays no output, it may be that the man page database has not been initialized. Type **`mandb`** as root to initialize the man page database.

Let's say that the two man pages in which I am interested are the `passwd` command (in section 1 of the man pages) and the `passwd` file (in section 5) man pages. Because just typing `man passwd` displays the section 1 page, I need to request explicitly the section 5 man page if I want to see that instead (`man 5 passwd`).

While you are displaying a man page, you can view different parts of the file using Page Down and Page Up keys (to move a page at a time). Use the Enter key or up and down arrows to move a line at a time. Type a forward slash (**/**) and type a term to search the document for that term. Type **n** to repeat the search forward or **N** to repeat the search backward. To quit the man page, type **q**.

3. Users & Groups

Linux is a multi-user OS, meaning that it provides features to help multiple individuals use the computer. Collectively, these features constitute *accounts*. Previous chapters of this book have referred to accounts in passing but haven't covered them in detail. This chapter changes that; it describes important account principles and a few commands you can use to begin investigating accounts. Related to accounts are *groups*, which are collections of accounts that can be given special permissions on the computer, so this chapter also describes groups. One account, known as `root`, has special privileges on the computer. You use this account to perform most system administration tasks, so you should understand this account before you tackle the administrative tasks described in the last few chapters of this book.

- **Understanding accounts**
- **Using account tools**
- **Working as `root`**

Understanding Accounts

Even a single-user workstation uses multiple accounts. Such a computer may have just one *user account*, but several *system accounts* help keep the computer running.

Accounts enable multiple users to share a single computer without causing each other too much trouble. They also enable system administrators to track who is using system resources and, sometimes, who is doing things they shouldn't be doing. Thus, account features help users use a computer and administrators administer it. Understanding these features is the basis for enabling you to manage accounts.

Some account features help you identify accounts and the files and resources associated with them. Knowing how to use these features will help you track down account-related problems and manage users of a computer.

Understanding Account Features

Certification Objective

Most account features are defined in the `/etc/passwd` file, which consists of colon-delimited lines, with each line defining a single account. An entry might resemble the following:

```
luke:x:1003:100:Luke Jones:/home/luke:/bin/bash
```

The information contained in the fields of this line includes the following:

Username An account's username is its most salient feature to humans. Most Linux account usernames consist of lowercase letters, and occasionally numbers, as in `luke` or `thx1138`. Underscores (_) and dashes (-) are also valid characters in Linux usernames, as are dollar signs ($) at the end.

Password User accounts are normally protected by a password, which is required to log into the computer. Direct login to most system accounts is disabled, so they lack passwords. (The `root` account is an important exception; it has a password in most distributions.) The password field in the `/etc/passwd` file usually contains an `x`, which is a code meaning that the password is stored in `/etc/shadow`, as described shortly.

UID In reality, the username is just a label that the computer displays to us numerically challenged humans. The computer uses a user identification (UID) number to track accounts. UID numbers begin with 0 (which refers to the `root` account) and can range as high as 4,294,967,295 on modern Linux systems. In most distributions, user accounts are numbered 1,000 and above, with lower numbers reserved for system accounts.

Some distributions number user accounts starting at 500 rather than 1,000.

GID Accounts are tied to one or more *groups*, which are similar to accounts in many ways; however, a group is a collection of accounts. One of the primary purposes of groups is to enable users to give certain users access to their files, while preventing others from accessing them. Each account is tied directly to a primary group via a group ID (GID) number (100 in the preceding example). Accounts can be tied to other groups by inclusion in the group's definition, as described in Chapter 14, "Creating Users and Groups."

File ownership and permissions are described in Chapter 15, "Setting Ownership and Permissions."

Comment field The *comment field* normally holds the user's full name (`Luke Jones` in this example), although this field can hold other information instead of or in addition to the user's name.

Home directory User accounts, and some system accounts, have *home directories*. A home directory is an account's "home base." Normally, ownership of an account's home directory belongs to the account. Certain tools and procedures make it easy for users to access their home directories; for instance, the tilde (~) refers to a user's home directory when used at the start of a filename.

Default shell A default text-mode shell is associated with every account. In Linux, this shell is normally Bash (`/bin/bash`), but individual users can change this if they like. Most non-`root` system accounts set the default shell to `/sbin/nologin` as an added security measure—this program displays a message stating that the account is not available. Using `/bin/false` works in a similar way, although without the explanatory message.

Although you might guess by its name that `/etc/passwd` holds password information, this isn't normally the case today, although it was many years ago. For historical reasons, `/etc/passwd` must be readable by all users, so storing passwords there, even in encrypted form, is risky. Thus, passwords today are stored in another file, `/etc/shadow`, that ordinary users can't read. This file associates an encrypted password, as well as other information, with an account. This information can disable an account after a period of time or if the user doesn't change the password within a given period of time. A typical `/etc/shadow` entry looks like this:

Passwords are stored using a *hash*, which is a one-way type of encryption. When a user types a password, it's hashed, and if the hashes match, access is granted.

```
luke:$1$E/moFkeT5UnTQ:15369:0:-1:7:-1:-1:
```

The meaning of each colon-delimited field on this line is as follows:

Username Each line begins with the username. Note that the UID is *not* used in `/etc/shadow`; the username links entries in this file to those in `/etc/passwd`.

Password The password is stored in encrypted form, so it bears no obvious resemblance to the actual password. An asterisk (`*`) or exclamation mark (`!`) denotes an account with no password (that is, the account doesn't accept logins—it's locked). This is common for accounts used by the system itself.

If you've forgotten the `root` password, you can boot with an emergency disc and copy the contents of a password field for an account whose password you do remember.

Last password change The next field (`15369` in this example) is the date of the last password change. This date is stored as the number of days since January 1, 1970.

Days until a change is allowed The next field (`0` in this example) is the number of days before a password change is allowed.

Days before a change is required This field is the number of days after the last password change before another password change is required.

Days of warning before password expiration If your system is configured to expire passwords, you may set it to warn the user when an expiration date is approaching. A value of `7`, as in the preceding example, is typical.

Days between expiration and deactivation Linux allows for a gap between the expiration of an account and its complete deactivation. An expired account either can't be used or requires that the user change the password immediately

after logging in. In either case, its password remains intact. A deactivated account's password is erased, and the account can't be used until it's reactivated by the system administrator.

Expiration date This field shows the date on which the account will expire. As with the last password change date, the date is expressed as the number of days since January 1, 1970.

Special flag This field is reserved for future use and normally isn't used or contains a meaningless value. This field is empty in the preceding example.

For fields relating to day counts, a value of `-1` or `99999` typically indicates that the relevant feature has been disabled. The `/etc/shadow` values are generally best left to modification through commands such as `usermod` (described in Chapter 14, "Creating Users and Groups") and `chage`. Understanding the format of the file enables you to review its contents or to replace the `root` password by cutting-and-pasting using an emergency disc, should you forget it.

The `/etc/shadow` file is normally stored with very restrictive permissions, such as `rw-------` (600), with ownership by `root`. (Precise permissions vary from one distribution to another, though.) This fact is critical to the shadow password system's utility because it keeps non-`root` users from reading the file and obtaining the password list, even in an encrypted form. By contrast, `/etc/passwd` must be readable by ordinary users and usually has `rw-r--r--` (644) permissions. If you manually modify `/etc/shadow`, be sure it has the correct permissions when you're done.

Examples of user files stored outside of the user's home directory include e-mail in `/var/spool/mail` and temporary files in `/tmp`.

It's important to realize that an account isn't a single entity like a program binary file. Account information is scattered across several configuration files, such as `/etc/passwd`, `/etc/shadow`, `/etc/group`, and possibly in other configuration files that refer to accounts. User files reside in the user's home directory and perhaps elsewhere. Thus, managing accounts can require doing more than just editing a file or two—although in the case of system accounts, that may be enough. For this reason, various utilities exist to help create, manage, and delete accounts, as described in the rest of this chapter and in Chapter 14.

Identifying Accounts

One way to identify user accounts is to use a GUI tool for account management. Such tools vary from one distribution to another, though. One example is the User Accounts tool in the System Settings control panel on a Fedora system. Accessing this option produces a window similar to the one shown in Figure 13.1.

This tool shows only user accounts, not system accounts. It enables changing a few features, such as a user's password, by clicking on them, but its usefulness as an account management tool is limited. Ordinary users, however, can reach it fairly easily and can use it to change their passwords. As an administrator, you can use this tool to quickly verify what accounts are active.

Passwords are normally displayed as dots or asterisks as a security feature.

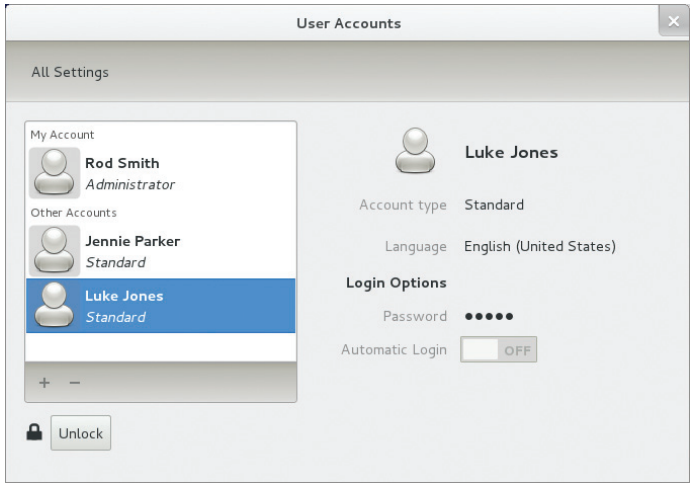


FIGURE 13.1 The User Accounts tool in System Settings provides minimal account information.

You can identify all of a computer's accounts by viewing the contents of the `/etc/passwd` file with `cat` or `less` or by loading the file in a text editor. Doing so will reveal all the accounts, including both system and user accounts.

Alternatively, if you're searching for information on a specific account, you can use `grep` to find it in `/etc/passwd`, as in **`grep Jones /etc/passwd`** to find information on any account that's tied to a user with the name Jones. (This specific example assumes that the string `Jones` appears in the `passwd` file, of course.)

An alternative that's similar to perusing `/etc/passwd` is to type **`getent passwd`**. This command retrieves entries from certain administrative databases, including the account database. In most cases, typing **`getent passwd`** produces results that are identical to typing **`cat /etc/passwd`**; however, sometimes the two aren't identical. The `/etc/passwd` file defines only local user accounts. It's possible to configure Linux to use a network account database to define some or all of its accounts. If you use such a configuration, typing **`getent passwd`** returns both local accounts and accounts defined on the network server.

Network Account Databases

Many networks employ network account databases. Such systems include the Network Information System (NIS), an update to this system called NIS+, the Lightweight Directory Access Protocol (LDAP), Kerberos realms, Windows NT 4.0 domains, and Active Directory (AD) domains. All of these systems move account database management onto a single centralized computer (often with one or more backup systems). The advantage of this approach to account maintenance is that users and administrators need not deal with maintaining accounts independently on multiple computers. A single account database can handle accounts on dozens (or even hundreds or thousands) of different computers, greatly simplifying day-to-day administrative tasks and simplifying users' lives. Using such a system, though, means that most user accounts won't appear in `/etc/passwd` and `/etc/shadow`, and groups may not appear in `/etc/group` (described shortly, in "Understanding Groups"). These files will still hold information on local system accounts and groups, though.

Linux can participate in these systems. In fact, some distributions provide options to enable such support at OS installation time. Typically, you must know the name or IP address of the server that hosts the network account database, and you must know what protocol that system uses. You may also need a password or some other protocol-specific information, and the server may need to be configured to accept accesses from the Linux system you're configuring.

Activating use of such network account databases after installing Linux is a complex topic that is not covered in this book. Such systems often alter the behavior of tools such as `passwd` and `usermod` (described in Chapter 14) in subtle or not-so-subtle ways. If you need to use such a system, you'll have to consult documentation specific to the service you intend to use. My book *Linux in a Windows World* (O'Reilly, 2005) covers this topic for Windows NT 4.0 domains, LDAP, and Kerberos; and Mark Minasi and Dan York's *Linux for Windows Administrators* (Sybex, 2002) covers this topic for Windows NT 4.0 domains and NIS.

Understanding Groups

Certification Objective

As noted earlier, groups are collections of accounts that are defined in the `/etc/group` file. Like `/etc/passwd`, `/etc/group` consists of a series of colon-delimited lines, each of which defines a single group. An example looks like this:

```
users:x:100:games,sally
```

The fields in /etc/group are:

Group name The first field, users in the preceding example, is the name of the group. You use it with most commands that access or manipulate group data.

Password Groups, like users, can have passwords. A value of x means that the password is defined elsewhere, and an empty password field means the group has no password.

GID Linux uses GID values, like UID values, internally. Translation to and from group names is done for the benefit of users and administrators.

User list You can specify users who belong to the group in a comma-delimited list at the end of the /etc/group line.

The use of group passwords is an advanced topic that's beyond the scope of this book.

It's important to recognize that users can be identified as members of a group in either of two ways:

- By specifying the group's GID in users' individual /etc/passwd entries. Because /etc/passwd only has room for one GID value, only one group can be defined in this way. This is the user's primary (or default) group.
- By specifying usernames in the user list in the /etc/group file. A single user can appear multiple times in /etc/group, and a single group can have multiple users associated with it in this way. If a user is associated with a group in this way but not via the user's /etc/passwd entry, this group association is secondary.

When you create new files, those files will be associated with your current group. When you log in, your current group is set to your primary group. If you want to create files that are associated with another group to which you belong, you can use the newgrp command, as in:

```
$ newgrp project1
```

This command makes project1 your current group, so that files you create will be associated with that group. Group ownership of files is important in file security, which is described in more detail in Chapter 15, "Setting Ownership and Permissions."

Using Account Tools

A few commands can help you learn about the users and groups on your computer. Most notably, the whoami and id utilities can tell you about your own identity, and the who and w utilities can give you information about who is currently using the computer.

Discovering Your Own Identity

If you've been using `su` to change your effective user ID, or if you maintain multiple accounts for yourself and you don't recall which one you used to log in, you might become confused about your current status. In such a case, the `whoami` command can come in handy: It displays your effective user ID:

```
$ whoami
luke
```

Certification Objective

This example reveals that the current account is `luke`. If you need more information, you can use the `id` utility:

```
$ id
uid=1003(luke) gid=100(users) groups=100(users),16(cron),18(audio)
```

This example shows information on both users and groups:

- Your user ID and username—`uid=1003(luke)` in this example
- Your current group—`gid=100(users)` in this example
- All your group memberships—the entries following `groups=` in this example

The `id` command displays both the numeric UID or GID values and the associated names. The current group is the one that's active, either by default or because you used the `newgrp` command.

You can limit `id`'s output by specifying a number of options, as summarized in Table 13.1. In addition, you can specify a username, as in **`id sally`**, to obtain information on that user rather than on yourself.

TABLE 13.1 Options for `id`

Long option	Short option	Effect
`--group`	`-g`	Displays only the effective group ID
`--groups`	`-G`	Displays all the groups to which you belong
`--user`	`-u`	Displays only the user data
`--name`	`-n`	Used in conjunction with `-g`, `-G`, or `-u`, displays only the name not the UID or GID
`--real`	`-r`	Used in conjunction with `-g`, `-G`, or `-u`, displays only the UID or GID, not the name

Learning Who's Online

Linux permits multiple users to access the computer simultaneously. Most often, this is done by means of remote access servers such as the Secure Shell (SSH); however, you can use Linux's virtual terminal (VT) feature to log in multiple times using a single keyboard and monitor. Sometimes you might want to know who is using the computer. You might do this before shutting down the computer, for instance, to ensure you don't inconvenience another user.

Certification Objective

To learn who is online, you can use a command known as `who`:

```
$ who
luke     :0           2012-03-06 13:27
luke     pts/0        2012-03-06 15:16 (:0.0)
sally    pts/7        2012-03-07 13:00 (callisto)
rod      pts/9        2012-03-07 13:22 (remote.example.org)
```

This example shows four logins—two by `luke`, one by `sally`, and one by `rod`. Information provided in the default output includes:

Username The first column of `who`'s output shows the username.

Terminal identifier The second column of `who`'s output shows a code associated with the terminal. In this example, `luke`'s first login shows `:0` as this identifier, which means it's a local GUI (X) login. The remaining logins all have terminal identifiers of the form `pts/#`, indicating text sessions. A text session can be a terminal launched in X, a text-mode console login, or a remote login via SSH or some other protocol.

Text-mode sessions can also be indicated by codes of the form `tty#`.

Login date and time `who` displays the date and time of each login. Thus, you can see that `luke`'s X session began almost one full day before `rod` logged in.

Remote host The final column of `who`'s output, if present, shows the login source. Console logins (including both text-mode and X-based logins) don't include a source. A source of the form `:#` or `:#.#`, as in `luke`'s second session, indicate a terminal opened in X. A hostname or IP address, as in `sally`'s and `rod`'s sessions, indicates remote access from the specified computer.

You can obtain additional information, most of which is obscure or specialized, by passing options to `who`. One that's more likely than others to be useful is `--count` (or `-q`), which produces a more compact summary of the data:

```
$ who -q
luke luke sally rod
# users=4
```

This output includes just the usernames and a line specifying the total number of sessions. (That number counts one user with multiple logins multiple times.) For information on additional `who` options, consult its `man` page.

Certification Objective

An alternative to `who` is `w`, which is similar to `who` but produces somewhat more verbose output:

```
$ w
 12:36:40 up 37 days, 45 min,  3 users,  load average: 0.00, 0.02, 0.05
USER   TTY     LOGIN@   IDLE   JCPU   PCPU WHAT
luke   :0      06Mar12 40:23   0.01s  0.01s -bash
luke   pts/0   06Mar12 40.00s  0.01s  0.01s nano
sally  pts/7   13:00   20:37m  0.00s  0.00s -bash
rod    pts/9   13:22   2:50   12:44m  0.00s /bin/sh /etc/xdg/xfce4/xinitrc
```

As you can see, `w` displays much of the same information as `who`, including the terminal identifier (TTY) and login time (in a different format). In addition, `w` displays some further information:

- The session's idle time tells you how long it's been since the user has interacted with this session. This information can help you identify sessions that the user may have abandoned.
- The `JCPU` column identifies the total amount of CPU time associated with the session. This can be useful debugging information if the computer has become sluggish because of out-of-control processes.
- The `PCPU` column identifies the total amount of CPU time associated with the current process running in the session. Again, this information can help you track down out-of-control processes.
- The `WHAT` column tells you what program the session is running.

Some configurations also display a `FROM` column, which shows a remote hostname. Using the `-f` option toggles this option on or off. A few other options can eliminate or modify `w`'s output. Consult the program's `man` page for details.

Working as *root*

Linux is modeled after Unix, which was designed as a multi-user OS. In principle, you can have thousands of accounts on a single Unix (or Linux) computer. One user, though, needs extraordinary power in order to manage the features of the computer as a whole. This is the `root` user, also known as the *superuser* or the *administrator*. Knowing why `root` exists, how to do things as `root`, and how to use `root` privileges safely is important for managing a Linux system.

Why Work as *root*?

Certification Objective

Most people use computers to do ordinary day-to-day computer tasks—browse the Web, write letters, manage a music collection, and so on. These activities are known collectively as *user tasks*, and they don't require special privileges. As just noted, a Linux computer can have many user accounts, and the users can use the computer from these user accounts (also known as *unprivileged accounts* or *unprivileged users*) to perform such user tasks.

Certification Objective

The `root` account, on the other hand, exists to enable you to perform *administrative tasks*. These tasks include installing new software, preparing a new disk for use in the computer, and managing ordinary user accounts. Such tasks require access to system files that ordinary users need not modify, or sometimes even read.

To facilitate performing these tasks, `root` can read and write every file on the computer. Since Linux relies on files to store system settings, this effectively gives `root` the power to change any detail of the OS's operation, which is the point of having a superuser account. If the computer is a workstation that's used by just one individual, you might wonder why the distinction between `root` and the user account is necessary. The explanation is that the power of the `root` account can lead to accidental damage. For instance, take the `rm` command. If you mistype an `rm` command as an ordinary user, you can accidentally delete your own files but not system files. Make the same mistake as `root`, however, and you can delete system files, perhaps making the computer unbootable. Thus, you should be cautious when using the `root` account—a topic I'll come back to shortly, in "Using `root` Privileges Safely."

Acquiring *root* Privileges

When you need to perform a command-line task that requires `root` privileges, you can do so in any of three ways:

Log in as `root` You can log in directly as `root` at a text-mode shell or by using a remote login tool such as SSH. You can even log into GUI mode as `root` on some Linux distributions, although some distributions disallow this by default because it's very dangerous.

Certification Objective

Use `su` The `su` command enables you to change your identity within a shell. Type **`su`** ***`username`*** to change your identity to that of the specified *`username`*. If you omit *`username`*, `root` is assumed, so typing **`su`** enables you to effectively become `root`. You must, however, know the password for the target account for

this command to work. After you acquire root privileges in this way, you can type as many commands as root as you like. When you're done, type **exit** to relinquish your superuser status. You can also use su to run a single command as root by using the -c option, as in **su -c *command*** to run *command* as root. If you use a dash (-) within the command, as in **su -** or **su - luke**, the program opens a login session, which runs the target user's login scripts. This can be important because these scripts often set environment variables that can be important for that user, such as $PATH.

su stands for *switch user* or *substitute user*.

Certification Objective

Use sudo The sudo command is similar to su, but it works for just one command at a time, which you type after sudo, similar to using su -c. For instance, typing **sudo cat /etc/shadow** enables you to see the contents of the /etc/shadow file, which is not readable by ordinary users. You must type either your own password or the root password, depending on the sudo configuration, when you use this program. (When using su -c, you must always type the root password.) The next command you type will be executed using your ordinary account privileges. By default, most distributions permit only users who are members of the wheel group to use sudo. Some distributions, such as Ubuntu, rely heavily on sudo and don't permit direct root logins by default.

If you acquire root privileges by logging in directly as root or by using su, your shell prompt will change:

```
[luke@wembleth ~]$ su
Password:
[root@wembleth luke]#
```

To use root commands in Ubuntu, you must either precede them with sudo (as in sudo cat /etc/shadow) or type sudo su to acquire a longer-lasting root shell.

In this example, the username has changed from luke to root; the directory has changed from ~ to luke (since for root, ~ refers to /root, not /home/luke); and the last character of the prompt has changed from a dollar sign ($) to a hash mark (#). Because I use just the last character of the prompt for most examples printed on their own lines in this book, such examples implicitly specify whether a command requires root privileges by the prompt used. For instance, consider accessing the /etc/shadow file mentioned earlier:

```
# cat /etc/shadow
```

The use of the hash mark prompt indicates that you *must* type this command as root.

Some of this book's chapters describe both GUI and text-mode methods of system administration. How, then, can you administer Linux in GUI mode if you must use the text-mode su (or sudo) command to acquire root privileges? One way is to launch a GUI administrative tool from a shell in which you've already used su. This approach works but is a bit inelegant if you're more comfortable

with a GUI than with a command line. Most distributions, therefore, provide an alternative: You can launch administrative tools from the computer's desktop menus and the GUI tools will then prompt you for the `root` password, as shown in Figure 13.2. If you type the password correctly, the program will launch. The result is similar to that of launching the program from a shell using `sudo`—it runs as `root`, but without making anything else run as `root`.

If you're a member of the `wheel` group, most distributions prompt you for your own password rather than the `root` password when using such tools.

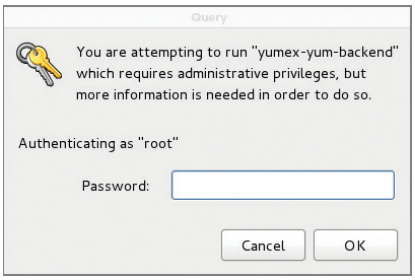


FIGURE 13.2 When you launch an administrative tool from a GUI, Linux asks you for a password.

SETTING THE *ROOT* PASSWORD

If you're using your own computer, you may already know or have the `root` password, since most distributions prompt you to enter the `root` password when you install the system. Some distributions, such as openSUSE, set the `root` password to be identical to the first user account's password by default. Ubuntu doesn't set a `root` password at all; instead, Ubuntu's security model favors the use of `sudo` or GUI equivalents.

If you've forgotten the `root` password, you can recover or reset it in several ways, such as:

- You can type **`sudo passwd root`** to reset the `root` password. As described in Chapter 14, `passwd` is the tool to set an account's password, and so this command will reset the `root` password. Used in this way, `sudo` will prompt you for a password (you should type your user password) followed by two prompts for a new `root` password. This method will work, however, only if you're an authorized user of `sudo` on your system.

(Continues)

SETTING THE *ROOT* PASSWORD *(Continued)*

- Reboot into an emergency system, such as Parted Magic (`http://partedmagic.com`), and mount your regular installation's root filesystem. You can then locate its `/etc/shadow` file (it will be located at another mount point, such as `/mnt/local/etc/shadow`) and load it into a text editor. Locate the entry for an account whose password you know and copy the second colon-delimited entry from that line into the equivalent space for the `root` account. Passwords are stored in an encrypted form, so you won't recognize the password you copy and paste in this way. After you save your changes, reboot and log into the `root` account. Once you've logged in, change the `root` password to something unique.

If you're using a school's or employer's computer, you should either refrain from attempting to perform tasks as `root` or obtain the `root` password through authorized channels. If you've been hired as a system administrator, your employer should provide this information, or specify another way to perform maintenance (such as using `sudo`). If you aren't the computer's official administrator, attempting to perform `root` tasks on it may be grounds for dismissal. In fact, you might even be sued or imprisoned for breaking into a computer without authorization!

Using *root* Privileges Safely

Note that emergency systems, such as Parted Magic, typically run all your commands as `root`, so be careful when using them!

As already described, `root` power is dangerous. For instance, preparing a disk requires creating a filesystem on the disk. This topic is not covered in detail in this book, but you should know that you can create a filesystem with `mkfs`, which takes a partition identifier as an option. Suppose that you want to create a new filesystem on `/dev/sda3`, so you quickly type a command:

```
# mkfs /dev/sda2
```

Did you catch the error? The command specifies `/dev/sda2` rather than `/dev/sda3`. If `/dev/sda2` held important data (such as your Linux installation or your `/home` directory), this mistake could be absolutely disastrous.

This is just one example of the trouble that improper use of `root` can cause. Others include intruders gaining `root` access to your computer; unintended changes to configuration files; damage to some (even if not all) of the computer's system files; and changes to ownership or permissions on ordinary user

files, rendering them inaccessible to their true owners. Thus, I recommend you take the following precautions whenever you need `root` access:

- Ask yourself if you really need `root` access. Sometimes there's a way to achieve a goal without superuser privileges or by using those privileges in a more limited way than you'd originally planned. For instance, you might find that only `root` can write to a removable disk. Such a problem can usually be overcome by adjusting permissions on the disk in one way or another, thus limiting the use of `root`.
- Before pressing the Enter key after typing any command as `root` (or clicking any confirmation button in a GUI program running as `root`), take your hands *off* the keyboard and mouse, look over the command, and verify that it's correct in every respect. A simple typo can cause a world of pain. This step has saved me from disaster more than once!
- Never run a suspicious program as `root`. On multi-user systems, unscrupulous users can try to trick administrators into running programs that will do nasty things or give the attacker `root` privileges. Programs downloaded from random Internet sites could in principle be designed to compromise your security, and such programs are much more dangerous when run as `root`.

If a program asks for your or the `root` password and it's not an administrative program you trust, be suspicious! Research the program before giving it your password!

- Use `root` privileges for as brief a period as possible. If you need to type just one or two commands as `root`, do so and then type **`exit`** in the `root` shell to log out or return to your normal privileges. Alternatively, use `sudo` to run the commands. It's easy to overlook the fact that you're using a `root` shell and therefore type commands as `root` that don't need that privilege. Every command typed as `root` is a risk.
- Never leave a `root` shell accessible to others. If you're performing `root` maintenance tasks and are called away, type **`exit`** in your `root` shell before leaving the computer.
- Be careful with the `root` password. Don't share the password with others, and be cautious about typing it in a public area or when others might be looking over your shoulder. If you're using Linux professionally, your employer may have guidelines concerning who may have `root` access to a computer. Learn and obey those rules. Be sure to select a strong `root` password, too.

Chapter 14 describes how to select a strong password.

Following these rules of thumb can help keep you from damaging your computer or giving somebody else `root` access to the computer.

Linux is a multi-user OS, meaning that a single Linux computer can support many users, each with a unique account. With this capability comes the need to manage users' accounts, and this chapter covers the procedures you'll use to do so. The chapter begins with information on how to create accounts. With accounts created, I then show you how to modify those accounts and, when necessary, delete them. Finally, groups are similar to accounts in many ways, so I also describe how to create and manage groups.

- **Creating new accounts**
- **Modifying accounts**
- **Deleting accounts**
- **Managing groups**

Creating New Accounts

In many environments, the task of adding accounts is quite common. Large businesses frequently hire new employees, universities see new students enrolling, and so on. Thus, you must know how to create new accounts. Before covering the mechanics of creating accounts, I describe the important issues of deciding how to use groups and selecting a good password. I then explain how to create accounts using both GUI and text-mode tools.

Deciding on a Group Strategy

As described in Chapter 13, "Understanding Users and Groups," Linux groups are collections of users. You can use groups to control who can access particular files. As will be described in Chapter 15, "Setting Ownership and Permissions," individuals can change the group affiliations and group permissions of their

own files. Thus, the way you use groups can influence your computer's overall security strategy. Two approaches are common:

User groups Each user can have an associated group; for instance, the user `luke` can have a group called `luke`. This user can then set group ownership on his files to `luke` or set group permissions to whatever is desired, and the system administrator can add users to the `luke` group. Thereafter, members of the `luke` group may access files in this group using the permissions determined by the user `luke`. This approach emphasizes controlling access to individual users' files.

By default, some distributions employ a user groups strategy and others use a project groups strategy. In the latter case, most users are in a group called `users` by default.

Project groups In this method, you create groups based on work projects, departmental affiliations, or other real-world groupings of users. For instance, you might have a group called `sales` for users in the sales department. Members of this group who want to share files with others in this group would assign group ownership and permissions appropriately and store files in an agreed-upon location. This approach works best when the computer is used by a large number of people who collaborate in easily defined groups.

These two approaches are not mutually exclusive; you can mix and match, or create your own approach. You should also realize that users can be members of multiple groups. In fact, this is required for the user groups approach to work at all—otherwise, groups are redundant with accounts.

If you use the project group approach, you should think about which group should be a new user's primary group. This is the group that will be assigned group ownership of the user's files by default.

Selecting a Good Password

Be sure to follow the advice in this section yourself, especially for the `root` password!

When you create an account, you should normally create a password for it. Sometimes, the user can type the password when the account is created. Other times, though, you'll need to select a password the user should then use. In such cases, you should instruct the user to change the password as soon as possible. In either case, you should educate users about how to select a good password.

Poor but common passwords include those based on the following:

- The names of family members, friends, and pets
- Favorite books, movies, television shows, or the characters in any of these
- Telephone numbers, street addresses, or Social Security numbers
- Any other meaningful personal information

- Any single word that's found in a dictionary (in *any* language)
- Any simple keyboard or alphanumeric combination, such as `qwerty` or `123456`

The best possible passwords are random collections of letters, digits, and punctuation. Unfortunately, such passwords are difficult to remember. A reasonable compromise is to build a password in two steps:

1. Choose a base that's easy to remember but difficult to guess.
2. Modify that base in ways that increase the difficulty of guessing the password.

One approach to building a base is to use two or more *unrelated* words, such as *bun* and *pen*. You can then merge these two words (`bunpen`). Another approach, and one that's arguably better than the first, is to use the first letters of a phrase that's meaningful to the user. For instance, the first letters of "yesterday I went to the dentist" become `yiwttd`. In both cases, the base should not be a word in any language. As a general rule, the longer the password, the better.

Many distributions place lower limits on password length, such as 6 or 8 characters.

With the base in hand, it's time to modify it to create a password. The user should apply at least a couple of several possible modifications:

Adding numbers or punctuation One important change is to insert random numbers or punctuation in the base. This step might yield, for instance, `bu3npe&n` or `y#i9wttd`. As a general rule, add at least two symbols or numbers.

Mixing case Linux uses case-sensitive passwords, so jumbling the case of letters can improve security. Applying this rule might produce `Bu3nPE&n` and `y#i9WttD`, for instance.

Reversing order A change that's very weak by itself but that can add somewhat to security when used in conjunction with the others is to reverse the order of some or all letters. You might apply this to just one word of a two-word base. This could yield `Bu3nn&EP` and `DttW9i#y`, for instance.

Grow the haystack A would-be intruder's task of discovering a password has been likened to finding a needle in a haystack. One way to make this task harder is to increase the size of the haystack. In password terms, this means making a password larger. You can do this by using larger words or phrases, of course, but this can make a password harder to remember and type. Even a size increase that simply repeats a single character can be helpful. Thus, you might turn the passwords into `Bu3nn&EPiiiiiiiiii` or `Dtt::::::::::W9i#y`.

Your best tool for getting users to pick good passwords is to educate them. Here are some insights to share with users:

- Passwords can be guessed by malicious individuals who know them or even who target them and look up personal information in telephone books, on Web pages, and so on.
- Although Linux encrypts its passwords internally, programs exist that feed entire dictionaries through Linux's password encryption algorithms for comparison to encrypted passwords. If a match is found, the password has been found.
- User accounts might be used as a first step toward compromising the entire computer or as a launching point for attacks on other computers.
- Users should *never* reveal their passwords to others, even people claiming to be system administrators—this is a common scam, but real system administrators don't need users' passwords.
- The same password should not be used on multiple systems because doing so quickly turns a compromised account on one computer into a compromised account on all of them.
- Writing passwords down or emailing them are both risky practices. Writing a password on a sticky note stuck to the computer's monitor is particularly risky.

Spam e-mails sometimes try to trick users into revealing passwords by claiming that an email, banking, or other account has been deactivated or compromised.

Telling your users these things will help them understand the reasons for your concern, and it's likely to help motivate at least some of them to pick good passwords.

Don't Use These Passwords!

If you do a Web search on *worst passwords* or a similar phrase, you'll quickly discover Web sites that list the most common and most easily guessed passwords that security researchers have uncovered. Details vary from one survey to another, but in one 2011 study (`http://www.splashdata.com/press/PR111121.htm`), the top 10 most common passwords were:

1. `password`
2. `123456`
3. `12345678`

(Continues)

Don't Use These Passwords! *(Continued)*

4. `qwerty`
5. `abc123`
6. `monkey`
7. `1234567`
8. `letmein`
9. `trustno1`
10. `dragon`

Such passwords are easily discovered by brute-force password-guessing programs and are included in collections of passwords distributed on the Internet. Using such a password is barely better than using no password at all. Do yourself a favor and use a better one!

Creating Accounts Using GUI Tools

Now that you have some idea of what type of group policy you want to use and how to create a good password, you can begin creating accounts. The easiest way to do this is with the help of a GUI tool; however, such tools vary from one Linux distribution to another. The most important variation is in how you access these tools. For instance:

- In Fedora 16, you can type **`system-config-users`** as `root` at a shell prompt to launch the User Manager utility. Alternatively, you can use a more limited System Settings utility similar to the one in Ubuntu.
- In openSUSE 12.1, you can launch the YaST configuration tool and open the User and Group Management tool within YaST.
- In Ubuntu 11.10, you can open the System Settings utility in the desktop environment and then click on the User Accounts option. Before you make changes, you must click the padlock icon and enter your password.

As an example of a GUI user management tool, Figure 14.1 shows Fedora's User Manager utility, which provides more features than the User Accounts utility that's part of the System Settings tool. You'll be better able to adjust account options with a more complete tool such as User Manager than with a simple tool such as User Accounts.

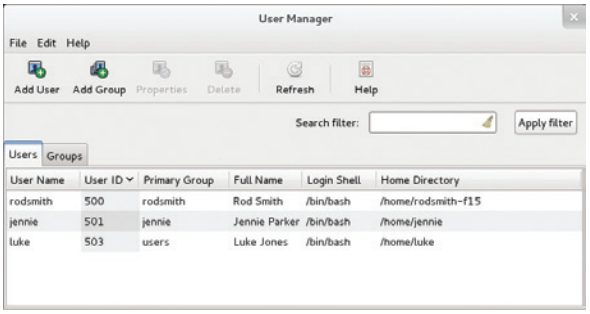


FIGURE 14.1 **The User Manager utility provides many options for creating and managing accounts.**

To add a user with User Manager, follow these steps:

1. Click Add User. The result is the Add New User dialog box shown in Figure 14.2.

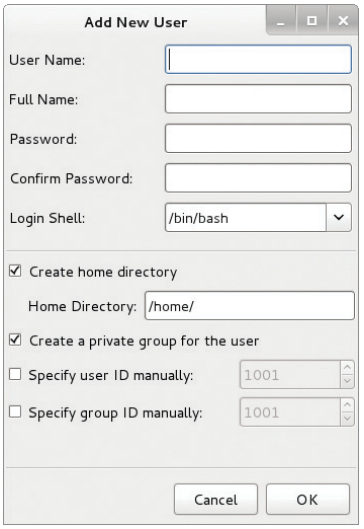


FIGURE 14.2 **You can enter all of the basic account information with the Add New User dialog box.**

2. Type the username in the User Name field. This is what the user will type at Linux login prompts.
3. Type the user's full name in the Full Name field. This entry is stored in the comment field of `/etc/passwd` and is displayed in various tools; for instance, it appears in the upper-right corner of the screen when the user logs into GNOME.

4. Type the password twice, once in the Password field and again in the Confirm Password field.
5. In most cases, you can click OK, since the default values for the remaining options are usually fine. (The Home Directory field will be filled in based on the username you enter.) You can, however, customize the login shell, home directory location, and other features before clicking OK.

If the user is present when you create the account, you can have the user type the password at this point.

The new account will appear in the User Manager list. You can subsequently modify it or delete it, as described later in this chapter, should the need arise.

Creating Accounts from the Shell

Certification Objective

With any distribution, you use the `useradd` utility to create an account from the command line. To use this utility, you type its name and the username you want to associate with a new account. You may also include options between `useradd` and the username, as summarized in Table 14.1.

You can call `useradd` as `adduser` if you prefer; both command names are equivalent.

TABLE 14.1 Options for `useradd`

Option name	Option abbreviation	Effect
`--comment` *comment*	`-c`	This parameter specifies the comment field for the user. (GUI tools generally describe this as the "full name.")
`--home` *home-dir*	`-d`	You specify the account's home directory with this parameter. It defaults to `/home/`*username*.
`--expiredate` *expire-date*	`-e`	You set the date on which the account will be disabled, expressed in the form *YYYY-MM-DD*, with this option. The default is for an account that does not expire.
`--inactive` *inactive-days*	`-f`	This parameter sets the number of days after a password expires, after which the account becomes completely disabled. A value of `-1` disables this feature and is the default.

(Continues)

TABLE 14.1 *(Continued)*

Option name	Option abbreviation	Effect
`--gid` *`default-group`*	`-g`	You set the name or GID of the user's default group with this option. The default for this value is a new group named after the user.
`--groups` *`group`*`[,...]`	`-G`	This parameter sets the names or GIDs of groups to which the user belongs. More than one may be specified by separating them with commas.
`--create-home`	`-m`	When this option is included, `useradd` creates a home directory for the user. This option is enabled by default.
`--skel` *`skeleton-dir`*	`-k`	Normally, default user configuration files are copied from `/etc/skel`, but you may specify another template directory with this option, which is valid only in conjunction with `-m`.
None	`-M`	This option forces the system to *not* automatically create a home directory.
`--shell` *`shell`*	`-s`	You set the name of the user's default login shell with this option. The default is `/bin/bash`.
`--uid` *`UID`*	`-u`	This parameter creates an account with the specified user ID value (*`UID`*).
`--non-unique`	`-o`	This parameter enables a single UID number to be reused; this option is passed when creating the second or subsequent account that reuses a UID.

(Continues)

TABLE 14.1 *(Continued)*

Option name	Option abbreviation	Effect
`--system`	`-r`	This parameter specifies the creation of a system account. `useradd` doesn't create a home directory for system accounts, and gives them UID values below 100.
`--no-user-group`	`-N`	This option disables creation of a group for the user.

The `useradd` command supports additional options; consult its `man` page for details.

Some of these options aren't readily accessible when creating accounts using GUI tools, but the details differ from one GUI utility to another. In some cases, options can be set in a GUI utility *after* the account has been created, but not when creating it.

A complete `useradd` command, including setting a few options, looks like this:

```
# useradd -e 2012-12-31 -u 1012 pamela
```

You might want to specify a UID to keep these values synchronized across computers that share files with the Network File System (NFS), which identifies file ownership via UIDs.

This example creates an account with a username `pamela` and a UID of 1012. The account expires on December 31, 2012. After that date, the user will be unable to log into the account, although her files won't be deleted, and `root` will be able to reactivate the account.

When you create an account with `useradd`, it will be in a *locked* state—the user will not be able to log in. To unlock it, you must use the `passwd` command, as described next, in "Modifying Accounts."

Behind the scenes, `useradd` (or by extension its GUI front-ends) modifies the contents of `/etc/passwd`, `/etc/shadow`, and `/etc/group`, all of which are described in detail in Chapter 13, "Understanding Users and Groups." If you use `--create-home` or `-m`, or if this option is used by default, the program creates a home directory and copies files from `/etc/skel` to that location. Creating an account will also usually create a *mail spool file*, in which the user's incoming e-mail will be stored. (This file may go unused on many desktop systems, but it can be very important if you run mail server software on the computer.) Thus, you can see that `useradd` makes quite a few modifications to your computer's files and directories in creating the account.

Modifying Accounts

As you've just learned, you can specify many options that affect accounts, such as giving an account a specific UID number, when creating an account. Sometimes, though, it's necessary to change account options after an account has been created. Fortunately, Linux provides both GUI and text-mode tools to help you do this. Before delving into operational details of these tools, though, you should understand when you might want to make changes to accounts, and know how to check whether a user is currently logged in.

Deciding When to Modify Accounts

In an ideal world, you'll create your accounts perfectly every time; however, sometimes this isn't possible. You might lack information that's necessary to create a perfectly tuned account (such as the length of time an employee will be with a company), or your needs might change after the account has been created. Some common specific causes of account changes include (but are by no means limited to) the following:

- A user might forget a password. The system administrator can change the password for any account without knowing the original password, so system administrators frequently have to help out users with faulty memories.
- Group affiliations may need to change because of restructuring of collaborative arrangements.
- Account expiration data may need to be updated. A student might enroll for another semester, for instance. Sometimes an expired account must be re-enabled.
- UID numbers may need to be synchronized with other computers, in order to facilitate file sharing across computers or for other reasons.
- On occasion a username may need to be changed—for instance, if a user marries and your site policy uses last names in the username.
- Users' home directories might change because you've added disk space and need to move some users' home directories to a new location.

When working in a GUI, all of the preceding changes can be handled from User Manager. When working in a text-mode shell, though, you'll need to master a few different programs to handle this range of account modifications.

Checking for Logged-in Users

Be aware that some account changes are potentially quite disruptive if the user is logged in at the moment you perform them. Changing the username and home directory, in particular, are likely to cause problems. Thus, you should make such changes only when the user is logged out. Several tools can help you check who's using the computer, and thus avoid problems:

who This utility, described in Chapter 13, produces a list of users who are currently logged into the computer, along with some details of their login sessions, such as their terminal identifiers and login dates.

w This command is similar to `who` in broad strokes, but it provides different details. Most notably, it identifies the program that's currently running in each session.

`last` This program produces a list of recent login sessions, including their starting and ending times, or a notice that the user is still logged in:

Certification Objective

```
$ last
jennie   tty3         Sun Jan  8 12:43   still logged in
jennie   tty3         Sun Jan  8 12:43 - 12:43  (00:00)
jennie   tty2         Sun Jan  8 12:36   still logged in
jennie   tty2         Sun Jan  8 12:36 - 12:36  (00:00)
```

One notable limitation of `last` is that it includes only text-mode logins. This makes its utility for identifying users who are currently using the computer rather limited, since such users are likely to be logged in using a GUI session.

Modifying Accounts Using GUI Tools

As with adding accounts, the procedure for modifying accounts using GUI tools varies from one tool to another. Most GUI tools provide similar options, although some are more complete than others. In this section, I describe how to modify accounts using Fedora's User Manager, which you can launch by typing **`system-config-users`** in a terminal window.

To make such changes, you should click the account and then either select File ➢ Properties from the menu or click the Properties icon in the toolbar. The result is a dialog box similar to the one shown in Figure 14.3. This dialog box presents account properties. Each of the four tabs provides access to particular types of data:

User Data As shown in Figure 14.3, you can adjust the account's username, the comment field (identified as Full Name), the password, the home directory,

If you launch User Manager as an ordinary user, you'll have to authenticate yourself by typing your or the `root` password, depending on your configuration.

The User Manager program does *not* rename or move the user's home directory on the disk; you must do this manually with `mv`.

and the user's login shell. If you type a weak password, the program warns you of this fact and asks for confirmation before accepting it.

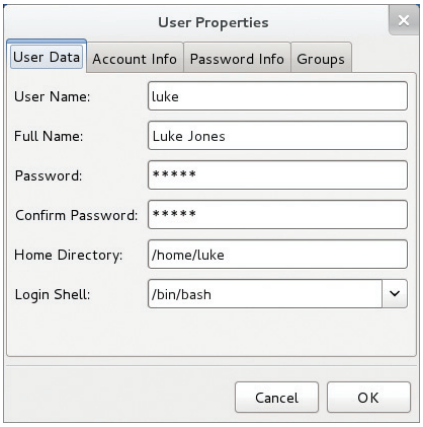


FIGURE 14.3 The User Properties dialog box enables you to edit a wide variety of account properties.

Locking an account's password prevents logins but doesn't completely disable the account.

Account Info This tab provides two options: You can enable or disable account expiration (including setting an expiration date if you enable it), and you can lock or unlock the password.

Password Info This tab enables you to set password expiration data. This information is distinct from account expiration data in that an expired account becomes completely disabled, whereas an expired password can be reset by the user, at least for a while. You can set various change values, such as the number of days after a change before the user must change the password again. You can also set an option to force the user to change the password on the next login. (This option will be cleared once this happens.)

If you want to add the user to an entirely new group, you must first create the new group, as described in the section "Managing Groups."

Groups This tab presents a complete list of groups on the computer. Check the box next to any group to which you want the user to belong. You can also set the user's primary group, but only to a group to which the user already belongs.

Users can change their own passwords by using GUI options in their desktop environments. For instance, in GNOME, selecting the My Account item from the menu that appears when they click their names in the upper-right corner of the screen produces the User Accounts dialog box, shown in Figure 14.4. Clicking the dots in the Password field produces a dialog box in which the user can type a new password. Administrators can also use this tool to add and modify accounts, but it's much less powerful than User Manager in this role.

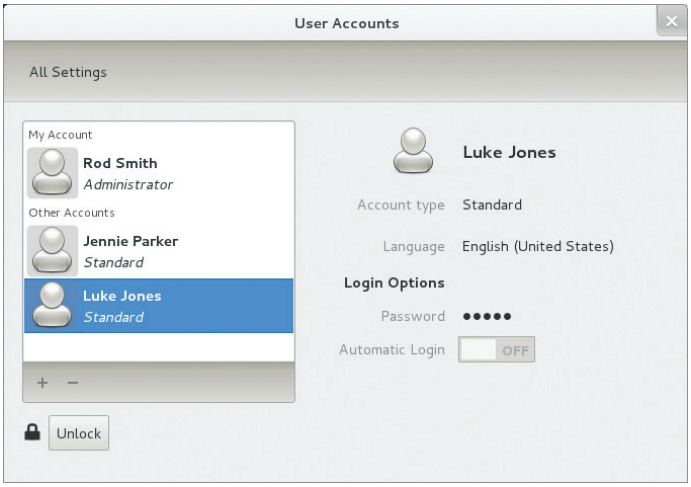


FIGURE 14.4 The User Accounts window enables users to change their own passwords.

Modifying Accounts from the Shell

One of the most frequent account modifications is to change a user's password, either as part of account creation or because a user has forgotten the password. You can make this change with the `passwd` program. Ordinary users can type this command name alone to change their own passwords, but not other users' passwords. The superuser, however, can pass a username to the command to change any account's password:

Certification Objective

In addition to setting passwords, the `passwd` utility enables you to adjust password expiration and aging options. Consult its `man` page for details.

```
# passwd pamela
Changing password for user pamela.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
```

As a security measure, the password you type does not echo to the screen as you type it. If the passwords you type don't match, the program refuses to accept your change and prompts you again for a fresh pair of passwords. The program also checks the strength of the password and refuses to accept one that it deems to be too weak.

Certification Objective

You can handle most other account modifications using the `usermod` program. This command works much like `useradd`, but instead of creating a new account, it modifies an existing one. Table 14.2 summarizes the most important `usermod` options.

TABLE 14.2 Options for `usermod`

Option name	Option abbreviation	Effect
`--append`	`-a`	Used with `--groups` (`-G`), causes the specified groups to be added to, rather than replace, the existing set of groups for the user.
`--comment` *comment*	`-c`	This parameter specifies the comment field for the user. (GUI tools generally describe this as the "full name.")
`--home` *home-dir*	`-d`	You specify the account's home directory with this parameter. It defaults to `/home/`*username*.
`--expiredate` *expire-date*	`-e`	You set the date on which the account will be disabled, expressed in the form *YYYY-MM-DD*, with this option. The default is for an account that does not expire.
`--inactive` *inactive-days*	`-f`	This parameter sets the number of days after a password expires, after which the account becomes completely disabled. A value of `-1` disables this feature and is the default.
`--gid` *default-group*	`-g`	You set the name or GID of the user's default group with this option. The default for this value is a new group named after the user.
`--groups` *group*`[,...]`	`-G`	This parameter sets the names or GIDs of groups to which the user belongs. More than one may be specified by separating them with commas.
`--login` *username*	`-l`	Changes the account's username to the specified value.

(Continues)

TABLE 14.2 *(Continued)*

Option name	Option abbreviation	Effect
`--lock`	`-L`	Locks the account's password, preventing logins.
`--move-home`	`-m`	When this option is included with `--home` (`-d`), `usermod` moves the user's existing home directory to the new location.
`--shell` *shell*	`-s`	You set the name of the user's default login shell with this option.
`--uid` *UID*	`-u`	This parameter changes the account's UID number to the specified value.
`--non-unique`	`-o`	This parameter enables a single UID number to be reused (use it in conjunction with `--uid`/`-u`).
`--unlock`	`-U`	This option unlocks a locked account password.

Many `usermod` options are identical to `useradd` options.

For example, consider the following use of `usermod`:

```
# usermod -u 1072 -m -d /home2/luke luke
```

This command makes three changes to the `luke` account:

- It changes the UID value to 1072.
- It changes the account's home directory to `/home2/luke`.
- It moves the contents of the account's original home directory to its new location.

You might issue a command like this one if you were migrating user accounts to an NFS server mounted at `/home2`. Such a change might require a new home directory location and a change in the UID value to match that used on the NFS server.

You should be careful when making changes to the UID value because, although `usermod` changes the UID values of files in common locations such as the user's home directory and e-mail files, it can miss user files in unusual locations.

If you need to make group changes that require adding new groups, consult the upcoming section "Managing Groups" for information on that topic.

The `--move-home` (`-m`) option will be quick when the move is on a single low-level filesystem but potentially slow when it crosses filesystem boundaries.

Deleting Accounts

Deleting accounts can sometimes be as important as adding or modifying them. Unused accounts can consume disk space. More important, they can be abused, either by their former owners or by others who might be able to break into an account if it has a weak password. Thus, you should routinely delete unused accounts. Before you do so, though, you should understand what happens when you delete an account and decide precisely how to do it, lest you create problems by deleting an account in an inappropriate way. With that knowledge in hand, you can delete accounts using either GUI or text-mode tools.

Avoiding Account-Deletion Pitfalls

Deleting an account may sound simple enough, but a mistake can cause problems, either immediately or in the future. In addition to obvious issues such as accidentally deleting the wrong account, you should consider two factors:

Consider archiving a deleted account's home directory to a long-term backup medium. This will enable you to recover the files should they become valuable in the future.

User file preservation One of the reasons I mentioned for deleting accounts was to regain the disk space consumed by the account. Recovering this disk space, however, can sometimes be the *wrong* thing to do. Users' files might be extremely valuable, either to the users themselves or to the organization that owns the computer. Thus, you should carefully consider whether to delete the user's home directory or do something else with it, such as move it into another user's home directory and change permissions on the files it contains. The same is true of the user's mail queue (normally stored in `/var/spool/mail/`*`username`*, where *`username`* is the username).

UID and GID reuse When an account is deleted, the account's UID and GID become available for reuse. In many cases these numbers will not be reused, since Linux assigns these values based on the highest current value. Thus, if you delete any but the highest-numbered current user, the user's old UID and GID numbers won't be reused unless intervening accounts are also deleted. Nonetheless, such deletions can happen, or you might override the default values. If a UID is reused, any files previously owned by the old user will suddenly appear to be owned by the new user. This might cause no problems, but it might cause confusion about who created the files. In some cases, it could even cause suspicion of wrongdoing to fall on the new user, if the old files contain information the new user shouldn't have or if they reside in directories to which the new user shouldn't have access.

If you want to avoid any chance of confusion or claims of misbehavior falling on new users because of UID or GID reuse, you can use the `find` command

to locate all files with particular UID or GID values, using the `-uid` and `-gid` options, as in:

```
# find / -uid 1004
```

This example finds all files on the computer with a UID of 1004. (Searching on a GID works the same way but using the `-gid` option.) You can then reassign ownership of these files using `chown` or delete them. Ordinarily, you'd issue this command only after deleting or reassigning ownership of the user's home directory, since that directory will probably contain far too many matching files.

You can issue this command as a non-`root` user, but it will return errors and it may miss some files. Thus, it's best to do this as `root`.

Deleting Accounts Using GUI Tools

As with other account management tasks, using a GUI is fairly intuitive, but details vary from one distribution to another. As an example, to delete an account from Fedora's User Manager (`system-config-user`), select the account and then either click Delete in the toolbar or select Edit ➢ Delete from the menu. The result is a confirmation dialog box similar to the one shown in Figure 14.5. Check or uncheck the box to delete the user's home directory, as you see fit and, if you're certain of the action, click Yes. The account will be immediately deleted.

If the user has many files and if you opt to delete those files, the deletion operation may take several seconds, or even minutes, to complete.

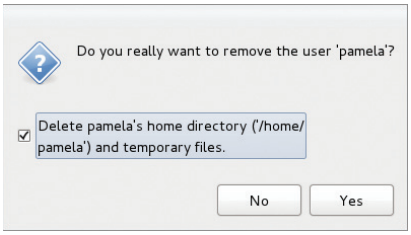


FIGURE 14.5 When you delete an account with User Manager, you may optionally delete the user's home directory.

If the user is currently logged in, User Manager will complain about this fact in the confirmation dialog box. You'll still be able to delete the account, but the user won't be immediately logged out.

Deleting Accounts from the Shell

Certification Objective

The `userdel` command deletes accounts from a text-mode shell. In its simplest form, you pass it a username and nothing more:

```
# userdel pamela
```

The program doesn't prompt you for confirmation; it just deletes the account. It does not, however, delete the user's home directory by default. To have it do so, pass it the `--remove` (`-r`) option.

If the user is currently logged in, `userdel` notifies you of that fact and does nothing. You can pass it the `--force` (`-f`) option to delete the account despite the fact that it's in use. To both force account deletion and remove the user's files, you can pass both options:

```
# userdel -rf pamela
userdel: user pamela is currently logged in
```

The program still complains about the user being logged in, but it deletes the account and files just the same.

Managing Groups

In many respects, groups are analogous to accounts. They're defined in similar files and managed with similar utilities. Groups are also tied to accounts in that accounts include group definitions. Until now, I've assumed you'll be using standard groups or the groups that are defined as part of account creation. Sometimes, though, you need to create, delete, or modify groups for specific purposes, such as if you use a project group strategy. As with account management, you can do this using either GUI or text-mode tools.

Managing Groups Using GUI Tools

If you want to manage system groups as well as user groups, select Edit ➢ Preferences and, in the resulting dialog box, uncheck Hide System Users And Groups.

Many GUI account maintenance tools, such as Fedora's User Manager (`system-config-user`), provide group management tools that are analogous to the user management tools described throughout this chapter. Referring to Figure 14.1, you'll see that the User Manager window includes both Users and Groups tabs. To manage groups, click the Groups tab. This produces a display resembling Figure 14.6.

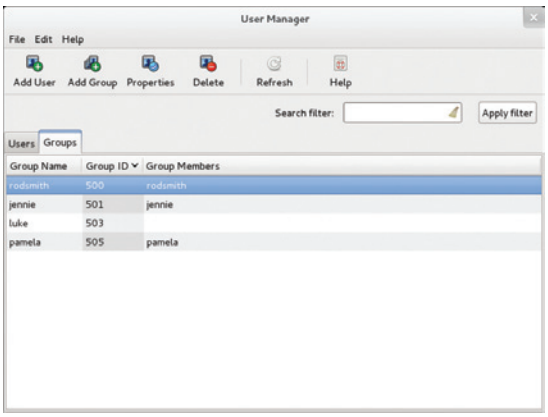


FIGURE 14.6 The User Manager utility enables you to manage groups as well as users.

You can add, modify, and delete groups in a manner that's closely analogous to adding, modifying, and deleting accounts. The number of options available is much smaller, though; for instance, when adding a group, you can specify only its name and GID. Groups don't have home directories, login shells, and so on. You might, of course, want specific users to be members of your new group from the start. To do so, follow these steps:

1. Create a group by clicking Add Group and specifying the group name in the resulting dialog box.
2. Select the group, click Properties, and select the Group Users tab. The result is the Group Properties dialog box shown in Figure 14.7.

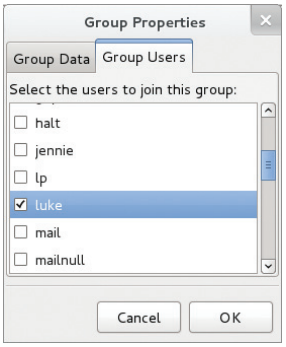


FIGURE 14.7 You can add users to a group using the Group Properties dialog box.

3. The Group Users tab contains an alphabetized list of users; locate the users who should be members of the group, select them, and click OK.

Alternatively, you can manage group membership by altering each user's group membership individually, as described earlier in "Modifying Accounts."

Managing Groups from the Shell

Certification Objective

You can create groups from the shell using the `groupadd` command, which works much like `useradd` for users but takes a smaller set of options, the most important of which appear in Table 14.3. Consult the program's `man` page for information on more obscure options.

TABLE 14.3 Options for `groupadd`

Option name	Option abbreviation	Effect
`--gid` *GID*	`-g`	You can provide a specific GID with this parameter. If you omit it, `groupadd` uses the next available GID.
`--non-unique`	`-o`	Normally, the GID you specify must be unused by other groups, but the `-o` parameter overrides this behavior, enabling you to create multiple groups that share one GID.
`--system`	`-r`	This parameter instructs `groupadd` to create a system group, which is one with a GID of less than 500 or 1,000, depending on the distribution. Non-system groups are normally used as user private groups.
`--force`	`-f`	Normally, if you try to create a group that already exists, `groupadd` returns an error message. This parameter suppresses that error message.

To modify existing groups, you have three options:

- The `usermod` command includes the `--groups` (`-G`) option to alter group membership on a user-by-user basis. You can use this feature to adjust who is a member of particular groups.
- You can use the `groupmod` command. You can use the `--gid` (`-g`) and `--non-unique` (`-o`) options from Table 14.3 with this command, as well as `--new-name` *name* (`-n` *name*), which changes the group's name. You cannot, however, change group membership with `groupmod`.
- You can directly edit the `/etc/group` file, which stores the group definition. This option is sometimes handy if you want to add several users to a new group; however, it's a bit risky, since a stray keypress could damage the file.

4. Managing Files

Much of what you do with a computer involves manipulating files. Most obviously, files hold the correspondence, spreadsheets, digital photos, and other documents you create. Files also hold the configuration settings for Linux—information on how to treat the network interfaces, how to access hard disks, and what to do as the computer starts up. Indeed, even access to most hardware devices and kernel settings is ultimately done through files. Thus, knowing how to manage these files is critically important for administering a Linux computer. This chapter begins with a description of the basic text-mode commands for manipulating files. Directories are files, too, so this chapter covers directories, including the commands you can use to create and manipulate them.

- **Manipulating files**
- **Manipulating directories**

Manipulating Files

If you've used Windows or Mac OS X, chances are you've used a GUI file manager to manipulate files. Such tools are available in Linux, as noted in Chapter 4, "Using Common Linux Programs," and you can certainly use a file manager for many common tasks. Linux's text-mode shells, such as Bash, provide simple but powerful tools for manipulating files, too. These tools can simplify some tasks, such as working with all the files with names that include the string `invoice`. Thus, you should be familiar with these text-mode commands.

To begin this task, I describe some ways you can create files. With files created, you can copy them from one location to another. You may sometimes want to move or rename files, so I explain how to do so. Linux enables you to create *links*, which are ways to refer to the same file by multiple names. If you never want to use a file again, you can delete it. *Wildcards* provide the means to refer to many files using a compact notation, so I describe them. Finally, I cover the case-sensitive nature of Linux's file manipulation commands.

Creating Files

Chapter 11, "Editing Files," describes how to create text files with the text-mode `pico`, and `nano`, and Vi editors.

Certification Objective

Normally, you create files using the programs that manipulate them. For instance, you might use a graphics program to create a new graphics file. This process varies from one program to another, but GUI programs typically use a menu option called Save or Save As to save a file. Text-mode programs provide similar functionality, but the details of how it's done vary greatly from one program to another.

One program deserves special mention as a way to create files: `touch`. You can type this program's name followed by the name of a file you want to create, such as **`touch newfile.txt`** to create an empty file called `newfile.txt`. Ordinarily, you don't need to do this to create a file of a particular type, since you'll use a specialized program to do the job. Sometimes, though, it's helpful to create an empty file just to have the file itself—for instance, to create a few "scratch" files to test some other command.

A programmer's tool known as `make` compiles source code if it's new, so programmers sometimes use `touch` to force `make` to recompile a source code file.

If you pass `touch` the name of a file that already exists, `touch` updates that file's access and modification time stamps to the current date and time. This can be handy if you're using a command that works on files based on their access times and you want the program to treat an old file as if it were new. You might also want to do this if you plan to distribute a collection of files and you want them all to have identical time stamps.

You can use a number of options with `touch` to modify its behavior. The most important of these are as follows:

Don't create a file The `-c` or `--no-create` option tells `touch` to not create a new file if one doesn't already exist. Use this option if you want to update time stamps but not accidentally create an empty file, should you mistype a filename.

Set the time to a specific value You can use `-d` *`string`* or `--date=`*`string`* to set the date of a file to that represented by the specified `string`, which can take any number of forms. For instance, **`touch -d "July 4 2012" afile.txt`** causes the date stamps on `afile.txt` to be set to July 4, 2012. You can achieve the same effect with `-t` `[[`*CC*`]`*YY*`]`*MMDDhhmm*`[.`*ss*`]`, where `[[`*CC*`]`*YY*`]`*MMDDhhmm* `[.`*ss*`]` is a date and time in a specific numeric format, such as `201207041223` for 12:23 PM on July 4, 2012.

Consult the `man` page for `touch` to learn about its more obscure options.

Copying Files

Certification Objective

If you're working in a text-mode shell, the `cp` command copies a file. (Its name is short for *copy*.) Its basic use is to pass it a source filename and a destination filename, a destination directory name, or both. Thus, there are three ways you

can use it, as outlined in Table 7.1. Although the example filenames in Table 7.1 suggest that the original file be in your current working directory, this need not be the case; *orig.txt* could include a directory specification, such as /etc/fstab or ../afile.txt.

Chapter 6, "Getting to Know the Command Line," covers various types of absolute and relative directory references.

TABLE 7.1 Examples of the use of cp

Example command	Effect
`cp orig.txt new.txt`	Copies *orig.txt* to *new.txt* in the current directory.
`cp orig.txt /otherdir`	Copies orig.txt to the */otherdir* directory. The copy will be called *orig.txt*.
`cp orig.txt /otherdir/new.txt`	Copies orig.txt to the */otherdir* directory. The copy will be called *new.txt*.

The critical point to understand is how the destination filename is specified. This can be less than obvious in some cases, since file and directory specifications can look alike. For instance, consider the following command:

```
$ cp outline.pdf ~/publication
```

This command can produce any of three results:

- If ~/publication is a directory, the result is a file called ~/publication/outline.pdf.
- If ~/publication is a file, the result is that this file will be replaced by the contents of outline.pdf.
- If ~/publication doesn't yet exist, the result is a new file, called ~/publication, which is identical to the original outline.pdf.

If you follow a directory name with a slash (/), as in ~/publication/, cp returns an error message if ~/publication doesn't exist or is a regular file.

Keeping these results straight can be confusing if you're new to command-line file copying. Thus, I encourage you to experiment by creating a test directory using mkdir (described later, in "Creating Directories"), creating subdirectories in this directory, and copying files into this test directory tree using all of these methods of referring to files. (This is the type of situation where touch can be handy for creating test files.)

The cp command provides many options that modify its behavior. Some of the more useful options enable you to modify the command's operation in helpful ways:

Force overwrite The -f or --force option forces the system to overwrite any existing files without prompting.

Use interactive mode The `-i` or `--interactive` option causes `cp` to ask you before overwriting any existing files.

Preserve ownership and permissions Normally, a copied file is owned by the user who issues the `cp` command and uses that account's default permissions. The `-p` or `--preserve` option preserves ownership and permissions, if possible.

> **Chapter 13, "Understanding Users and Groups," describes Linux accounts. Chapter 15, "Setting Ownership and Permissions," describes file permissions.**

Perform a recursive copy If you use the `-R` or `--recursive` option and specify a directory as the *source*, the entire directory, including its subdirectories, is copied. Although `-r` also performs a recursive copy, its behavior with files other than ordinary files and directories is unspecified. Most `cp` implementations use `-r` as a synonym for `-R`, but this behavior isn't guaranteed.

Perform an archive copy The `-a` or `--archive` option is similar to `-R`, but it also preserves ownership and copies links as is. The `-R` option copies the files to which symbolic links point rather than the symbolic links themselves. (Links are described in more detail later in this chapter, in "Using Links.")

Perform an update copy The `-u` or `--update` option tells `cp` to copy the file only if the original is newer than the target or if the target doesn't exist.

This list of `cp` options is incomplete but covers the most useful options. Consult `cp`'s `man` page for information about additional `cp` options.

Moving and Renaming Files

Certification Objective

In a text-mode shell, the same command, `mv`, is used to both move and rename files and directories. Its use is very similar to that of `cp`; for instance, if you wanted to move `outline.pdf` to `~/publication`, you would type:

```
$ mv outline.pdf ~/publication
```

> **Linux uses `mv` for renaming files because the two operations are identical when the source and destination directories are the same.**

If you specify a filename with the destination, the file will be renamed as it's moved. If you specify a filename and the destination directory is the same as the source directory, the file will be renamed but not moved. In other words, `mv`'s effects are much like `cp`'s, except that the new file replaces, rather than supplements, the original.

Behind the scenes, `mv` does the following:

- When the source and target are on the same filesystem, `mv` rewrites directory entries without actually moving the file's data.
- When you move a file from one filesystem to another, `mv` copies the file and then deletes the original file.

The `mv` command takes many of the same options as `cp` does. From the earlier list, `--preserve`, `--recursive`, and `--archive` don't apply to `mv`, but the others do.

Using Links

Certification Objective

Sometimes it's handy to refer to a single file by multiple names. Rather than create several copies of the file, you can create multiple *links* to one file. Linux supports two types of links, both of which are created with the `ln` command:

Hard link A *hard link* is a duplicate directory entry. Both entries point to the same file. Because they work by tying together low-level filesystem data structures, hard links can exist only on a single filesystem. In a hard link scenario, neither filename holds any sort of priority over the other; both tie directly to the file's data structures and data. Type **`ln` *`origname linkname`***, where *`origname`* is the original name and *`linkname`* is the new link's name, to create a hard link.

Symbolic link A *symbolic link* (aka a *soft link*) is a file that refers to another file by name. That is, the symbolic link is a file that holds another file's name, and when you tell a program to read to or write from a symbolic link file, Linux redirects the access to the original file. Because symbolic links work by filename references, they can cross filesystem boundaries. Type **`ln -s` *`origname linkname`*** to create a symbolic link.

Symbolic links are similar to *shortcuts* on the Windows desktop.

You can identify links in long directory listings (using the `-l` option to `ls`) in a couple of ways. An example will illustrate this:

```
$ ln report.odt hardlink.odt
$ ln -s report.odt softlink.odt
$ ls -l
total 192
-rw-r--r-- 2 rod users 94720 Jan 10 11:53 hardlink.odt
-rw-r--r-- 2 rod users 94720 Jan 10 11:53 report.odt
lrwxrwxrwx 1 rod users    10 Jan 10 11:54 softlink.odt -> report.odt
```

This example began with a single file, `report.odt`. The first two commands created two links, a hard link (`hardlink.odt`) and a symbolic link (`softlink.odt`). Typing **`ls -l`** shows all three files. The original file and the hard link can be identified as links by the presence of the value `2` in the second column of the `ls -l` output; this column identifies the number of filename entries that point to the file, so a value higher than `1` indicates that a hard link exists. The symbolic link is denoted by an `l` (a lowercase *L*, not a digit *1*) in the first character of the `softlink.odt` file's permissions string (`lrwxrwxrwx`). Furthermore, the symbolic link's filename specification includes an explicit pointer to the linked-to file.

Both types of links are useful for referring to files by multiple names or in multiple directories. For instance, if you write a letter that you send to multiple recipients, you might want to store copies in directories devoted to each recipient. In such a situation, either type of link will probably work fine, but each type

has implications. Most importantly, if you use symbolic links, deleting the original file makes the file completely inaccessible; the symbolic links remain but point to a non-existent file. If you use hard links, by contrast, you must delete *all* the copies of the file to delete the file itself. This is because hard links are duplicate directory entries that point to the same file, whereas symbolic links are separate files that refer to the original file by name.

If you modify a file by accessing its soft link, or by any hard-linked name, you should be sure that the program you use will modify the original file. Some programs create a backup of the original file that you can use to recover the original in case you find that your changes were in error. Most editors do this in such a way that the backup is a new file, and write changes to the original file, thus affecting the original file as well as the link. Some programs, though, rename the original file and then write a new file with the changes. If a program does this and you've accessed the file via a link, the linked-to file will be unaffected by your changes. If in doubt, test your program to be sure it does what you expect.

If you want to create a link to a directory, be aware that you can normally do this only via symbolic links. Hard links between directories are potentially dangerous in terms of low-level filesystem data structures, so the `ln` utility permits only the superuser to create such links. Even then, most filesystems disallow hard links between directories, so in practice even `root` usually can't create them. Any user can create symbolic links to a directory, though.

Linux installations make use of links (mostly symbolic links) in various places. For instance, system startup scripts are often referred to via symbolic links located in directories dedicated to specific startup conditions, known as *runlevels*. Runlevel management is beyond the scope of this book.

The `rm` command's name is (very!) short for *remove*.

Deleting Files

The `rm` command deletes files in a text-mode shell. As you might expect, you pass the names of one or more files to this command:

```
$ rm outline.pdf outline.txt
```

Distributions sometimes set the `-i` option by default for `root`, but not for ordinary users.

This example deletes two files, `outline.pdf` and `outline.txt`. If you want to delete an entire directory tree, you can pass `rm` the `-r`, `-R`, or `--recursive` option along with a directory name:

```
$ rm -r oldstuff/
```

The `-i` option causes `rm` to prompt before deleting each file. This is a useful safety measure. You can use the `-f` (`--force`) option to override this setting, if `-i` is configured as the default. Several other options to `rm` exist; consult its `man` page to learn about them.

It's important to realize that `rm` does *not* implement any functionality like a file manager's "trash can." Once you delete a file with `rm`, it's gone, and you can't recover it except by using low-level filesystem tools—a topic that's well beyond the scope of this book. Thus, you should be careful when using `rm`—and even more careful when using it with its `-r` option or as `root`!

Using Wildcards

Certification Objective

You can use *wildcards* to refer to files. (Using wildcards is also sometimes called *globbing*.) A wildcard is a symbol or set of symbols that stands in for other characters. Three classes of wildcards are common in Linux:

? A question mark (`?`) stands in for a single character. For instance, `b??k` matches `book`, `balk`, `buck`, or any other four-character filename that begins with `b` and ends with `k`.

***** An asterisk (`*`) matches any character or set of characters, including no character. For instance, `b*k` matches `book`, `balk`, and `buck` just as does `b??k`. `b*k` also matches `bk`, `bbk`, and `backtrack`.

Bracketed values Characters enclosed in square brackets (`[]`) normally match any character in the set. For instance, `b[ao][lo]k` matches `balk` and `book` but not `buck`. It's also possible to specify a range of values; for instance, `b[a-z]ck` matches `back`, `buck`, and other four-letter filenames of this form whose second character is a lowercase letter. This differs from `b?ck`—because Linux treats filenames in a case-sensitive way and because `?` matches any character (not just any letter), `b[a-z]ck` doesn't match `bAck` or `b3ck`, although `b?ck` matches both of these filenames.

Wildcards are implemented in the shell and passed to the command you call. For instance, if you type **`ls b??k`**, and that wildcard matches the three files `balk`, `book`, and `buck`, the result is precisely as if you'd typed **`ls balk book buck`**.

The way Bash expands wildcards can lead to unexpected, and sometimes undesirable, consequences. For instance, suppose you want to copy two files, specified via a wildcard, to another directory, but you forget to give the destination directory. The `cp` command will interpret the command as a request to copy the first of the files over the second.

Understanding Case Sensitivity

Certification Objective

Linux's native filesystems are case-sensitive, which means that filenames that differ only in case are distinct files. For instance, a single directory can hold files called `afile.txt`, `Afile,txt`, and `AFILE.TXT`, and each is a distinct file. This

case sensitivity also means that, if you type a filename, you must enter it with the correct case—if a file is called `afile.txt` but you type its name as `Afile.txt`, the program you're using will tell you that the file doesn't exist.



Apple's Hierarchical File System Plus (HFS+) supports both case-sensitive and case-insensitive variants. Apple uses the case-insensitive mode by default.

This is different from what happens in Windows or (usually) in Mac OS X, in which filenames that differ only in case are treated identically. This means that, in these OSs, you can't have two files that differ only in case in the same directory, and you can specify a filename using any case variant you like. Windows also creates a short filename (8 characters with an optional 3-character extension) for every file with a longer name, to help out older software that works only with such filenames. Linux doesn't create such alternate filenames.

Case sensitivity is primarily a function of the filesystem, not of the operating system. Thus, if you access a non-Linux filesystem (on a removable disk, a non-Linux partition on a dual-boot computer, or using a network filesystem), you may find that case-insensitive rules will apply. This is particularly likely when accessing File Allocation Table (FAT) and New Technology File System (NTFS) volumes, which are common on Windows computers, external hard disks, and USB flash drives. A further twist on this rule is that many Linux programs, such as Bash, assume case sensitivity even on case-insensitive filesystems. Features such as command completion, described in Chapter 6, "Getting to Know the Command Line," may work only if you use the case in which filenames are recorded, even on case-insensitive filesystems.

Ordinarily, case sensitivity creates few real problems, particularly if you use GUI programs that enable you to point-and-click to select files. You should be aware of these issues, however, when copying files or directories to FAT, NTFS, HFS+, or other case-insensitive filesystems. If a directory you want to copy contains files with names that differ only in case, you'll end up with a disk that contains just one of the offending files.

Manipulating Directories

No doubt you're familiar with the concept of directories, although you may think of them as "folders," since most GUI file managers represent directories using file folder icons. Naturally, Linux provides text-mode commands to manipulate directories. These include directory-specific commands to create and delete directories, as well as use of some of the file-manipulation commands described earlier to manage directories.

Creating Directories

Certification Objective

You can use the `mkdir` command to create a directory. Ordinarily, you'll use this command by typing the name of one or more directories following the command:

```
$ mkdir newdir
$ mkdir dirone newdir/dirtwo
```

The first example creates just one new directory, `newdir`, which will then reside in the current directory. The second example creates two new directories: `dirone` and `newdir/dirtwo`. In this example, `mkdir` creates `dirtwo` inside the `newdir` directory, which was created with the preceding command.

In most cases, you'll use `mkdir` without options, other than the name of a directory, but you can modify its behavior in a few ways:

Set mode The `-m` *mode* or `--mode=`*mode* option causes the new directory to have the specified permission mode, expressed as an octal number. (Chapter 15, "Setting Ownership and Permissions," describes these topics in more detail.)

Create parent directories Normally, if you specify the creation of a directory within a directory that doesn't exist, `mkdir` responds with a `No such file or directory` error and doesn't create the directory. If you include the `-p` or `--parents` option, though, `mkdir` creates the necessary parent directory. For instance, typing **`mkdir first/second`** returns an error message if `first` doesn't exist, but typing **`mkdir -p first/second`** succeeds, creating both `first` and its subdirectory, `second`.



Chapter 6 includes information on how to specify locations other than the current directory, as well as how to change your current directory with the `cd` command.

Deleting Directories

Certification Objective

The `rmdir` command is the opposite of `mkdir`; it destroys a directory. To use it, you normally type the command followed by the names of one or more directories you want to delete:

```
$ rmdir dirone
$ rmdir newdir/dirtwo newdir
```

These examples delete the three directories created by the `mkdir` commands shown earlier.

Like `mkdir`, `rmdir` supports few options, the most important of which handle these tasks:

Ignore failures on non-empty directories Normally, if a directory contains files or other directories, `rmdir` doesn't delete it and returns an error message.

With the `--ignore-fail-on-non-empty` option, `rmdir` still doesn't delete the directory, but it doesn't return an error message.

Delete tree The `-p` or `--parents` option causes `rmdir` to delete an entire directory tree. For instance, typing **`rmdir -p newdir/dirtwo`** causes `rmdir` to delete `newdir/dirtwo`, then `newdir`. You could use this command rather than the second one shown earlier to delete both of these directories.

You should understand that `rmdir` can delete only *empty* directories; if a directory contains any files at all, it won't work. (You can use the `-p` option, however, to delete a set of nested directories, as long as none of them holds any non-directory file.) Of course, in real life you're likely to want to delete directory trees that hold files. In such cases, you can use the `rm` command, described earlier, in "Deleting Files," along with its `-r` (or `-R` or `--recursive`) option:

```
$ rm -r newdir
```

This command deletes `newdir` and any files or subdirectories it might contain. This fact makes `rm` and its `-r` option potentially dangerous, so you should be particularly cautious when using it.

Linux Security Features

When you log in as an ordinary user, you can accidentally delete your own files if you err in your use of `rm` or various other commands. You cannot, however, do serious damage to the Linux installation itself. This is because Unix was designed as a multi-user OS with multi-user security features in mind, and because Linux is a clone of Unix, Linux has inherited these security features. Among these features are the concepts of file ownership and file permissions. You can only delete your own files—or more precisely, you can only delete files if you have write access to the directories in which they reside. You have such access to your own home directory, but not to the directories in which Linux system files reside. Thus, you can't damage these Linux system files.

Chapter 13, "Understanding Users and Groups," covers these concepts in more detail. Chapter 13 also describes how you can acquire the power to administer the computer. With this power comes the ability to damage the system, though, so you should be careful to do so only when necessary.

Managing Directories

Directories are just special files—they're files that hold other files. Thus, you can use most of the file-manipulation tools described elsewhere in this chapter to manipulate directories. There are some caveats, though:

- You can use `touch` to update a directory's time stamps, but you can't use `touch` to create a directory; `mkdir` handles that task.
- You can use `cp` to copy a directory; however, you must use the `-r`, `-R`, `--recursive`, `-a`, or `--archive` option to copy the directory and all its contents.
- You can use `mv` to move or rename a directory.
- You can use `ln` with its `-s` option to create a symbolic link to a directory. No common Linux filesystem supports hard links to directories, though.

As an example, suppose you have a directory in your home directory called `Music/Satchmo`, which contains Louis Armstrong music files. You want to reorganize this directory so that the files appear under the performer's last name, but you want to retain access to the files under the name `Satchmo`, since your music players refer to them this way. You could type the following commands to achieve this goal:

```
$ cd ~/Music
$ mv Satchmo Armstrong
$ ln -s Armstrong Satchmo
```

Alternatively, you could omit the first command and specify the complete path to each of the directories or links in the `mv` and `ln` commands. As written, the first two of these commands rename the `~/Music/Satchmo` directory to `~/Music/Armstrong`. The final command creates a symbolic link, `~/Music/Satchmo`, that points to `~/Music/Armstrong`.

The Essentials and Beyond

Much of what you do with a computer qualifies as file management. Thus, you must understand the basic tools for managing files in Linux. These include commands to create, delete, copy, move, and rename files, as well as to create links to files. Directories in Linux are just files that contain other files, so most of the same commands you can use on files also work on directories. Special commands to create and delete directories exist, too.

(Continues)

Computer documents come in many forms, but one of the most basic and flexible is text files. Because of their importance and ubiquity, you must be able to edit text files. This chapter covers this task, with an emphasis on the simple text-mode `pico`, `nano`, and Vi editors. I begin by describing some of the roles that text files play. I then describe how to select a text editor. To edit text files, of course, you must be able to start the editor, either on an existing document or to create a new one. The `pico` and `nano` editors are quite similar to each other, so I describe their operation together, followed by Vi, which is a much more unusual editor by modern standards. I conclude this chapter with a look at conventions used in configuration files and common formatted text files—two types of text files you'll probably have to edit sooner or later.

- **Understanding the role of text files**
- **Choosing an editor**
- **Launching an editor**
- **Editing files with `pico` or `nano`**
- **Editing files with Vi**
- **Using configuration file conventions**
- **Editing formatted text files**

Understanding the Role of Text Files

A text editor enables you to edit documents that are stored in a plain text format—typically using the American Standard Code for Information Interchange (ASCII), but such files increasingly use Unicode formats to support additional characters. These formats store text documents that, by themselves, include no special formatting or embedded features. That is, text files can't include graphics, use multiple fonts, emphasize words by italicizing them, or use other features that you probably associate with word processors. (As described shortly, though, markup tools provide a partial exception to this rule.)

ASCII and Unicode

ASCII dates to the 1960s. It's a 7-bit code, meaning that it supports a maximum of 2^7, or 128, characters. (In practice, ASCII uses 8 bits, so an extra 128 characters are available. These encode various ***control characters*** or are used in ASCII extensions.) Combined with the fact that it was created to encode the letters used in English, digits, and symbols, ASCII's limited character count makes it rather unhelpful for many non-English languages—there just aren't enough characters available to handle all the characters for all non-English languages.

Over the years, extensions to and variants of ASCII have been used to support additional characters and alphabets that ASCII doesn't support. One way to do this is to use a ***code page*** to specify, essentially, an alphabet. Each code page specifies a variant of ASCII that's suitable for a particular alphabet. For instance, code page 866 encodes Cyrillic (the alphabet used by Russian and most other Slavic languages). The problem with code pages is that you can generally use only one code page at a time.

Unicode is a more modern approach. It provides a much larger character set, enabling the encoding of any alphabet in common use on Earth, including the huge logographic writing systems used in languages such as Chinese and Japanese. The problem is that Unicode requires many more bits, and several ways to encode it efficiently exist. Fortunately, these Unicode Transformation Format (UTF) schemes are limited in number compared to code pages. Some, such as UTF-8, map the first characters in the same way ASCII does, so an ASCII file is also a valid UTF-8 file. Many text editors today handle UTF-8 (or other Unicode formats) automatically, so you can use a text editor to write text files in any language you like. You may still need to set localization options to tell Linux what sort of keyboard you use and what code page to use by default for programs that still rely on code pages.

Text files encode the ends of lines using one or two special ASCII characters. End-of-line encoding differs between Unix (or Linux) and Windows, but most programs can handle either method.

Formatted text files encode special formatting using unique character sequences. Although you can edit such files with a text editor, specialized editors also exist for many such file types.

Text files consist of lines, which can vary in length from 0 characters to the file's entire size. Frequently, line lengths end at 80 characters or less to make viewing and printing the files easier on conventional terminals and printers, which often have 80-character line-length limits.

Text files can hold any number of types of data. Some of these you might want to create or edit as an ordinary user, and others are important for administering a Linux system. The main file types include the following:

- Human language files
- Programming language files
- Formatted text files

- Program and system configuration files
- Program log files

Some files contain elements of multiple categories. E-mail, for instance, can be stored in text files. An e-mail file consists largely of human language, but e-mail messages include *headers*, which describe the origin and destination computers, along with information on how the message traveled from one site to the other, which is similar to formatted text or log file data.

Unlike Unix and Linux, many Windows configuration and log files use binary formats, necessitating the use of specialized utilities to manage them.

Choosing an Editor

All Linux distributions ship with many text editors. Broadly speaking, text editors fall into one of two categories: text-mode and GUI. Beginners are generally more comfortable with GUI editors, and they can be more convenient to use even for experts. Sometimes, though, the X Window System (X for short) doesn't start properly, so you may have to use a text-mode editor. Thus, I recommend that you familiarize yourself with at least one text-mode editor. Some popular text-mode editors include the following:

Most Linux distributions use a version of Vi called Vi Improved, or Vim; you can still launch it by typing `vi`.

Vi This editor is a Unix staple. It's small and is usually installed by default, so you can be fairly certain it's present on any Linux computer. It is, however, strange by modern standards; it uses multiple editing *modes*, and you must switch between them to accomplish various tasks. Many longtime Unix and Linux administrators like Vi for its flexibility, power, and small size.

emacs This editor is another Unix staple. It's a big editor with lots of features, so it's less likely to be installed by default, particularly on small emergency systems. Its operating model is more like those of text editors with which novices are familiar, but its commands can seem rather odd.

Bash's text-editing commands are modeled on those of `emacs`, so learning `emacs` can improve your ability to manage Bash.

pico Several small editors are modeled after `emacs` but omit many of its advanced features in an effort to simplify the editor. One of these editors is `pico`, which originated as part of the `pine` e-mail package.

nano This editor is a clone of `pico` that adds extra features. Many distributions install `nano` by default.

Of the preceding editors, `nano` is probably the best one to start with; it's common enough that you're likely to find it already installed on your system, and its method of operation is less idiosyncratic than that of Vi. Although `emacs` is a capable editor, it's less likely to be installed by default, which can be a drawback,

Some distributions provide `pico` and some don't. Some don't provide `pico`, but cause `nano` to launch when you use the `pico` command.

particularly when using an emergency recovery system. For the most part, `pico` works just like `nano`, but because `nano` adds to `pico`'s features, `nano` is a more capable editor. Figure 11.1 shows `nano` in operation within a text-mode login session, editing a file called `fstab`.

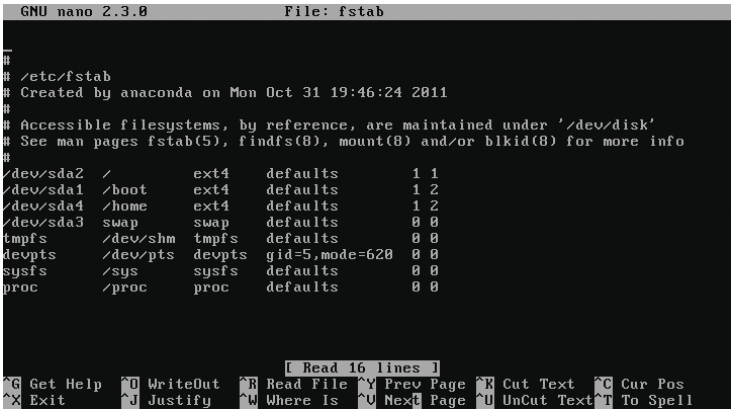


FIGURE 11.1 The `nano` editor enables you to edit a text file in text mode.

As with text-mode editors, several GUI editors are available, including the following:

`emacs` This editor is both a text-mode editor and a GUI editor. The GUI features of `emacs`, however, are sometimes a bit odd; for instance, the scroll bar to move through the file appears on the left side of the window rather than the more common right side.

`gedit` The GNU Network Object Model Environment (GNOME) desktop environment has an associated text editor known as `gedit`. It's a fairly typical text editor, and it's often installed by default.

KWrite and Kate Just as `gedit` is associated with GNOME, KWrite and Kate are editors that are associated with the K Desktop Environment (KDE). KWrite is slightly more sophisticated than `gedit`, and Kate adds some more features, but neither is nearly as powerful as `emacs`.

NEdit This editor is not tied to any particular desktop environment, but it's also unlikely to be installed by default. It's similar in power to KWrite.

For a new Linux user, any of these is a good starting GUI editor; all offer the basic features you need for light text file editing. Which you choose may depend on which is installed by default on your system. In the long term, you should probably try a variety of editors to find the one you like best.

Launching an Editor

If you're comfortable with a GUI desktop environment, you can launch a GUI text editor using the desktop environment's menu system or by using a file manager to browse to a file you want to edit and double-clicking that file. This approach works well for files that you own, but if you need to edit system files, you're likely to lack permissions to save changes if you launch a text editor in this way.

Another approach to launching a text editor is to do so from the command line. You can do this as a normal user or as `root`, so you can edit text files that you own as well as system files. You can also select the editor you want to use—you just need to know its filename, which you type to launch the editor:

```
$ nano
```

This example launches `nano`. When it opens, you'll see a display like that shown in Figure 11.1, but the bulk of the window or text-mode console will be empty, since you didn't specify a filename. Instead, the center of the top line will read `New Buffer`. You can begin typing your file. When you save the file (as described later, in "Saving Your Changes from `nano`"), `nano` will ask for a filename. Alternatively, you can provide a filename when you launch the program:

```
$ nano great_american_novel.txt
```

This example opens the `great_american_novel.txt` file and displays it. If the file doesn't exist, `nano` displays `New File` near the bottom of the display, where `Read 16 lines` appears in Figure 11.1. If you mistype the filename, `nano` will show an empty file, so if you see an empty file instead of the file you were expecting to see, you may have mistyped the filename. If you see `Warning: no write permission` in the third line from the bottom, this means that you've loaded a file that you have no permission to write. You'll need to run `nano` as `root` or modify file permissions, as described in Chapter 15, "Setting Ownership and Permissions," if you want to save the file.

Chapter 13, "Understanding Users and Groups," describes accounts, including the `root` account, in more detail.

Editing Files with *pico* or *nano*

Certification Objective

If you're familiar with text-mode text editors, you should have few problems learning `pico` or `nano`. If you've only used GUI editors to edit text, though, you'll have to learn a few conventions about how to use the keyboard. You must move about the document using the keyboard rather than the mouse, for instance. You can insert, replace, and delete text much as you do in a GUI text editor or word processor.

Using Text Editor Conventions

Every text editor has its own conventions for information it displays on the screen, how to manipulate text, and so on. Most text-mode text editors are similar up to a point—for instance, one or more lines at the top or bottom of the display typically show summary information or brief command prompts. Figure 11.1 shows this information for `nano`, which includes:

The title bar The first line of the display is the title bar. This line includes `nano`'s version number, the name of the file being edited, and the modification status.

The status bar The third line from the bottom of the display is reserved for status information and interactions with the user. This line will prompt you for information such as a filename to write when you save your document or terms you want to find in the document when you perform a search operation.

The shortcut list The bottom two lines of the editor show a summary of some of the most common operations, along with the keystrokes that trigger them.

A carat (^) preceding a letter refers to a control character. In this book, I indicate such key combinations with "Ctrl+" rather than a carat.

In addition to control characters, `nano` uses *meta* characters to activate some functions. These key combinations use the Esc key followed by another key. In `nano`'s documentation, meta sequences are denoted by `M-`*k*, where *k* is a key. In this book, I use Esc rather than M to prefix a meta key combination. For instance, Figure 11.1 describes `M-?` as the key sequence to move to the last line of the document. Note that these are distinct keystrokes, unlike Ctrl key sequences; that is, you would press Esc, release it, and *then* press the question mark (?) key, including its Shift modifier, to move to the last line of the document.

Exploring Basic *nano* Text-Editing Procedures

You can use `pico` instead of `nano` if you prefer; it works almost identically for this task.

As a method of learning `nano`, consider the task of editing `/etc/fstab` to add a new disk to the computer. (Don't worry too much about this file's purpose or format; I'm simply using it as a concrete example for demonstrating the editor.) Listing 11.1 shows the original `/etc/fstab` file used in this example. If you want to follow along, enter it using a text editor with which you're already familiar, and save it to a file on your disk. Alternatively, copy `/etc/fstab` from your own system to your home directory and use it, but be aware that it won't be identical to the one shown in Listing 11.1. Don't edit your *real* `/etc/fstab` file as a learning exercise; a mistake could render your system unbootable the next time you reboot. Instead, work on a copy of this file.

Listing 11.1: Sample `/etc/fstab` file

```
/dev/sda2  /          ext4    defaults        1 1
/dev/sda1  /boot      ext4    defaults        1 2
/dev/sda4  /home      ext4    defaults        1 2
/dev/sda3  swap       swap    defaults        0 0
tmpfs      /dev/shm   tmpfs   defaults        0 0
devpts     /dev/pts   devpts  gid=5,mode=620  0 0
sysfs      /sys       sysfs   defaults        0 0
proc       /proc      proc    defaults        0 0
```

The first step to using `nano` is to launch it and have it load the file. In this example, type **`nano fstab`** while in the directory holding the file. The result should resemble Figure 11.1 (shown earlier), which illustrates `nano` editing Listing 11.1 but with some comments at the start of the file.

You can add a new entry to `fstab` in a couple of ways. The first way is to create a new empty line and type an entry manually. You can do so as follows:

1. Press the down arrow key repeatedly until the cursor appears over the `t` in `tmpfs`.
2. Press the Enter key. This action opens a new line between the `/dev/sda3` line and the `tmpfs` line.
3. Press the up arrow key once to reposition the cursor on the empty new line.
4. Type the following new line:

```
/dev/sdb1  /home2    ext4    defaults        0 0
```

A second way of creating a new entry demonstrates how to cut and paste text in the editor:

1. Move the cursor to the beginning of the `/dev/sdb1` line you've just created by using the arrow keys; you should see the cursor resting on the `/` in `/dev/sdb1`.
2. Press Esc-6. This keystroke copies the line on which the cursor resides into a cut/copy buffer.
3. Press Ctrl+U. This keystroke pastes the contents of the cut/copy buffer into the file in the current location.
4. Use the arrow keys to move the cursor to the right of `/dev/sdb1`.

Comments and other configuration file conventions are described later, in "Using Configuration File Conventions."

Alternatively, Ctrl+K *cuts* text, meaning that it's removed from the file and copied to the buffer. This is useful to move lines to a new location in the file.

You can make more extensive changes to the `fstab` file, if you like, but be sure to work from a *copy* of the file!

5. Press the Backspace key once and then press the 2 key to change `/dev/sdb1` to `/dev/sdb2`.
6. Use the arrow keys to move the cursor to the left of `/home2`.
7. Press the Backspace key once and then press the 3 key to change `/home2` to `/home3`.

The result of these changes is a new line, very similar to the one you added earlier:

```
/dev/sdb2   /home3    ext4    defaults        0 0
```

You can make additional changes in a similar way. Although `nano` lacks a GUI, most of its principles are the same as the ones in GUI text editors and word processors; you just need to know the keystroke to activate the feature you want. Pressing Ctrl+G produces a help listing that summarizes the program's features, which can be handy as you get to know the editor.

Some additional features you might want to use include the following:

Move to the start or end of the file You can use the arrow keys, Page Up, Page Down, Home, and End to move the cursor around in ways that are common to other editors. To move to the start of the file, press Esc-\; and to move to the end of the file, press Esc-/.

Copy or move multiple lines If you need to copy or move multiple consecutive lines, you can repeat the Esc-6 or Ctrl+K operation; `nano` retains all the lines you copy or cut, so that when you press Ctrl+U, all of them will be pasted back.

Insert a file Pressing Ctrl+R or F5 enables you to insert another file in the current one, at the cursor's current position.

Search for a string Pressing Ctrl+W or F6 activates a search feature. When activated, `nano` prompts you for a search term. Type it, followed by the Enter key, and `nano` finds the next instance of that search term in the file. When you press Ctrl+W or F6 again, the default search term is the last one used, so you can search repeatedly for the same term by pressing Ctrl+W or F6 followed by Enter for each search operation. Alternatively, Esc-W or F16 repeats the last search.

Replace a string You can replace one string with another by pressing Ctrl+\, Esc-R, or F14. The program prompts you to enter a search term and the term you want to replace it. The search then commences, and `nano` asks you to verify each replacement. If you want to replace *all* the occurrences without prompting, you can press the A key at the first prompt.

Saving Your Changes from *nano*

Once you've made changes to a text file, of course, you probably want to save them. One way to do this is with the Ctrl+O option. (That's a letter O, not a digit 0.) When you press this key, nano asks:

```
Write Selection to File:
```

Ordinarily, the prompt will include the file's original name, so you can press the Enter key to save the file using that name. If you want to use a different name, though, you can delete the old one and type a new name. If you launched nano without specifying a filename, you can type one at this prompt.

Another way to save the file is to press Ctrl+X. This command exits from nano, but if you've modified the file, it produces the following prompt:

```
Save modified buffer (ANSWERING "No" WILL DESTROY CHANGES) ?
```

Type **y** at this prompt to save the file. The program then shows you the filename prompt that you'd have seen if you'd pressed Ctrl+O, so you can change the filename if you like. Once nano saves the file, it terminates.

Editing Files with Vi

Certification Objective

Most Linux distributions ship with a variant of Vi known as Vim, or "Vi Improved." The information presented here applies to both Vi and Vim.

Vi was the first full-screen text editor written for Unix. It's designed to be small and simple. Vi is small enough to fit on tiny, floppy-based emergency boot systems. For this reason alone, Vi is worth learning; you may need to use it in an emergency recovery situation. Vi is, however, a bit strange, particularly if you're used to GUI text editors. To use Vi, you should first understand the three modes in which it operates. Once you understand those modes, you can begin learning about the text-editing procedures Vi implements. You'll also examine how to save files and exit Vi.

Understanding Vi Modes

At any given moment, Vi is running in one of three modes:

Command mode This mode accepts commands, which are usually entered as single letters. For instance, i and a both enter insert mode, although in somewhat different ways, as described shortly, and o opens a line below the current one.

Ex mode To manipulate files (including saving your current file and running outside programs), you use ex mode. You enter ex mode from command mode by typing a colon (:), typically directly followed by the name of the ex-mode command you want to use. After you run the ex-mode command, Vi returns automatically to command mode.

Insert mode You enter text in insert mode. Most keystrokes result in text appearing on the screen. One important exception is the Esc key, which exits insert mode and returns to command mode.

If you're not sure what mode Vi is in, press the Esc key. Doing so returns you to command mode, from which you can reenter insert mode, if necessary.

Unfortunately, terminology surrounding Vi modes is inconsistent at best. For instance, command mode is sometimes referred to as normal mode, and insert mode is sometimes called edit mode or entry mode. Ex mode often isn't described as a mode at all, but is referred to as *colon commands*.

Exploring Basic Vi Text-Editing Procedures

As a method of learning Vi, consider the task of editing `/etc/fstab` to add a new hard disk—the same task described earlier, in "Exploring Basic `nano` Text-Editing Procedures," with respect to `nano`. Listing 11.1 (in that earlier section) shows the original `/etc/fstab` file used in this example. If you want to follow along, enter it using a text editor with which you're already familiar, and save it to a file on your disk. Don't edit your *real* `/etc/fstab` file as a learning exercise; a mistake could render your system unbootable the next time you reboot. Instead, work on a copy of this file.

The first step to using Vi is to launch it and have it load the file. In this example, type **`vi fstab`** while in the directory holding the file. The result should resemble Figure 11.2, which shows Vi displaying the original file in a text session. Although not shown in Figure 11.2, some systems display a line of tildes (~) down the left side of the screen to indicate the end of the file. The bottom line shows the status of the last command—an implicit file load command that loaded 16 lines and 611 characters (`16L, 611C`) from the `fstab` file.

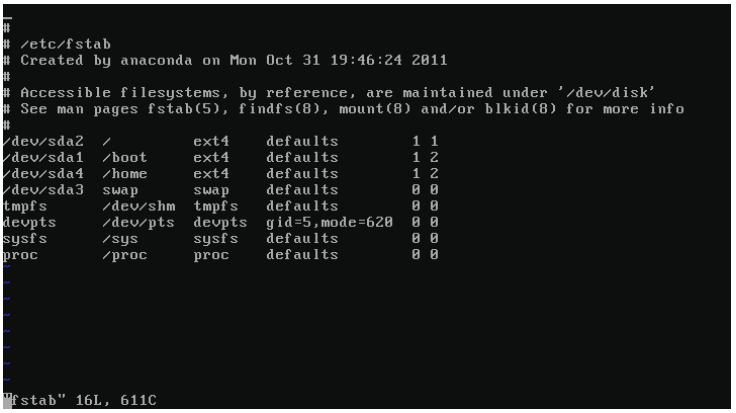


FIGURE 11.2 The last line of a Vi display is a status line that shows messages from the program.

As with `nano`, you can add a new entry to `fstab` using Vi either by typing it in its entirety or by duplicating an existing line and then modifying one copy. To do it the first way, follow these steps:

1. Move the cursor to the beginning of the `/dev/sda3` lines by using the arrow keys.
2. Press the O (letter O, not number 0) key. This opens a new line immediately below the current line, moves the cursor to that line, and enters insert mode.
3. Type a new entry, such as the following:

```
/dev/sdb1   /home2     ext4     defaults         0 0
```

4. Press the Esc key to return to command mode.

To practice making changes by modifying an existing entry, follow these steps:

1. Move the cursor to the beginning of the `/dev/sdb1` line you just created by using the arrow keys, if necessary; you should see the cursor resting on the first `/` of `/dev/sdb1`.
2. You must now *yank* one line of text. This term is used much as *copy* is used in most text editors—you copy the text to a buffer from which you can later paste it back into the file. To yank text, you use the `yy` command, preceded by the number of lines you want to yank. Thus, type **`1yy`** (*do not* press the Enter key, though). The `dd` command works much like `yy`, but it deletes the lines as well as copying them to a buffer. Both `yy` and `dd` are special cases of the `y` and `d` commands, respectively, which yank or delete text in amounts specified by the next character, as in `dw` to delete the next word.
3. Move the cursor to the line *before* the one where you want the new line to appear.
4. Type **`p`** (again, without pressing the Enter key). Vi pastes the contents of the buffer starting on the line after the cursor. The file should now have two identical `/dev/sdb1` lines. The cursor should be resting at the start of the second one. If you want to paste the text into the document starting on the line *before* the cursor, use an uppercase `P` command.
5. Move the cursor to the `1` in `/dev/sdb1` on the line you've just pasted. You're about to begin customizing this line.
6. Until now, you've operated Vi in command mode. You can use any of several commands to enter insert mode. At this point, the most appropriate is `R`, which enters insert mode so that it's configured for

text replacement rather than insertion. If you prefer to insert text rather than overwrite it, you can use `i` or `a` (the latter advances the cursor one space, which is sometimes useful at the end of a line). For the purposes of these instructions, type **R** to enter insert mode. You should see `-- REPLACE --` appear in the status line.

7. Type 2 to change `/dev/sdb1` to `/dev/sdb2`.
8. Use the arrow keys to move the cursor to the `2` in `/home2`. You must modify this mount point name.
9. Type 3 to change `/home2` to `/home3`.
10. Exit insert mode by pressing the Esc key.
11. Save the file and quit by typing **:wq**. This is an ex-mode command, as described shortly. (The `ZZ` command is equivalent to `:wq`.)



You can make more extensive changes to the `fstab` file, if you like, but be sure to work from a *copy* of the file!

Many additional commands are available that you may want to use in some situations. Here are some of the highlights:

Change case Suppose you need to change the case of a word in a file. Instead of entering insert mode and retyping the word, you can use the tilde (~) key in command mode to change the case. Position the cursor on the first character you want to change, and press ~ repeatedly until the task is done.

Undo To undo any change, type **u** in command mode.

Open text In command mode, typing **o** (a lowercase letter O) opens text—that is, it inserts a new line immediately below the current one and enters insert mode on that line.

Search To search forward for text in a file, type **/** in command mode, followed immediately by the text you want to locate. Typing **?** searches backward rather than forward.

Change text The `c` command changes text from within command mode. You invoke it much like the `d` or `y` command, as in `cw` to change the next word or `cc` to change an entire line.

Go to a line The `G` key brings you to a line that you specify. The `H` key "homes" the cursor—that is, it moves the cursor to the top line of the screen. The `L` key brings the key to the bottom line of the screen.

Replace globally To replace all occurrences of one string with another, type **:%s/*original*/*replacement***, where *original* is the original string and *replacement* is its replacement. Change `%` to a starting line number, comma, and ending line number to perform this change on a small range of lines.

Vi offers a great deal more depth than is presented here; the editor is quite capable, and some Linux users are very attached to it. Entire books have been written about Vi. Consult one of these, or a Vi Web page like `http://www.vim.org`, for more information.

Saving Your Changes from Vi

To save changes to a file, type **`:w`** from command mode. This enters ex mode and runs the `w` ex-mode command, which writes the file using whatever filename you specified when you launched Vi. Related commands enable other functions:

Edit a new file The `:e` command edits a new file. For instance, **`:e /etc/inittab`** loads `/etc/inittab` for editing. Vi won't load a new file unless the existing one has been saved since its last change or unless you follow `:e` with an exclamation mark (`!`).

Include an existing file The `:r` command includes the contents of an old file in an existing one.

Execute an external command The ex-mode command `:!` executes the external command that you specify. For instance, typing **`:!ls`** runs `ls`, enabling you to see what files are present in the current directory.

Quit Use the `:q` command to quit the program. As with `:e`, this command won't work unless changes have been saved or you append an exclamation mark to the command (as in `:q!`).

You can combine ex commands such as these to perform multiple actions in sequence. For instance, typing **`:wq`** writes changes and then quits from Vi.

Using Configuration File Conventions

Most Linux system configuration files reside in the `/etc` directory tree, but your user configuration files reside in your home directory, usually in files or subdirectories with names that begin with a dot (`.`), such as `.bashrc`, which is the Bash configuration file. Configuration file formats vary from one program to another. Therefore, whether you want to edit a system configuration file or a personal configuration file, you're well advised to consult the documentation for the program in question. There are, however, some features that are common to many configuration files.

One common purpose for configuration files is to set certain program features to particular values. For instance, you might want to tell a mail server program



The Bash configuration file is actually a *shell script*—that is, a program. Chapter 12, "Creating Scripts," describes how to create and edit Bash scripts.

to send outgoing mail to another computer, or you might want to tell a GUI program about its window's height and width. Most configuration files do this by setting a named *variable* to a *value*, by placing both on the same line, sometimes with an equal sign between them. For instance, the main Postfix mail server configuration file (`/etc/postfix/main.cf`) uses an equal sign to set the `relayhost` variable to a value, as in:

```
relayhost = franklin.pangaea.edu
```

The `relayhost` variable, when set, tells Postfix to relay outgoing mail through the named system—`franklin.pangaea.edu`, in this example. Another example is the `/etc/ntp.conf` file, which holds data for the Network Time Protocol (NTP) daemon, which synchronizes your computer's clock with other computers' clocks. You can specify an outside clock source with the `server` variable:

```
server tardis.example.com
```



Don't be too concerned with Postfix or NTP configuration specifically; I merely used them as convenient examples.

This line tells NTP to use `tardis.example.com` as a time source. Whether or not a configuration file requires an equal sign is a design decision of the program's author. Some configuration files use still more complex formats, but assigning values to variables is quite common in configuration files.

The `/etc/fstab` file used earlier as an example file doesn't really set variables in either of these ways, but it does link various types of data together, one set per line. The information includes the device filename for a partition, its mount point, its filesystem, and its mount options. Such collections are common in configuration files, but the details differ. For instance, `/etc/fstab` separates elements with *whitespace*—that is, spaces or tabs. Some other files use other separating characters, such as the colon (`:`) that separates user account information in the `/etc/passwd`, `/etc/shadow`, and `/etc/group` files. (Chapter 13 describes these files in more detail.)

Another common feature of configuration files is *comments*—that is, English (or other natural language) text that describes the purpose of a file, a block of lines, or even a single line. Typically, comments begin with one or more special characters. The comment then continues until the end of the line or, sometimes, until another character or set of characters terminates the comment.

The single most common comment character in configuration files is a hash mark (`#`), which is used to begin a comment. Such comments typically end with the end of the line on which they appear. For instance, the `/etc/ntp.conf` file might include the following lines:

```
# You do need to talk to an NTP server or two (or three).
# server wells.pangaea.edu
server tardis.example.com
```

The first line is a comment that describes the purpose of the following lines. The second line illustrates an important secondary use of comments: By adding the comment character to the start of the line, you disable it. Thus, the `wells.pangaea.edu` computer is not configured as an NTP server. Commenting out configuration file entries in this way is useful when you're testing a setup—perhaps you suspect problems with `wells.pangaea.edu`, so you comment its entry out as a temporary measure. You can easily uncomment the line by removing the hash mark and the following space.

Some servers use other characters or strings to denote comments. For instance, the sendmail e-mail server uses the string `dnl` to denote comments. If you're examining an unfamiliar configuration file, you should keep an eye out for comments. Even if you don't know the usual comment character used in the file, you can usually figure it out by spotting plain English text in the file.

Knowing these two conventions alone (variable assignment and comments) can help you determine what a configuration file is doing, and therefore enable you to change how it works.

Some programs provide default configuration files that have quite wordy comments. Such files can help you figure out how to adjust the program's configuration.

Editing Formatted Text Files

As noted earlier, in "Understanding the Role of Text Files," formatted text files use ASCII or Unicode formats, in conjunction with conventions for a layout language, to describe text that's more complex than ASCII or Unicode alone can handle. A common formatted text file format is the Hypertext Markup Language (HTML), which is used on the Web—most Web pages use HTML.

Although a full tutorial on how to edit HTML, or any other formatted text file, is beyond the scope of this book, you may want to take a quick look at this type of file so that you can at least begin to understand it, and perhaps make small changes to such files. To that end, Listing 11.2 shows a simple HTML file.

Listing 11.2: A sample HTML file

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<HTML>
<HEAD>
<TITLE>Linux Essentials</TITLE>
</HEAD>
<BODY BGCOLOR="#FFFFFF" TEXT="#000000">
<CENTER><H1 ALIGN="CENTER">Linux Essentials</H1></CENTER>
<P>The <I>Linux Essentials</I> book covers the essential
tools for using Linux!</P>
</BODY>
</HTML>
```

HTML tags are case-insensitive. An extended version of HTML, known as XHTML, uses case-sensitive lowercase tags.

In HTML, most formatting information appears in angle brackets (<>). These *tags* are usually paired, with the ending tag carrying the same name as the beginning one, except that the ending tag's name includes a slash (/). For instance, <P> begins a paragraph and </P> ends it. Likewise, <I> begins italic text and </I> ends it.

In Listing 11.2, the first line identifies the version of HTML the document uses. This enables the Web browser to adjust the way it handles the file. The <HTML> tag begins the bulk of the document. The <HEAD> and <BODY> tags identify two key parts of the document—the header that normally appears in a Web browser's title bar and the main body of the document.

Some opening tags include additional information. For instance, the <BODY> tag in Listing 11.2 specifies the background color (BGCOLOR="#FFFFFF") and text color (TEXT="#000000").

The purpose of describing this is not to make you experts on HTML—it's to point out that you can often parse and even edit formatted text files with minimal knowledge. For instance, you can change the text in the body of Listing 11.2, add new paragraphs by mimicking the way the first one is specified, and so on. If you make a mistake, though, the results can be strange. For instance, if you type Listing 11.2 into a text editor, save it, and point a Web browser to the file, you can see what it looks like as a Web page. If you neglect to type the </I> tag in the paragraph, though, all the text after the book's title will be italicized. If you were to enter another paragraph after the first one, it would be italicized, too! The Web browser will keep on italicizing text until it encounters a </I> tag.

The Essentials and Beyond

Plain text files, which encode text using ASCII or Unicode, are important on most computer platforms, but they're particularly important on Linux. This is because many of Linux's configuration files use plain text formats, so understanding how to use an editor, such as `pico`, nano, or Vi, to edit these files enables you to edit a wide variety of configuration files. You should also understand the conventions used by certain configuration file formats, such as common features of configuration files and formatted text files. Even a basic understanding of what you're likely to find will help you edit such files when the need arises.

Suggested Exercises

- Launch nano to create a new file and type in a complete paragraph from this chapter. Proofread the text and correct any typos you find. (If you found none, congratulations! Now create a few "errors" and correct them.)

(Continues)

Although the word "computer" suggests a device for performing mathematical computations, much of a computer's job has more to do with data storage and retrieval than with computation. This chapter covers some of the tools you can use to search, extract, and archive data.

The chapter begins with a look at *regular expressions*, which are a way to describe patterns you might want to look for in data files. You can use regular expressions with many commands, two of which (`find` and `grep`) I describe in more detail. This chapter also covers tools that you can use to redirect programs' input and output, which is a useful trick in many situations. Finally, I describe some tools for creating archive files, which can be useful in transferring many files over a network or in creating backups.

- **Using regular expressions**
- **Searching for and extracting data**
- **Redirecting input and output**
- **Archiving data**

Using Regular Expressions

Certification Objective

Many Linux programs employ regular expressions, which are tools for expressing patterns in text. Regular expressions are similar in principle to the wildcards that can be used to specify multiple filenames, as described in Chapter 7, "Managing Files." At their simplest, regular expressions can be plain text without adornment. Certain characters are used to denote patterns, though.

Documentation sometimes uses the abbreviation *regexp* to refer to a regular expression.

Two forms of regular expression are common: basic and extended. Which form you must use depends on the program; some accept just one form, but others can use either type, depending on the options passed to the program. (Some programs use their own minor or major variants on either of these classes of regular expression.) The differences between basic and extended regular expressions are complex and subtle, but the fundamental principles of both are similar.

The simplest type of regular expression is an alphabetic or alphanumeric string, such as `HWaddr` or `Linux3`. These regular expressions match any string of the same size or longer that contains the regular expression. For instance, the `HWaddr` regular expression matches `HWaddr`, `This is the HWaddr`, and `The HWaddr is unknown`. The real strength of regular expressions comes in the use of non-alphanumeric characters, which activate advanced matching rules. The most powerful basic regular expression features include the following:

Bracket expressions Characters enclosed in square brackets (`[]`) constitute bracket expressions, which match any one character within the brackets. For instance, the regular expression `b[aeiou]g` matches the words `bag`, `beg`, `big`, `bog`, and `bug`. Including a carat (`^`) after the opening square bracket matches against any character *except* the ones specified. For instance, `b[^aeiou]g` matches `bbg` or `bAg` but not `bag` or `beg`.

Range expressions A range expression is a variant on a bracket expression. Instead of listing every character that matches, range expressions list the start and end points separated by a dash (`-`), as in `a[2-4]z`. This regular expression matches `a2z`, `a3z`, and `a4z`.

Any single character The dot (`.`) represents any single character except a newline. For instance, `a.z` matches `a2z`, `abz`, `aQz`, or any other three-character string that begins with `a` and ends with `z`.

Start and end of line The carat (`^`) represents the start of a line, and the dollar sign (`$`) denotes the end of a line.

Repetition A full or partial regular expression may be followed by a special symbol to denote repetition of the matched item. Specifically, an asterisk (`*`) denotes zero or more matches. The asterisk is often combined with the dot (as in `.*`) to specify a match with any substring. For instance, `A.*Lincoln` matches any string that contains `A` and `Lincoln`, in that order—`Abe Lincoln` and `Abraham Lincoln` are just two possible matches.

Escaping If you want to match one of the special characters, such as a dot, you must *escape* it—that is, precede it with a backslash (`\`). For instance, to

match a computer hostname (say, `twain.example.com`), you must escape the dots, as in `twain\.example\.com`.

Extended regular expressions add more features you can use to match in additional ways:

Additional repetition operators These operators work like an asterisk, but they match only certain numbers of matches. Specifically, a plus sign (`+`) matches one or more occurrences and a question mark (`?`) specifies zero or one match.

Multiple possible strings The vertical bar (`|`) separates two possible matches; for instance, `car|truck` matches either `car` or `truck`.

Parentheses Ordinary parentheses (`()`) surround subexpressions. Parentheses are often used to specify how operators are to be applied; for example, you can put parentheses around a group of words that are concatenated with the vertical bar to ensure that the words are treated as a group, any one of which may match, without involving surrounding parts of the regular expression.

Whether you use basic or extended regular expressions depends on which form the program supports. For programs such as `grep` that support both, you can use either; which you choose is mostly a matter of personal preference. Note that a regular expression that includes characters associated with extended regular expressions will be interpreted differently depending on which type you're using. Thus, it's important to know which type of regular expression a program supports, or how to select which type to use if the program supports both types.

Regular expression rules can be confusing, particularly when you're first introduced to them. Some examples of their use, in the context of the programs that use them, will help. The next section provides such examples, with reference to the `find` and `grep` programs.

Technically, of these three programs, only `grep` uses regular expressions, although `find` supports pattern matching using a similar mechanism.

Searching for and Extracting Data

Two commands, `grep` and `find`, both use regular expressions and are helpful in locating data. The `grep` utility locates files by scanning their contents. The `grep` program also returns some of the data included in files, which can be useful if you want to extract just a little data from a file or from a program's output. As its name suggests, `find` locates files. It uses surface features, such as the filename and the file's date stamps. Another command, `wc`, provides basic statistics on text files.

Using *grep*

Certification Objective

If you don't specify a filename, `grep` uses standard input. This can be useful with pipelines, as described shortly, in "Redirecting Input and Output."

The `grep` command searches for files that contain a specified string and returns the name of the file and (if it's a text file) a line of context for that string. To use `grep`, you type the command's name, an optional set of options, a regular expression, and an optional filename specification. The `grep` command supports a large number of options, the most common of which appear in Table 10.1.

TABLE 10.1 Common `grep` options

Option (long form)	Option (short form)	Description
`--count`	`-c`	Instead of displaying the lines that contain matches to the regular expression, display the number of lines that match.
`--file=`*`file`*	`-f` *`file`*	This option takes pattern input from the specified file rather than from the command line. The `fgrep` command is a shortcut for this option.
`--ignore-case`	`-i`	You can perform a case-insensitive search, rather than the default case-sensitive search, by using the `-i` or `--ignore-case` option.
`--recursive`	`-R or -r`	This option searches in the specified directory and all subdirectories rather than simply the specified directory. You can use `rgrep` rather than specify this option.
`--extended-regexp`	`-E`	The `grep` command uses basic regular expressions by default. To use an extended regular expression, you can pass this option. Alternatively, you can call `egrep` rather than `grep`; this variant command uses extended regular expressions by default.

A simple example of `grep` uses a regular expression with no special components:

```
$ grep -r eth0 /etc/*
```

This example finds all the files in `/etc` that contain the string `eth0` (the identifier for the first Ethernet device on most distributions). Because the example includes the `-r` option, it searches recursively, so `grep` searches files in subdirectories of `/etc` as well as those in `/etc` itself. For each matching text file, the line that contains the string is printed.

Ramping up a bit, suppose you want to locate all the files in `/etc` that contain the string `eth0` or `eth1`. You can enter the following command, which uses a bracket expression to specify both variant devices:

```
$ grep eth[01] /etc/*
```

A still more complex example searches all files in `/etc` that contain the number `127` and, later on the same line, the hostname `twain.example.com` or `localhost`. This task requires using several of the regular expression features. Expressed using extended regular expression notation, the command looks like this:

```
$ grep -E "127.*(twain\.example\.com|localhost)" /etc/*
```

This command illustrates another feature you may need to use: shell quoting. Because the shell uses certain characters, such as the vertical bar and the asterisk, for its own purposes, you must enclose certain regular expressions in quotes lest the shell attempt to parse the regular expression as shell commands.

You can use `grep` in conjunction with commands that produce a lot of output in order to sift through that output for the material that's important to you. (Several examples throughout this book use this technique.) For example, suppose you want to find the process ID (PID) of a running `xterm`. You can use a pipe to send the result of a `ps` command through `grep`:

```
# ps ax | grep xterm
```

The result is a list of all running processes called `xterm`, along with their PIDs. You can even do this in series, using `grep` to further restrict the output on some other criterion, which can be useful if the initial pass still produces too much output. For instance, suppose you want to find kernel messages relating to low-speed Universal Serial Bus (USB) devices. As noted in Chapter 9, "Using Programs and Processes," you can use `dmesg` to review kernel messages, but it produces copious output. You can pass this output through `grep` to find messages related to USB devices or to messages that include the word `low`, but either search alone still produces a lot of irrelevant hits. Using `grep` twice

Ordinary users can't read some files in `/etc`. Thus, if you type this command as a non-`root` user, you'll see error messages relating to `grep`'s inability to open files.

Type this command on your computer. It should match at least one file, `/etc/hosts`. Type it again *without* the `-E` option to see its operation using basic regular expressions.

The upcoming section, "Redirecting Input and Output," describes pipes in more detail.

in succession successfully limits the output to lines that include both search strings:

```
$ dmesg | grep -i usb | grep low
usb 4-1.1: new low speed USB device number 3 using uhci_hcd
usb 4-1.2.2: new low speed USB device number 5 using uhci_hcd
usb 4-1.2.2: new low speed USB device number 8 using uhci_hcd
```

This differs from using a single `grep` command with both search terms (using a regular expression such as `usb.*low`) in that the order in which the search terms appear is important for the single search but not when using `grep` twice. You can also use different options when you call `grep` twice, as in the use of `-i` when searching on `usb` but not when searching on `low` in this example.

Using *find*

Certification Objective

The `find` utility implements a brute-force approach to finding files. This program finds files by searching through the specified directory tree, checking filenames, file creation dates, and so on to locate the files that match the specified criteria. Because of this method of operation, `find` tends to be slow, but it's very flexible and is very likely to succeed, assuming the file for which you're searching exists. To use `find`, type its name, optionally followed by a pathname and a series of options, some of which use specifications that are similar to regular expressions.

In practice, you must use a pathname or a search criterion with `find`, and often both.

You can specify one or more paths in which `find` should operate; the program will restrict its operations to these paths. The `man` page for `find` includes information about its search criteria, but Table 10.2 summarizes common criteria.

TABLE 10.2 Common `find` search criteria

Option	Description
`-name` *pattern*	You can search for files using their names with this option. Doing so finds files that match the specified *pattern*. This *pattern* is not technically a regular expression, but it does support many regular expression features.
`-perm` *mode*	If you need to find files that have certain permissions, you can do so by using the `-perm` *mode* expression. The *mode* may be expressed either symbolically or in octal form. If you precede mode with a `+`, `find` locates files in which any of the specified permission bits are set. If you precede *mode* with a `-`, `find` locates files in which *all* the specified permission bits are set.

Chapter 15, "Setting Ownership and Permissions," covers file permissions.

TABLE 10.2 *(Continued)*

Option	Description
`-size` *n*	You can search for files based on size with this expression. Normally, *n* is specified in 512-byte blocks, but you can modify this by trailing the value with a letter code, such as *c* for characters (bytes) or k for kilobytes.
`-group` *name*	This option searches for files that belong to the specified group.
`-gid` *GID*	This expression searches for files whose group ID (GID) is set to *GID*.
`-user` *name*	This option searches for files that are owned by the specified user.
`-uid` *UID*	You can search for files by user ID (UID) number using this option.
`-maxdepth` *levels*	If you want to search a directory and, perhaps, some limited number of subdirectories, you can use this expression to limit the search.

There are many variant and additional options; `find` is a very powerful command. As an example of its use, consider the task of finding all C source code files, which normally have names that end in `.c`, in all users' home directories. If these home directories reside in `/home`, you might issue the following command:

```
# find /home -name "*.c"
```

The result will be a listing of all the files that match the search criteria.

If you lack permission to list a directory's contents, `find` will return that directory name and the error message `Permission denied`.

Using *wc*

Certification Objective

A file's size in bytes, as revealed by `ls` or searched for using `find`, can be a useful metric. This size value isn't always the most useful one for text files, though. For instance, you might need to know how many words or lines are in a text file—say because you're writing a 1,000-word essay or you want to know how many pages a text document will consume when printed at 52 lines per page. The `wc` utility provides this information. By default, it displays a count of lines (newline characters, to be precise), words, and bytes for each file you pass it. For instance,

you can discover this information for all the files with `.txt` extensions in your home directory:

```
$ wc ~/*.txt
  1471   1839  71039 /home/rodsmith/500.txt
    31    180   1236 /home/rodsmith/Commands Outputs.txt
  1012   8909  69689 /home/rodsmith/parts.txt
    19     33    383 /home/rodsmith/Problem.txt
     4      4     31 /home/rodsmith/stuff.txt
```

This output reveals that the file `500.txt` contains 1,471 lines, 1,839 words, and 71,039 bytes, and provides similar statistics for the remaining files.

You can pass options to limit or expand `wc`'s output, as summarized in Table 10.3. Of the options in Table 10.3, `-c`, `-l`, and `-w` are the defaults, so typing **`wc *file.txt*`** is equivalent to typing **`wc -clw *file.txt*`**. The program's `man` page describes a few more options, but the ones in Table 10.3 are the ones you're most likely to use.

TABLE 10.3 Common `wc` options

Option (long form)	Option (short form)	Description
`--bytes`	`-c`	Displays the file's byte count
`--chars`	`-m`	Displays the file's character count
`--lines`	`-l`	Displays the file's newline count
`--words`	`-w`	Displays the file's word count
`--max-line-length`	`-L`	Displays the length of the longest line in the file

Some text files use multi-byte encodings, meaning that one character can consume more than one byte. Thus, `-c` and `-m` may not produce identical results, although they often do.

Be aware that `wc` works correctly on plain text files, but it may produce incorrect or even nonsensical results on formatted text files, such as Hypertext Markup Language (HTML) files or word processor files. You're better off using a word processor or other specialized editor to find the number of words and other statistics for such files.

Redirecting Input and Output

If the output of a program becomes annoying, or if you want to save it for future reference, you can *redirect* it to a file. You can also redirect the input to a program from a file. Although input redirection may sound strange, some

programs rely on this feature to enable them to process data, such as raw text files fed through a program that searches the text for patterns. In addition to redirecting output to files or input from files, you can pass one program's output to another one as its input. A related technique involves the `xargs` command, which enables you to generate command-line options from files or other programs' output.

Using Basic Redirection Operators

Certification Objective

Redirection is achieved with the help of *redirection operators*, which are short strings that appear after the command and its arguments. Table 10.4 shows the most common redirection operators. Be aware that output comes in two types:

Standard output This is normal program messages.

Standard error This contains error messages.

Having two types of output enables them to be separated so that error messages don't confuse programs that might be expecting certain types of input from another program.

TABLE 10.4 Common redirection operators

Redirection operator	Effect
>	Creates a new file containing standard output. If the specified file exists, it's overwritten.
>>	Appends standard output to the existing file. If the specified file doesn't exist, it's created.
2>	Creates a new file containing standard error. If the specified file exists, it's overwritten.
2>>	Appends standard error to the existing file. If the specified file doesn't exist, it's created.
&>	Creates a new file containing both standard output and standard error. If the specified file exists, it's overwritten.
<	Sends the contents of the specified file to be used as standard input.
<<	Accepts text on the following lines as standard input.
<>	Causes the specified file to be used for both standard input and standard output.

As an example of redirecting output, consider a `grep` command to search for information on a particular user in all the configuration files in `/etc`. Without redirection, such a command might look like this:

```
$ grep david /etc/*
```

This command will return a series of output lines like the following:

```
Binary file /etc/aliases.db matches
/etc/group:wheel:x:10:root,david
/etc/group:audio:x:18:mythtv,david,pulse
/etc/group:cdrom:x:19:haldaemon,david
```

Such output can be quite lengthy, and you might want to peruse it later. To do so, you could redirect the output like this:

```
$ grep david /etc/* > david-in-etc.txt
```

If you then wanted to see the output, you could use `cat`:

```
$ cat david-in-etc.txt
```

Many programs have options, set on the command line or in configuration files, that affect how verbose their output is. Check a program's `man` page to learn about such options.

In this example you haven't gained anything compared to simply typing **`grep david /etc/*`**, but you might in other cases. For instance, suppose a command is producing copious error messages. You might then redirect standard error to a file and load the file into a text editor so that you can browse through it, search for strings that might be relevant, and so on, even as you attempt to run the command, or a modified version of it, once more.

This example illustrates how standard error and standard output are separate. If you type **`grep *david* /etc/*`** as a normal user (perhaps substituting your own username for *`david`*), you're likely to see output such as that shown earlier, specifying the files in which your username appears; however, you're also likely to see error messages, since you lack permission to read some of the files in `/etc`:

```
grep: /etc/securetty: Permission denied
grep: /etc/shadow: Permission denied
```

The information on the files in which *`david`* appears is shown via standard output, but the errors are shown via standard error. If you're not interested in the errors, you can redirect them to `/dev/null`—a device file that serves as a "dumping ground" for data you want to discard:

```
$ grep david /etc/* 2> /dev/null
```

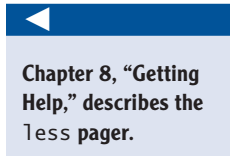
Likewise, if you redirect standard output to a file but do *not* redirect standard error, you'll see the error messages on your screen, but the file you create (such as `david-in-etc.txt` from the earlier command) will not contain the error

messages. You may want to try all the different types of output redirection using **`grep *david* /etc/*`** to get a feel for how they work.

Using Pipes

Another type of redirected output is a *pipe* or *pipeline*. In a pipe, the standard output from one program is redirected as the standard input to a second program. You create a pipe by using a vertical bar (|), which is usually a shifted character above the Enter key, between the two commands. Pipelines can be useful when applied in various ways. For instance, you might pipe the lengthy output of a program through the `less` pager, which enables you to page up and down through the output, or use `grep` to search for keywords in the output:

Certification Objective




```
$ dmesg | grep sda
```

The `dmesg` command displays messages from the Linux kernel about hardware and other low-level activities. Thus, the result of these two commands is that you'll see any kernel messages that include the string `sda`—an identifier associated with the first hard disk.

Generating Command Lines

Sometimes you'll find yourself constructing a series of commands that are similar to each other but not similar enough to enable you to use their normal options to substitute a single command. For instance, suppose you want to remove every file in a directory tree with a name that ends in a tilde (~). (This filename convention denotes backup files created by certain text editors.) With a large directory tree, this task can be daunting; the usual file-deletion command (`rm`, described in more detail in Chapter 7) doesn't provide an option to search for and delete every file in a directory tree that matches such a specific criterion. One command that can do the search part of the job, though, is `find`, which is described in more detail earlier. If you could combine the output of `find` to create a series of command lines using `rm`, the task would be solved. This is precisely the purpose of the `xargs` command, which builds a command from its standard input. The basic syntax for this command is as follows:

Certification Objective

```
xargs [options] [command [initial-arguments]]
```

The *`command`* is the command you want to execute, and *`initial-arguments`* is a list of arguments you want to pass to the command. The *`options`* are `xargs` options; they aren't passed to *`command`*. When you run `xargs`, it runs *`command`* once for every word passed to it on standard input, adding that word to the

argument list for *command*. If you want to pass multiple options to the command, you can protect them by enclosing the group in quotation marks.

For instance, consider the task of deleting all those backup files, denoted by tilde characters. You can do this by piping the output of `find` to `xargs`, which then calls `rm`:

```
$ find ./ -name "*~" | xargs rm
```

The first part of this command (`find ./ -name "*~"`) finds all the files in the current directory (`./`) or its subdirectories with a name that ends in a tilde (`*~`). This list is then piped to `xargs`, which adds each one to its own `rm` command.

A tool that's similar to `xargs` in many ways is the backtick (`` ` ``), which is a character to the left of the 1 key on most keyboards. The backtick is *not* the same as the single quote character (`'`), which is located to the right of the semicolon (`;`) on most keyboards.

Text within backticks is treated as a separate command whose results are substituted on the command line. For instance, to delete those backup files, you can type the following command:

```
$ rm `find ./ -name "*~"`
```

Archiving Data



Another archive program, `cpio`, is sometimes used in Linux. It's similar in principle to `tar`, but different in operational details.

A file archiving tool collects a group of files into a single "package" file that you can easily move around on a single system; back up to a recordable DVD, USB flash drive, tape, or other removable media; or transfer across a network. Linux supports several archiving commands, the most prominent being `tar` and `zip`. In addition to understanding these commands, you should be familiar with the consequences of using compression with them.

Using *tar*

The `tar` program's name stands for "tape archiver." Despite this fact, you can use `tar` to archive data to your hard disk or other media, not just to tapes. In fact, *tarballs* (archive files created by `tar` and typically compressed with `gzip` or `bzip2`) are often used for transferring multiple files between computers in one step, such as when distributing source code.

The `tar` program is a complex package with many options, but most of what you'll do with the utility can be covered with a few common commands. Table 10.5 lists the primary `tar` commands, and Table 10.6 lists the qualifiers that modify what the commands do. Whenever you run `tar`, you use exactly one command, and you usually use at least one qualifier.

TABLE 10.5 `tar` commands

Command	Abbreviation	Description
`--create`	`c`	Creates an archive
`--concatenate`	`A`	Appends `tar` files to an archive
`--append`	`r`	Appends non-`tar` files to an archive
`--update`	`u`	Appends files that are newer than those in an archive
`--diff` or `--compare`	`d`	Compares an archive to files on disk
`--list`	`t`	Lists an archive's contents
`--extract` or `--get`	`x`	Extracts files from an archive

Unlike most single-letter program options in Linux, you can use single-letter `tar` commands and qualifiers without a leading dash (`-`).

TABLE 10.6 `tar` qualifiers

Qualifier	Abbreviation	Description
`--directory` *dir*	`C`	Changes to directory `dir` before performing operations
`--file` [*host*`:`]*file*	`f`	Uses the file called `file` on the computer called `host` as the archive file
`--listed-incremental` *file*	`g`	Performs an incremental backup or restore, using `file` as a list of previously archived files
`--one-file-system`	(none)	Backs up or restores only one filesystem (partition)
`--multi-volume`	`M`	Creates or extracts a multi-tape archive
`--tape-length N`	`L`	Changes tapes after `N` kilobytes

(Continues)

TABLE 10.6 *(Continued)*

Qualifier	Abbreviation	Description
`--same-permissions`	`p`	Preserves all protection information
`--absolute-paths`	`P`	Retains the leading `/` on filenames
`--verbose`	`v`	Lists all files read or extracted; when used with `--list`, displays file sizes, ownership, and time stamps
`--verify`	`W`	Verifies the archive after writing it
`--exclude` *file*	(none)	Excludes *file* from the archive
`--exclude-from` *file*	`X`	Excludes files listed in *file* from the archive
`--gzip` or `--ungzip`	`z`	Processes an archive through `gzip`
`--bzip2`	`j` (some older versions used `I` or `y`)	Processes an archive through `bzip2`
`--xz`	`J`	Processes an archive through `xz`

Of the commands listed in Table 10.5, the most commonly used are `--create`, `--extract`, and `--list`. The most useful qualifiers from Table 10.6 are `--file`, `--listed-incremental`, `--one-file-system`, `--same-permissions`, `--gzip`, `--bzip2`, `--xz`, and `--verbose`. If you fail to specify a filename with the `--file` qualifier, `tar` will attempt to use a default device, which is often (but not always) a tape device file.



The `tar` utility preserves Linux's ownership and permission information, even when the archive is stored on a filesystem that doesn't support such metadata.

As an example, consider archiving and compressing the `my-work` subdirectory of your home directory to a USB flash drive mounted at `/media/flash`. The following command will do the trick:

```
$ tar cvfz /media/flash/my-work.tgz ~/my-work
```

If you then transfer this drive to another system, mount it at `/media/usb`, and want to extract the archive, you can do so with another command:

```
$ tar xvfz /media/usb/my-work.tgz
```

This command creates a subdirectory called `my-work` in the current working directory and populates it with the files from the archive. If you don't know what's in an archive, it's a good practice to examine it with the `--list` command before extracting its contents. Although common practice creates tarballs that store files within a single subdirectory, sometimes tarballs drop many files in the current working directory, which can make them difficult to track down if you run the command in a directory that already has many files.

Using Compression

In Linux, the `gzip`, `bzip2`, and `xz` programs all compress individual files. For instance, you might compress a large graphics file like this:

```
$ bzip2 biggraphics.tiff
```

The result is a file with a name like the original but with the addition of a new filename extension to identify it as a compressed format. In this specific case, the result would be `biggraphics.tiff.bz2`. Most graphics programs won't read files compressed in this way, though. To use a file that's been compressed, you must uncompress it with a matching program. Table 10.7 summarizes the compression programs, their matching uncompression programs, and the filename extensions they create. As a general rule, `gzip` provides the least compression and `xz` the most.

Once you've compressed a text file, you won't be able to search it with `grep` without first uncompressing it. The `zgrep` variant can search files compressed with `gzip`, though.

TABLE 10.7 Compression and uncompression programs and filename extensions

Compression program	Uncompression program	Filename extension
`gzip`	`gunzip`	`.gz`
`bzip2`	`bunzip2`	`.bz2`
`xz`	`unxz`	`.xz`

The `tar` program provides explicit support for all three of these compression standards, and compressed tarballs often have their own unique filename extensions (`.tgz`, `.tbz` or `.tb2`, and `.txz` for tarballs compressed with `gzip`, `bzip2`, and `xz`, respectively). Using two extensions on tarballs, as in `archive.tar.bz2`, is also common. When you compress a tarball, even by using the `-z`, `-j`, or `-J` option to `tar`, the compression program works on the tarball with all its files rather than to the individual files within the tarball. This can improve the

compression ratio compared to compressing individual files and then bundling them together, but it makes it harder to extract data from a file if it becomes damaged.

Compression works better with some file types than with others. Typically, plain text files compress extremely well, binary program files compress moderately well, and pre-compressed data (such as most video file formats) compress poorly or may even expand in size when compressed again. You should be aware of this fact to know whether to apply compression to your archives. For instance, if you back up a Linux installation, which consists of program and configuration files, without user data, the result is likely to consume about half the space it does on your hard disk. On the other hand, if you want to back up a directory containing MP3 audio files or JPEG graphics, both of which are pre-compressed, you're better off not applying compression.

The `gzip`, `bzip2`, and `xz` compression programs all apply *lossless* compression, meaning that the data recovered by uncompressing the file is identical to what went into it. Some graphics, audio, and audio-visual file formats apply *lossy* compression, in which some data are discarded. When done properly and at low to moderate levels of compression, you'll be hard-pressed to notice the loss, since the algorithms discard data that humans have a difficult time perceiving. When a user applies lossy algorithms aggressively, though, we do notice the effect. Lossy compression tools should never be used on program files, system configuration files, or most user data files; any loss in such files could be disastrous. That's why `tar` supports only lossless compression tools.

Using *zip*

Outside of the Unix and Linux world, the zip file format is a common one that fills a role similar to a compressed tarball. Linux provides the `zip` command to create zip files and the `unzip` utility to extract files from a zip archive. Zip files typically have filename extensions of `.zip`.

In most cases, you can create a zip archive by passing the utility the name of a target zip file followed by a filename list:

```
$ zip newzip.zip afile.txt figure.tif
```

This command creates the `newzip.zip` file, which holds the `afile.txt` and `figure.tif` files. (The original files remain on your disk.) In some cases you'll need to use options to `zip` to achieve the desired results. Table 10.8 summarizes the most important `zip` options; however, the program supports many more. Consult its `man` page for details.

TABLE 10.8 Common `zip` options

Option (long form)	Option (short form)	Description
N/A	`-0` through `-9`	Sets the amount of compression; `-0` applies no compression, `-1` applies minimal (but fast) compression, and so on through `-9`, which applies maximal (but slow) compression.
`--delete`	`-d`	Deletes the specified files from the archive file.
`--encrypt`	`-e`	Encrypts the archive with a password. (`zip` prompts you for this password.)
`--freshen`	`-f`	Updates files in an archive if they've changed since the original archive's creation.
`--fix` or `--fixfix`	`-F` or `-FF`	Performs repairs on a damaged archive file. The `--fix`/`-F` option performs minimal repairs, whereas `--fixfix`/`-FF` is more thorough.
`--filesync`	`-FS`	Updates files in an archive if they've changed since the original archive's creation and deletes files from the archive if they've been deleted on the filesystem.
`--grow`	`-g`	Appends files to an existing archive file.
`--help`	`-h` or `-?`	Displays basic help information.
`--move`	`-m`	Moves files into the zip archive; that is, the original files are deleted.
`--recurse -paths`	`-r`	Includes files and subdirectories inside directories you specify.
`--split -size` *size*	`-s` *size*	Creates a potentially multi-file archive, with each file no larger than *size* bytes. (A `k`, `m`, `g`, or `t` can be appended to the size to specify larger units.)

(Continues)

TABLE 10.8 *(Continued)*

Option (long form)	Option (short form)	Description
`--exclude` *files*	`-x` *files*	Excludes the specified files.
`--symlinks`	`-y`	Includes symbolic links as such. (Ordinarily, `zip` includes the linked-to files.)

Of the options in Table 10.8, the `-r` option is probably the most important, at least if you want to compress an entire directory tree. If you fail to use this option, your archive will contain no subdirectories. Given the speed of modern CPUs, using `-9` on a regular basis also makes sense.

To uncompress and extract files in a `zip` archive file, you can use the `unzip` program:

```
$ unzip anarchive.zip
```

Zip files typically contain "loose" files in the main directory, so you should generally extract zip archives in an empty subdirectory you create for this purpose.

This example uncompresses the files in the `anarchive.zip` file into the current directory. Like `zip`, `unzip` supports a large number of options, the most important of which appear in Table 10.9.

TABLE 10.9 Common `unzip` options

Option	Description
`-f`	Freshens files from the archive; that is, extracts only those files that exist on the main filesystem and that are newer in the archive than on the main filesystem
`-l`	Lists files in the archive but does not extract them
`-p`	Extracts files to a pipeline
`-t`	Tests the integrity of files in the archive
`-u`	Updates files; similar to `-f`, but also extracts files that don't exist on the filesystem

(Continues)

TABLE 10.9 *(Continued)*

Option	Description
`-v`	Lists files in the archive in a more verbose format than `-l` does
`-L`	Converts filenames to lowercase if they originated on an uppercase-only OS, such as DOS
`-n`	Never overwrites existing files
`-o`	Overwrites existing files without prompting

As a general rule, using `unzip` without any options except for the input filename works well; however, you might want to use one or more of its options on occasion. The `-l` option is particularly useful for examining the archive's contents without extracting it.

The Essentials and Beyond

Managing your files often requires locating them, and tools such as `grep` and `find` help you with this task. The `grep` utility in particular makes use of regular expressions, which provide a way to describe patterns you might want to find in files or in the output of another program. You can redirect such output into `grep` (or other programs or files) using redirection operators, and many Linux command-line tools and techniques rely on such redirection. The `tar` and `zip` programs both enable you to create archive files that hold many other files. In fact, the tarballs that `tar` creates are a common means of distributing source code and even binary programs between Linux computers.

Suggested Exercises

- Use `find` and `grep` to locate files in your own directory and on the Linux computer at large. For instance, try locating references to your own username in configuration files in `/etc`.
- Use `gzip`, `bzip2`, and `xz` to compress a couple instances of files of various types, such as text files and digital photos. What file types compress well? Which compression tool works best for each file type?

(Continues)

5. Ownerships & Permissions

As a multi-user OS, Linux provides tools to help you secure your files against unwanted access—after all, you wouldn't want another user to accidentally (or intentionally) read personal files or even delete your files! Linux handles these tasks through two features of files and directories: their *ownership* and their *permissions*. Every file has an associated owner (that is, an account with which it's linked), and also an associated group. Three sets of permissions define what the file's owner, members of the file's group, and all other users can do with the file. Thus, ownership and permissions are intertwined, although you use different text-mode commands to manipulate them. (GUI tools often combine the two, as described in this chapter.)

- **Setting ownership**
- **Setting permissions**

The *set user ID (SUID)* and *set group ID (SGID)* permission bits, described in Chapter 16, "Navigating the Linux Filesystem," can modify the account and group associated with a program.

Setting Ownership

Linux's security model is based on that of Unix, which was designed as a multi-user OS. This security model therefore assumes the presence of multiple users on the computer, and provides the means to associate individual files with the users who create them—that is, files have *owners*. You should thoroughly understand this concept, and with that understanding, you can change a file's ownership, using either a GUI file manager or a text-mode shell.

Ownership also applies to running programs (that is, *processes*). Most programs you run are tied to the account you used to launch them. This identity, in conjunction with the file's ownership and permissions, determines whether a program may modify a file.

Understanding Ownership

Chapter 13, "Understanding Users and Groups," and Chapter 14, "Creating Users and Groups," described Linux's system of accounts. These accounts are the basis of file ownership. Specifically, every file has an owner—an account with which it's associated. This association occurs by means of the account's *user ID (UID)* number. Every file is also associated with a *group* by means of a *group ID (GID)* number.

As described later, in the section "Setting Permissions," access to the file is controlled by means of permissions that can be set independently for the file's owner, the file's group, and all other users of the computer. As `root`, you can change the owner and group of any file. The file's owner can also change the file's group, but only to a group to which the user belongs.

The same principles of ownership apply to directories as apply to files: directories have owners and groups. These can be changed by `root` or, to a more limited extent, by the directory's owner.

Cross-Installation UIDs and GIDs

You may use multiple Linux installations, either dual-booting on one computer or installed on multiple computers. If so, and if you transfer files from one installation to another, you may find that the ownership of files seems to change as you move them around. The same thing can happen with non-Linux Unix-like OSs, such as Mac OS X. The reason is that the filesystems for these OSs store ownership and group information using UID and GID numbers, and a single user or group can have different UID or GID numbers on different computers, even if the name associated with the account or group is identical.

This problem is most likely to occur when using native Linux or Unix filesystems to transfer data, including both disk-based filesystems (such as Linux's ext2fs or Mac OS X's HFS+) or the Network File System (NFS) for remote file access. This problem is less likely to occur if you use a non-Linux/Unix filesystem, such as the File Allocation Table (FAT) or the New Technology File System (NTFS) for disks, or the Server Message Block/Common Internet File System (SMB/CIFS; handled by Samba in Linux) for network access.

If you run into this problem, several solutions exist, but many of them are beyond the scope of this book. One you can use, though, is to change the UID or GID mappings on one or more installations so that they all match. Chapter 14 describes how to change a user's UID number with `usermod`, and to change a group's GID number with `groupmod`. When transferring data via removable disks, using FAT or NTFS can be a simple solution, provided you don't need to preserve Unix-style permissions on the files.

Setting Ownership in a File Manager

As described in Chapter 4, "Using Common Linux Programs," a *file manager* enables you to manipulate files. You're probably familiar with file managers in Windows or Mac OS X. Linux's ownership and permissions are different from those of Windows, though, so you may want to know how to check on, and perhaps change, ownership features using a Linux file manager. As noted in Chapter 4, you have a choice of several file managers in Linux. Most are similar in broad strokes but differ in some details. In this section, I use Nautilus, the GNU Network Object Model Environment (GNOME) file manager, as an example.

If you want to change the file's owner, you must run Nautilus as `root`, but you can change the file's group to any group to which you belong as an ordinary user. The procedure to perform this task as `root` is as follows:

1. Launch a terminal window.
2. In the terminal window, type **`su`** to acquire `root` privileges.
3. In the terminal window, type **`nautilus`** to launch Nautilus. You can optionally include the path to the directory in which you want Nautilus to start up. If you don't include a path, Nautilus will begin by displaying the contents of the `/root` directory.
4. Locate the file whose ownership you want to adjust and right-click it.
5. In the resulting menu, select Properties. The result is a Properties dialog box.
6. In the Properties dialog box, click the Permissions tab. The result resembles Figure 15.1.
7. To change the file's owner, select a new owner in the Owner field. This action is possible only if you run Nautilus as `root`.
8. To change the file's group, select a new group in the Group field. If you run Nautilus as an ordinary user, you will be able to select any group to which you belong, but if you run Nautilus as `root`, you will be able to select any group.
9. When you've adjusted the features you want to change, click Close.

If you want to change a file's group but not its owner, and if you're a member of the target group, you can launch Nautilus as an ordinary user. You can then pick up the preceding procedure at step 4.

If you're using Ubuntu, you may instead need to use `sudo` to launch Nautilus.

The `/root` directory is the `root` account's home directory.

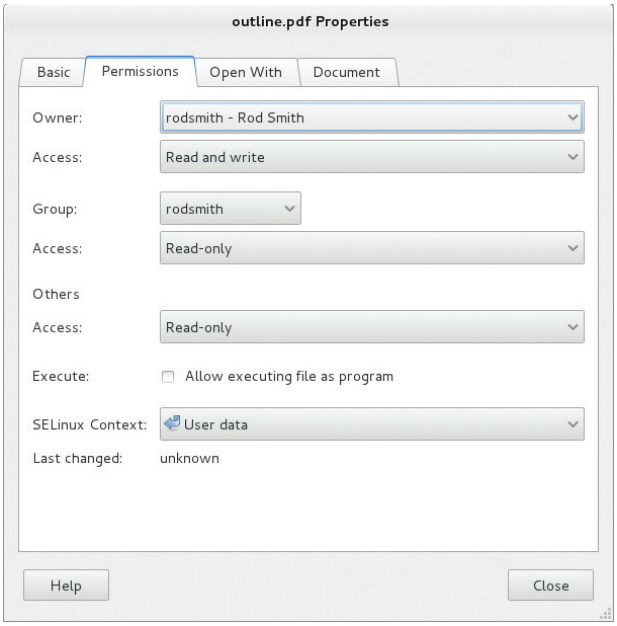


FIGURE 15.1 **Linux file managers give you access to the file's ownership and permission metadata.**

You should be *extremely* cautious about running Nautilus as `root`. If you forget you're running this program as `root`, you can easily create new files as `root`, which will require additional `root`-privilege actions to correct by changing file ownership. It's also easy to accidentally delete critical system files as `root` that you could not delete as an ordinary user. For these reasons, I recommend that you use a text-mode shell to adjust file ownership. The change in the prompt makes it easier to notice you're running as `root`, and if you're used to using a GUI, you're less likely to launch additional programs as `root` from a text-mode shell than from Nautilus.

Setting Ownership in a Shell

Certification Objective

The command to change the ownership of a file in the preferred text-mode manner is `chown`. In its most basic form, you pass it the name of a file followed by a username:

The chown **command's name stands for *change owner*.**

```
# chown bob targetfile.odf
```

This example gives ownership of `targetfile.odf` to `bob`. You can change the file's principal owner and its group with a single command by separating the owner and group with a colon (`:`):

```
# chown bob:users targetfile.odf
```

This example gives ownership of `targetfile.odf` to `bob` and associates the file with the `users` group. To change the group without changing the owner, you can omit the owner, leaving the colon and group name:

```
$ chown :users targetfile.odf
```

Alternatively, you can use the `chgrp` command, which works in the same way but changes *only* the group and does not require the colon before the group name:

Certification Objective

```
$ chgrp users targetfile.odf
```

Note that the commands used to change the owner require `root` privileges, whereas you can change the group as an ordinary user—but only if you own the file and belong to the target group.

The `chown` and `chgrp` commands both support a number of options that modify what they do. The most useful of these is `-R` (or `--recursive`), which causes a change in ownership of all the files in an entire directory tree. For instance, suppose that the user `mary` has left a company, and an existing employee, `bob`, must access her files. If `mary`'s home directory was `/home/mary`, you might type:

```
# chown -R bob /home/mary
```

This command gives `bob` ownership of the `/home/mary` directory, all the files in the `/home/mary` directory, including all its subdirectories, the files in the subdirectories, and so on. To make the transition a bit easier for `bob`, you might also want to move `mary`'s former home directory into `bob`'s home directory.

Setting Permissions

File ownership is meaningless without some way to specify what particular users can do with their own or other users' files. That's where permissions enter the picture. Linux's permission structure is modeled after that of Unix, and it requires a bit of explanation before you tackle the issue. Once you understand the basics, you can begin modifying permissions, using either a GUI file manager or a text-mode shell. You can also set default permissions for new files you create.

Understanding Permissions

To understand Unix (and hence Linux) permissions, you may want to begin with the display created by the `ls` command, which lists the files in a directory, in conjunction with its `-l` option, which creates a long directory listing that

Certification Objective

includes files' permissions. For instance, to see a long listing of the file `test`, you might type:

```
$ ls -l test
-rwxr-xr-x  1 rodsmith users      111 Apr 13 13:48  test
```

This line consists of several sections, which provide assorted pieces of information on the file:

Permissions The first column (`-rwxr-xr-x` in this example) is the file's permissions, which are of interest at the moment.

Number of links The next column (`1` in this example) shows the number of hard links to the file—that is, the number of unique filenames that may be used to access the file.

Username The next column (`rodsmith` in this example) identifies the file's owner by username.

Group name The file's group (`users` in this example) appears next.

File size This example file's size is quite small—111 bytes.

Time stamp The time stamp (`Apr 13 13:48` in this example) identifies the time the file was last modified.

Filename Finally, `ls -l` shows the file's name—`test` in this example.

The string that begins this output (`-rwxr-xr-x` in this example) is a symbolic representation of the permissions string. Figure 15.2 shows how this string is broken into four parts:

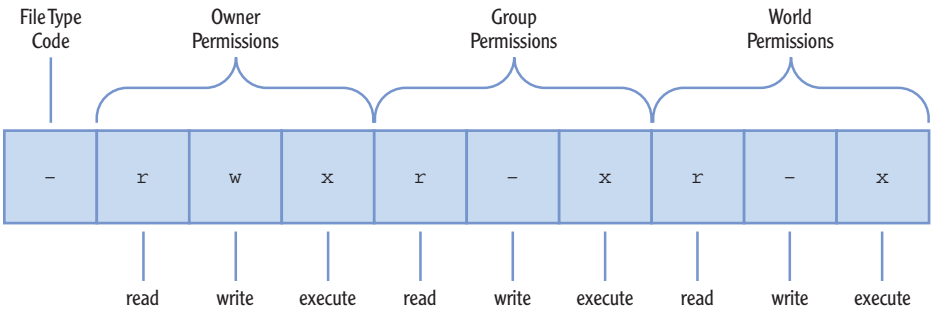


FIGURE 15.2 A symbolic representation of file permissions is broken into four parts.

The file type code The first character is the file type code, which represents the file's type, as summarized in Table 15.1. This type character is sometimes omitted from descriptions when the file type is not relevant or when it's identified in some other way.

Chapter 6, "Getting to Know the Command Line," introduced the `ls` command, and describes additional `ls` options.

Chapter 7, "Managing Files," describes links in more detail.

Most of the files you'll manipulate are normal files, directories, and symbolic links.

Owner permissions These permissions determine what the file's owner can do with the file.

Group permissions These permissions determine what members of the file's group (who aren't its owner) can do with the file.

World (or "other") permissions These permissions determine what users who aren't the file's owner or members of its group can do with the file

TABLE 15.1 Linux file type codes

Code	Name	Meaning
-	Normal data file	May be text, an executable program, graphics, compressed data, or just about any other type of data.
d	Directory	Disk directories are files, but they contain filenames and pointers to those named files' data structures.
l	Symbolic link	The file contains the name of another file or directory. When Linux accesses the symbolic link, it tries to read the linked-to file.
p	Named pipe	A pipe enables two running Linux programs to communicate with each other in a one-way fashion.
s	Socket	A socket is similar to a named pipe, but it permits network and bidirectional links.
b	Block device	A file that corresponds to a hardware device to and from which data is transferred in blocks of more than one byte. Disk devices (hard disks, floppies, CD-ROMs, and so on) are common block devices.
c	Character device	A file that corresponds to a hardware device to and from which data is transferred in units of one byte. Examples include parallel and RS-232 serial port devices.

Setting the execute bit on a non-program file doesn't turn it into a program, of course; it just indicates that a user may run a file that is a program.

In each of the three sets of permissions, the string identifies the presence or absence of each of three types of access: read, write, and execute. Read and write permissions are fairly self-explanatory. If the execute permission is present, it means that the file may be run as a program. The absence of the permission is denoted by a dash (-) in the permission string. The presence of the permission is indicated by the letter `r` for read, `w` for write, or `x` for execute.

Thus, the example permission string `-rwxr-xr-x` means that the file is a normal data file and that its owner, members of the file's group, and all other users can read and execute the file. Only the file's owner has write permission to the file.

Another representation of permissions is possible. It's compact but a bit confusing: It takes each of the three permissions groupings of the permission string (omitting the file type code) and converts it into a number from 0 to 7 (that is, a *base 8* or *octal* number). The result is a three-digit octal number. Each number is constructed by starting with a value of 0 and then:

- Adding 4 if read permissions are present
- Adding 2 if write permissions are present
- Adding 1 if execute permissions are present

The resulting three-digit code represents permissions for the owner, the group, and the world. Table 15.2 shows some examples of common permissions and their meanings.

These procedures actually involve binary numbers and logical, not arithmetic, operations. The arithmetic description is easier to understand, though.

There are 512 possible combinations of permissions, so Table 15.2 is incomplete. It shows the most common and useful combinations, though.

TABLE 15.2 Example permissions and their interpretations

Permission string	Octal code	Meaning
`rwxrwxrwx`	777	Read, write, and execute permissions for all users.
`rwxr-xr-x`	755	Read and execute permission for all users. The file's owner also has write permission.
`rwxr-x---`	750	Read and execute permission for the owner and group. The file's owner also has write permission. Other users have no access to the file.
`rwx------`	700	Read, write, and execute permissions for the file's owner only; all others have no access.
`rw-rw-rw-`	666	Read and write permissions for all users. No execute permissions for anybody.
`rw-rw-r--`	664	Read and write permissions for the owner and group. Read-only permission for all others.

(Continues)

TABLE 15.2 *(Continued)*

Permission string	Octal code	Meaning
`rw-rw----`	660	Read and write permissions for the owner and group. No world permissions.
`rw-r--r--`	644	Read and write permissions for the owner. Read-only permission for all others.
`rw-r-----`	640	Read and write permissions for the owner, and read-only permission for the group. No permission for others.
`rw-------`	600	Read and write permissions for the owner. No permission for anybody else.
`r--------`	400	Read permission for the owner. No permission for anybody else.

Several special cases apply to permissions:

Directory execute bits Directories use the execute bit to grant permission to search the directory. This is a highly desirable characteristic for directories, so you'll almost always find the execute bit set when the read bit is set.

Directory write permissions Directories are files that are interpreted in a special way. As such, if a user can write to a directory, that user can create, delete, or rename files in the directory, even if the user isn't the owner of those files and does not have permission to write to those files.

The usual rules for writing to directories can be modified with the *sticky bit*, which is described in Chapter 16.

Symbolic links Permissions on symbolic links are always 777 (`rwxrwxrwx`, or `lrwxrwxrwx`, to include the file type code). This access applies only to the link file itself, not to the linked-to file. In other words, all users can read the contents of the link to discover the name of the file to which it points, but the permissions on the linked-to file determine its file access. Changing the permissions on a symbolic link affects the linked-to file.

`root` Many of the permission rules don't apply to `root`. The superuser can read or write any file on the computer—even files that grant access to nobody (that is, those that have 000 permissions). The superuser still needs an execute bit set to run a program file.

Setting Permissions in a File Manager

The procedure for setting permissions in a file manager is similar to that for setting the ownership of a file:

- You normally adjust these settings using the same dialog box used to adjust ownership, such as the Nautilus dialog box shown earlier in Figure 15.1.
- You don't need to be `root` to adjust the permissions of files you own.
- You should use `root` access for this job only on files you don't own.

Details vary in other file managers, but the principles are the same as those described here.

As seen in Figure 15.1, there are three Access items, associated with the Owner, the Group, and Others:

- The Owner item provides two options: Read-Only and Read and Write.
- The Group and Others items both provide Read-Only and Read and Write plus the None option. You can use these options to set the read and write permission bits on your file.

Nautilus requires setting the execute bit separately, by checking the Allow Executing File As Program box. This check box sets all three execute permission bits; you can't control execute permission more precisely with Nautilus. You also can't adjust the execute permissions on directories with Nautilus.

Setting Permissions in a Shell

Certification Objective

In a text-mode shell, you can use `chmod` to change permissions. This command is rather complex, mostly because of the complex ways that permissions may be changed. You can specify the permissions in two forms: as an octal number or in a symbolic form, which is a set of codes related to the string representation of the permissions.

The `chmod` command's name stands for *change mode*, *mode* being another name for permissions.

The octal representation of the mode is the same as that described earlier and summarized in Table 15.2. For instance, to change permissions on `report.tex` to `rw-r--r--`, you can issue the following command:

```
$ chmod 644 report.tex
```

A symbolic mode, by contrast, consists of three components:

- A code indicating the permission set you want to modify—`u` for the user (that is, the owner), `g` for the group, `o` for other users, and `a` for all permissions

- A symbol indicating whether you want to add (+), delete (-), or set the mode equal to (=) the stated value
- A code specifying what the permission should be, such as the common `r`, `w`, or `x` symbols, or various others for more advanced operations

Using symbolic modes with `chmod` can be confusing, so I don't describe them fully here; however, you should be familiar with a few common types of use, as summarized in Table 15.3. Symbolic modes are more flexible than octal modes because you can specify symbolic modes that modify existing permissions, such as adding or removing execute permissions without affecting other permissions. You can also set only the user, group, or world permissions without affecting the others. With octal modes, you must set all three permission bits equal to a value that you specify.

As with the `chown` and `chgrp` commands, you can use the `-R` (or `--recursive`) option to `chmod` to have it operate on an entire directory tree.

TABLE 15.3 Examples of symbolic permissions with `chmod`

Command	Initial permissions	End permissions
`chmod a+x bigprogram`	`rw-r--r--`	`rwxr-xr-x`
`chmod ug=rw report.tex`	`r--------`	`rw-rw----`
`chmod o-rwx bigprogram`	`rwxrwxr-x`	`rwxrwx---`
`chmod g-w,o-rw report.tex`	`rw-rw-rw-`	`rw-r-----`

Setting the umask

The *user mask*, or *umask*, determines the default permissions for new files. The umask is the value that is *removed* from 666 (`rw-rw-rw-`) permissions when creating new files, or from 777 (`rwxrwxrwx`) when creating new directories. For instance, if the umask is 022, then files will be created with 644 permissions by default, and new directories will have 755 permissions. Note that the removal operation is not a simple subtraction but a bitwise removal. That is, a 7 value in a umask removes the corresponding `rwx` permissions; but for files, for which the starting point is `rw-`, the result is `---` (0), not –1 (which is meaningless).

You can adjust the umask with the `umask` command, which takes the umask value, as in **`umask 022`**. Typically, this command appears in a system configuration file, such as `/etc/profile`, or in a user configuration file, such as `~/.bashrc`.

6. Programs & Processes

Computers are dynamic and multi-purpose machines; they do a variety of jobs using many tools. This chapter describes the ways you can manage these tools. One aspect of this task is installing, uninstalling, and upgrading software packages. Another aspect of software management is in managing programs once they're running. (Running programs are often called *processes*.) Finally, this chapter covers *log files*, which record details of what running programs do—particularly programs that run automatically and in the background.

- **Understanding package management**
- **Understanding the process hierarchy**
- **Identifying running processes**
- **Using log files**

Understanding Package Management

Package management is an area of Linux that varies a lot from one distribution to another. Nonetheless, certain principles are common across most Linux distributions, so I describe these principles, followed by some of the basics of the two major Linux package management systems. I then describe how to manage packages using both the RPM Package Management (RPM; a recursive acronym) and Debian package systems.

Linux Package Management Principles

Certification Objective

If you've installed software in Windows, you're likely familiar with the procedure of double-clicking on an installer program, which places all the files

associated with a program where they should go. A Windows software installer is similar to a Linux package file, but there are differences. Linux packages have the following characteristics:

Many program packages depend on *library* packages; libraries provide code that can be used by many programs.

- Each package is a single file that can be stored on a disk or transmitted over the Internet.
- Linux package files, unlike Windows installers, are not programs; packages rely on other programs to do the work of installing the software.
- Packages contain *dependency* information—that is, they can tell the packaging software what other packages or individual files must be installed in order for the package to work correctly.
- Packages contain version information, so that the packaging software can tell which of two packages is more recent.
- Packages contain architecture information to identify the CPU type (*x*86, *x*86-64, PowerPC, and so on) for which they're intended. A separate code identifies packages that are architecture-independent, such as fonts and desktop themes.
- *Binary* packages (that is, those that contain executable programs that are CPU-specific) are typically built from *source* packages (which contain source code that a programmer can understand). It's possible to build a new binary package, given the source package, which can be useful in some unusual circumstances.

You can compile and install software from source code manually, without using a packaging tool. This advanced topic is beyond the scope of this book.

Certification Objective

The package software maintains a database of information about installed packages (the *package database*). This information includes the names and version numbers of all the installed packages, as well as the locations of all the files installed from each package. This information enables the package software to quickly uninstall software, establish whether a new package's dependencies have been met, and determine whether a package you're trying to install has already been installed and, if so, whether the installed version is older than the one you're trying to install.

Packages can, and frequently do, contain files that will be installed to many directories on the computer. This fact makes tracking package contents critical.

Understanding Package Systems

As noted earlier, two package systems, RPM and Debian, are common, although others exist as well. These systems differ in various technical details, as well as

in the commands used to manage packages and in the format of the package files they use. You cannot install a Debian package on an RPM-based system, or vice versa. Indeed, installing a package intended for one distribution on another is a bit risky even when they use the same package type. This is because a non-native package may have dependencies that conflict with the needs of native packages.

Table 1.1 in Chapter 1, "Selecting an Operating System," summarizes some features of several popular Linux distributions, including the package system each uses.

Originally, package systems worked locally—that is, to install a package on your computer you would first have to download a package file from the Internet or in some other way. Only then could you use a local command to install the package. This approach, however, can be tedious when a package has many dependencies—you might attempt an installation, find unmet dependencies, download several more packages, find that one or more of *them* has unmet dependencies, and so on. By the time you've tracked down all of these depended-upon packages, you might need to install a dozen or more packages. Thus, modern distributions provide network-enabled tools to help automate the process. These tools rely on network software *repositories*, from which the tools can download packages automatically. The network-enabled tools vary from one distribution to another, particularly among RPM-based distributions.

Certification Objective

In practice, then, the process of managing software in Linux involves using text-mode or GUI tools to interface with a software repository. A typical software installation task works something like this:

1. You issue a command to install a program.
2. The software locates dependencies of the specified program and notifies you of any additional software that must be installed.
3. You issue a final approval for software installation (or decide against it, in which case the process stops).
4. The software downloads all the necessary packages.
5. The software installs all the packages.

You can configure most distributions to use local media instead of or in addition to Internet repositories.

Upgrading software works in a similar way, although upgrades are less likely to require downloading depended-upon packages. Removing software can be done entirely locally, of course. Many distributions automatically check with their repositories from time to time and notify you when updates are available. Thus, you can keep your system up to date by clicking a few buttons when you're prompted to do so. As an example, Figure 9.1 shows Fedora 16's Software Update utility, which shows a list of available updates.

Immediately after installing a distribution, you may find that a large number of updates are available.

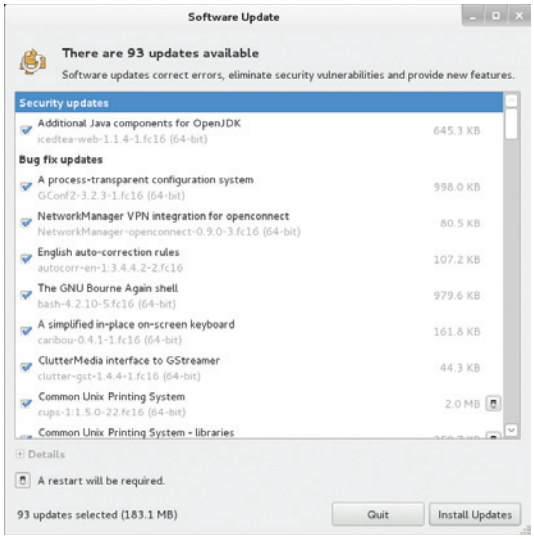


FIGURE 9.1 **Most Linux distributions tell you when updates are available for your software.**

Package management necessarily involves `root` access, which is described in more detail in Chapter 13, "Understanding Users and Groups." If you follow the automatic prompts to update your software, you can keep the system up to date by entering the `root` password, or on some distributions your regular password, when the update software prompts for it.

Managing RPM Systems

RPM-based distributions include Red Hat, Fedora, CentOS, SUSE Enterprise, openSUSE, and Mandriva. The basic tool for installing software on these distributions is the text-mode `rpm` command. This program works on local files, though; to use a network repository, you must use another tool, which varies by distribution:

- Red Hat, Fedora, and CentOS use the text-mode `yum` or various GUI front-ends to it, such as Yumex.
- SUSE Enterprise and openSUSE use `zypper` or a GUI front-end such as YaST 2.
- Mandriva uses the text-mode `urpmi` or the GUI Rpmdrake.

Because of the variability between these distributions, particularly for network-enabled updates, providing a complete description of all of these tools is impractical here. Fortunately, the GUI tools are easy to use and accessible. Even the

text-mode tools are fairly straightforward, although you may need to consult their `man` pages to learn the details. Typically, they use logical subcommands, such as `install` to install a package, as in:

```
# yum install yumex
```

You might use this command to install the GUI Yumex tool on a Red Hat, Fedora, or CentOS system. Similarly, you can remove a specific package by using the `remove` subcommand or upgrade all of a computer's packages by using `upgrade`:

```
# yum remove zsh
# yum upgrade
```

These examples upgrade the computer and then remove the `zsh` package. Both of these commands will produce a number of lines of output, and you may be asked to verify their actions. Consult the `man` page for `yum` (or whatever package management software your distribution uses) to learn more about this tool.

If you need to deal with RPM package files directly, you should be aware that they have filename extensions of `.rpm`. These files also usually include codes for architecture type (such as `i386` or `x86_64`), and often codes for the distribution for which they're intended (such as `fc16` for Fedora 16). For instance, `samba-3.6.1-77.fc16.x86_64.rpm` is a package file for the `samba` package, version 3.6.1, release 77, for Fedora 16, on the *x*86-64 platform.

If you want to both upgrade software and remove packages, it's generally best to remove software first. This can obviate some downloads, reducing the upgrade time.

Managing Debian Systems

The Debian GNU/Linux distribution created its own package system, and distributions based on Debian, such as Ubuntu and Mint, use the same system. Atop the basic Debian package system lies the Advanced Package Tool (APT), which provides access to network repositories.

The `dpkg` command is the lowest-level interface to the Debian package system; it's roughly equivalent to the `rpm` utility on RPM-based systems. Several tools provide text-mode and graphical interfaces atop `dpkg`, the most important of these being the text-mode `apt-get` and the GUI Synaptic. As their names imply, `apt-get` and Synaptic provide access to network repositories via APT. Figure 9.2 shows Synaptic in use.

Debian package files have names that end in `.deb`. Like RPM packages, these names typically include codes for the software version and architecture (such as `i386` or `amd64`). For instance, `samba_3.6.1-3_amd64.deb` is a Debian package file for the `samba` package, version 3.6.1, revision 3, for AMD64 (*x*86-64) CPUs. You can install such files using `dpkg` or `apt-get`, or you can use `apt-get` to download a package and its dependencies from the Internet, using its `install` command, as in:

```
# apt-get install samba
```

Third-party implementations of APT for many RPM-based distributions also exist. See `http://apt4rpm.sourceforge.net` for details. At least one RPM-based distribution, PCLinuxOS, uses APT natively.

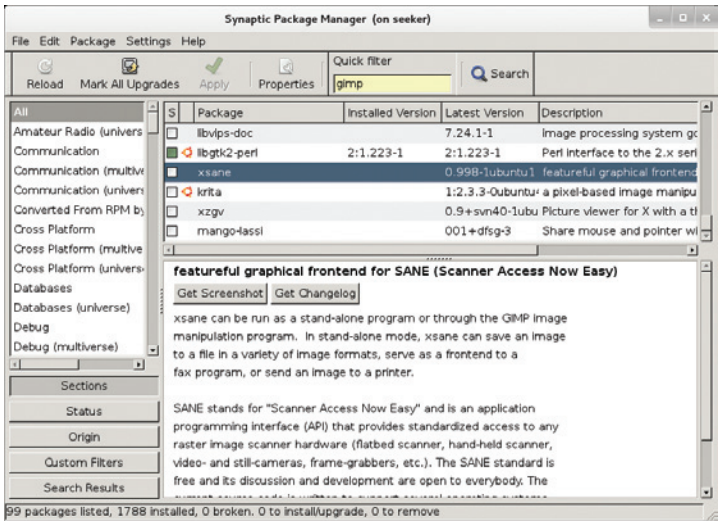


FIGURE 9.2 **Synaptic enables you to search for, select, install, and uninstall software on Debian-based systems.**

As with RPM packages, you can remove packages or upgrade your computer's software, too:

```
# apt-get remove zsh
# apt-get upgrade
```

APT is a powerful tool, as is the underlying `dpkg`. You should consult these programs' `man` pages to learn more about how to use these programs.

Understanding the Process Hierarchy

The Linux *kernel* is the core of a Linux installation. The kernel manages memory, provides software with a way to access the hard disk, doles out CPU time, and performs other critical low-level tasks. The kernel is loaded early in the boot process, and it's the kernel that's responsible for managing every other piece of software on a running Linux computer.

One of the many ways that the kernel imposes order on the potentially chaotic set of running software is to create a sort of hierarchy. When it boots, the kernel runs just one program—normally `/sbin/init`. The `init` process is then responsible for starting all the other basic programs that Linux must run, such as the programs that manage logins and always-up servers. Such programs, if launched directly by `init`, are called its *children*. The children of `init` can in turn launch their own children. This happens when you log into Linux. The process that launched a given process is called its *parent*.

You can change which program runs as the first process by adding the `init=` option to your boot loader's kernel option line, as in `init=/bin/bash` to run `bash`.

The result of this system is a treelike hierarchy of processes, as illustrated in Figure 9.3. ("Trees" in computer science are often depicted upside down.) Figure 9.3 shows a small subset of the many processes that run on a typical Linux installation: just a few processes associated with a text-mode login, including the `login` tool that manages logins, a couple of `bash` shells, and a few user programs. A working Linux system will likely have dozens or hundreds of running processes. The one on which I'm typing these words has 213 processes going at once!

Certification Objective

Occasionally, a process will terminate but leave behind children. When this happens, `init` "adopts" those child processes.

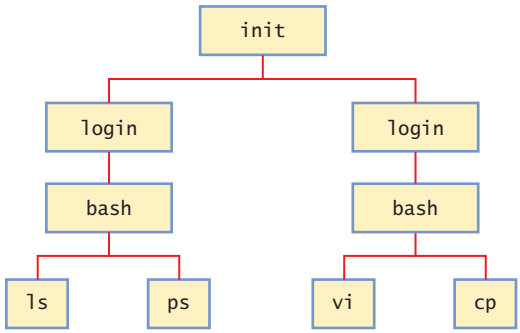


FIGURE 9.3 **Linux processes are arranged in a hierarchical tree.**

Every process has associated with it a process ID (PID) number. These numbers begin with 1, so `init`'s PID is normally 1. Each process also has a parent process ID (PPID) number, which points to its parent. Many of the tools for managing processes rely on these numbers, and particularly on the PID number.

Internally, the kernel maintains process information in the *process table*. Tools such as `ps` and `top` (described shortly) enable you to view and manipulate this table.

Identifying Running Processes

Before you can manage processes, you must be able to identify them. The `ps` and `top` utilities can help you identify processes. In either case, you can search for processes in various ways, such as by name or by resource use. You may also want to identify how much memory your processes are consuming, which you can do with the `free` command.

Using *ps* to Identify Processes

Certification Objective

The simplest tool for identifying processes is `ps`, which produces a process listing. Listing 9.1 shows an example of `ps` in action. In this example, the `-u` option restricts output to processes owned by the specified user (`rodsmith`), while `--forest` creates a display that shows parent/child relationships.

Listing 9.1: Output of `ps -u rodsmith --forest`

```
$ ps -u rodsmith --forest
  PID TTY          TIME CMD
 2451 pts/3    00:00:00 bash
 2551 pts/3    00:00:00 ps
 2496 ?        00:00:00 kvt
 2498 pts/1    00:00:00 bash
 2505 pts/1    00:00:00  \_ nedit
 2506 ?        00:00:00      \_ csh
 2544 ?        00:00:00          \_ xeyes
19221 ?        00:00:01 dfm
```

The version of `ps` used in most Linux distributions combines features from several earlier `ps` implementations. The result is a huge selection of sometimes redundant options.

Listing 9.2 shows a second example of `ps`. In this example, the `u` option adds informational columns, while `U rodsmith` restricts output to processes owned by `rodsmith`. The `ps` command supports a huge number of options (consult its `man` page for details).

Listing 9.2: Output of `ps u U rodsmith`

```
$ ps u U rodsmith
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
rodsmith 19221  0.0  1.5   4484  1984 ?        S    May07   0:01 dfm
rodsmith  2451  0.0  0.8   1856  1048 pts/3    S    16:13   0:00 -bash
rodsmith  2496  0.2  3.2   6232  4124 ?        S    16:17   0:00 /opt/kd
rodsmith  2498  0.0  0.8   1860  1044 pts/1    S    16:17   0:00 bash
rodsmith  2505  0.1  2.6   4784  3332 pts/1    S    16:17   0:00 nedit
rodsmith  2506  0.0  0.7   2124  1012 ?        S    16:17   0:00 /bin/cs
rodsmith  2544  0.0  1.0   2576  1360 ?        S    16:17   0:00 xeyes
rodsmith  2556  0.0  0.7   2588   916 pts/3    R    16:18   0:00 ps u U
```

Because `ps ax` produces commands with their options, using `grep` to search for a string in the output returns the searched-for command, as well as the `grep` command itself.

Given the large number of `ps` options, different users can have different favorite ways to use the program. I find that typing **`ps ax`** usually produces the information I want, including PID values and command names (including command-line options) for all the processes on the computer. Adding `u` (as in **`ps aux`**) adds usernames, CPU loads, and a few other tidbits. The sheer scope of the information produced, however, can be overwhelming. One way to narrow this scope is to pipe the results through `grep`, which eliminates lines that don't include the search criterion you specify. For instance, if you want to know the PID number for the `gedit` process, you can do so like this:

```
$ ps ax | grep gedit
27946 pts/8    Sl     0:00 gedit
27950 pts/8    S+     0:00 grep --colour=auto gedit
```

This command reveals that `gedit` has a PID value of 27946. This is usually the most important information when you use `ps`, since you'll use the PID value to change a process's priority or terminate it.

Using *top* to Identify Processes

Certification Objective

Although `ps` can return process priority and CPU use information, the program's output is usually sorted by PID number, and provides information at only a single moment in time. If you want to quickly locate CPU- or memory-hogging processes, or if you want to study how resource use varies over time, another tool is more appropriate: `top`. This program is essentially an interactive version of `ps`. Figure 9.4 shows `top` running in a GNOME Terminal window.

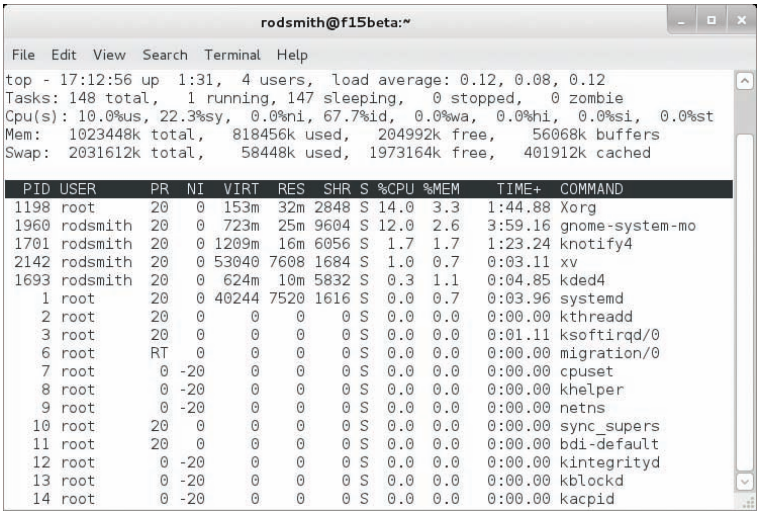


FIGURE 9.4 **The `top` command shows system summary information and information about the most CPU-intensive processes on a computer.**

By default, `top` sorts its entries by CPU use, and it updates its display every few seconds. You'll need to be familiar with the purposes and normal habits of programs running on *your* system in order to determine if a CPU-hungry application is misbehaving; the legitimate needs of different programs vary so much that it's impossible to give a simple rule for judging when a process is consuming too much CPU time.

You can do more with `top` than watch it update its display. When it's running, you can enter any of several single-letter commands, some of which prompt you for additional information, as summarized in Table 9.1. Additional commands are described in `top`'s `man` page.

TABLE 9.1 Common `top` commands

Command	Description
`h` or `?`	These keystrokes display help information.
`k`	You can kill a process with this command. The `top` program will ask for a PID number, and if it's able to kill the process, it will do so.
`q`	This option quits from `top`.
`r`	You can change a process's priority with this command.
`s`	This command changes the display's update rate, which you'll be asked to enter (in seconds).
`P`	This command sets the display to sort by CPU usage, which is the default.
`M`	You can change the display to sort by memory usage with this command.

One of the pieces of information provided by `top` is the *load average,* which is a measure of the demand for CPU time by applications. In Figure 9.4, you can see three load-average estimates on the top line; these correspond to the current load average and two previous measures. Load averages can be interpreted as follows:

- A system on which no programs are demanding CPU time has a load average of 0.
- A system with one program running a CPU-intensive task has a load average of 1.
- Higher load averages on a single-CPU system reflect programs competing for available CPU time.
- On a computer with multiple CPUs or CPU cores, load averages can reach the number of CPUs or cores before competition for CPU time begins. For instance, a load average of 4.0 on a system with a 4-core CPU reflects processes demanding exactly as much CPU time as the computer has available.

Most computers sold today are multi-core models, but single-core models dominated the marketplace prior to about 2006.

The `w` command, described in Chapter 13, can tell you how much CPU time entire terminal sessions are consuming.

The load average can be useful in detecting runaway processes. For instance, if a system normally has a load average of 0.5 but it suddenly gets stuck at a load average of 2.5, a couple of CPU-hogging processes may have *hung*—that is, become unresponsive. Hung processes sometimes needlessly consume a lot of CPU time. You can use `top` to locate these processes and, if necessary, kill them.

Measuring Memory Use

Certification Objective

The kernel grants programs access to sets of memory addresses, which the programs can then use. When a program is done, it should release its memory back to the kernel.

Processes consume a number of system resources, the most important of these being CPU time and memory. As already noted, `top` sorts your processes by CPU time by default, so you can identify processes that are consuming the most CPU time. You can press the M key within `top` to have it sort by memory use, thus identifying the processes that are consuming the most memory. As with CPU time, you can't say that a process is consuming too much memory simply because it's at the top of the list, though; some programs legitimately consume a great deal of memory. Nonetheless, sometimes a program consumes too much memory, either because of inefficient coding or because of a *memory leak*—a type of program bug in which the program requests memory from the kernel and then fails to return it when it's done with the memory. A program with a memory leak consumes increasing amounts of memory, sometimes to the point where it interferes with other programs. As a short-term solution, you can usually terminate the program and launch it again, which resets the program's memory consumption, something like draining a sink that's filled with water from a leaky faucet. The problem will recur, but if the memory leak is small enough, you'll at least be able to get useful work done in the meantime.

If you want to study the computer's overall memory use, the `free` command is useful. This program generates a report on the computer's total memory status:

```
$ free
             total       used       free     shared    buffers     cached
Mem:       7914888    7734456     180432          0     190656    3244720
-/+ buffers/cache:    4299080    3615808
Swap:      6291452    1030736    5260716
```

The `Mem:` line reveals total random access memory (RAM) statistics, including the total memory in the computer (minus whatever is used by the motherboard and kernel), the amount of memory used, and the amount of free memory. This example shows that most of the computer's memory is in use. Such a state is normal, since Linux puts otherwise unused memory to use as buffers and caches, which help speed up disk access. Thus, the `Mem:` line isn't the most useful; instead, you should examine the `-/+ buffers/cache:` line, which shows the total memory used by the computer's programs. In this example, 4,299,080 KiB of 7,914,888 KiB are in use, leaving 3,615,808 KiB free. In other words, a bit over half the computer's memory is in use by programs, so there should be no memory-related performance problems.

The `Swap:` line reveals how much *swap space* Linux is using. Swap space is disk space that's set aside as an adjunct to memory. Linux uses swap space when it runs out of RAM, or when it determines that RAM is better used for buffers or caches than to hold currently inactive programs. In this example, 1,030,736 KiB of swap space is in use, with 6,291,452 KiB total, for 5,260,716 free. Swap space

use is generally quite low, and if it rises very much, you can suffer performance problems. In the long run, increasing the computer's RAM is generally the best solution to such problems. If you're suffering from performance problems because of excessive swap use and you need immediate relief, terminating some memory-hogging programs can help. Memory leaks, described earlier, can lead to such problems, and terminating the leaking program can restore system performance to normal.

The `free` command supports a number of options, most of which modify its display format. The most useful of these is `-m`, which causes the display to use units of mebibytes (MiB) rather than the default of kibibytes (KiB).

Using Log Files

Many programs that run in the background (that is, daemons) write information about their normal operations to *log files*, which are files that record such notes. Consulting log files can therefore be an important part of diagnosing problems with daemons. The first step in doing this is to locate your log files. In some cases, you may need to tell the program to produce more verbose output to help track down the problem, so I provide some pointers on how to do that. Finally, I describe the kernel ring buffer, which isn't technically a log file but can fill a similar role for kernel information.

Locating Log Files

Certification Objective

Log file details vary between distributions, so some of the files in Table 9.2 may not be present on your system, or the files you find may have different names.

Linux stores most log files in the `/var/log` directory tree. Some log files reside in that directory, but some servers create entire subdirectories in which to store their own log files. Table 9.2 summarizes some common log files on many Linux systems. In addition, many server programs not described in this book add their own log files or subdirectories of `/var/log`. If you experience problems with such a server, checking its log files can be a good place to start troubleshooting.

TABLE 9.2 Important log files

Log file	Contents
`boot.log`	This file summarizes the services that are started late in the boot process via SysV startup scripts.

(Continues)

TABLE 9.2 *(Continued)*

Log file	Contents
`cron`	This file summarizes processes that are run at regular intervals via the `cron` daemon. Although this book doesn't cover `cron`, a problem with it can cause glitches that recur at regular intervals, so you should be aware of it.
`cups/`	This directory holds log files related to the Linux printing system.
`gdm/`	This directory holds log files related to the GNOME Display Manager (GDM), which handles GUI logins on many systems.
`messages` or `syslog`	This is a general-purpose log file that contains messages from many daemons that lack their own dedicated log files.
`secure`	You can find security-related messages in this file, including notices of when users employ `su`, `sudo`, and similar tools to acquire `root` privileges.
`Xorg.0.log`	Information on the most recent startup of the X Window System (X) appears in this log file.

Log files are frequently *rotated*, meaning that the oldest log file is deleted, the latest log file is renamed with a date or number, and a new log file is created. For instance, if it's rotated on December 1, 2012, `/var/log/messages` will become `/var/log/messages-20121201`, `/var/log/messages-1.gz`, or something similar, and a new `/var/log/messages` will be created. This practice keeps log files from growing out of control.

Most log files are plain text files, so you can check them using any tool that can examine text files, such as `less` or a text editor. One particularly handy command is `tail`, which displays the last ten lines of a file (or as many lines as you specify with the `-n` option). For instance, typing **`tail /var/log/messages`** shows you the last ten lines of that file.

Note that not all programs log messages. Typically, only daemons do so; ordinary user programs display error messages in other ways—in GUI dialog boxes or in a text-mode terminal. If you think a program should be logging data but you can't find it, consult its documentation. Alternatively, you can use `grep` to try to find the log file to which the program is sending its messages. For instance, typing **`grep sshd /var/log/*`** finds the files in which the string `sshd` (the SSH daemon's name) appears.

Log file rotation occurs late at night, so it won't happen if you shut off your computer. Leave it running overnight periodically to ensure log files are rotated.

Certification Objective

Creating Log Files

Some programs create their own log files; however, most rely on a utility known generically as the *system log daemon* to do this job. This program's process name is generally `syslog` or `syslogd`. Like other daemons, it's started during the boot process by the system startup scripts. Several system log daemon packages are available. Some of them provide a separate tool, `klog` or `klogd`, to handle logging messages from the kernel separately from ordinary programs.

You can modify the behavior of the log daemon, including adjusting the files to which it logs particular types of messages, by adjusting its configuration file. The name of this file depends on the specific daemon in use, but it's typically `/etc/rsyslog.conf` or something similar. The details of log file configuration are beyond the scope of this book, but you should be aware that such details can be altered. This fact accounts for much of the distribution-to-distribution variability in log file features.

Once it's running, a log daemon accepts messages from other processes using a technique known as *system messaging*. It then sorts through the messages and directs them to a suitable log file depending on the message's source and a priority code.

Producing More Verbose Log File Entries

Sometimes log files don't provide enough information to pin down the source of a problem. Fortunately, many programs that produce log file output can be configured to produce *more* such output. Unfortunately, doing so can sometimes make it harder to sift through all the entries for the relevant information.

The procedure for increasing the verbosity of log file output varies from one program to another. Typically, you must set an option in the program's configuration file. You should consult the program's documentation to learn how to do this.

Because the kernel ring buffer has a limited size, its earliest entries can be lost if the computer runs for a long time or if something produces many entries.

Certification Objective

Examining the Kernel Ring Buffer

The kernel ring buffer is something like a log file for the kernel; however, unlike other log files, it's stored in memory rather than in a disk file. Like regular log files, its contents continue to change as the computer runs. To examine the kernel ring buffer, you can type **`dmesg`**. Doing so creates copious output, though, so you'll typically pipe the output through `less`:

```
$ dmesg | less
```

Alternatively, if you know that the information you want will be associated with a particular string, you can use `grep` to search for it. For instance, to find kernel ring buffer messages about the first hard disk, `/dev/sda`, you might type the following:

```
$ dmesg | grep sda
```

Kernel ring buffer messages can be particularly arcane; however, they can also be invaluable in diagnosing hardware and driver problems, since it's the kernel's job to interface with hardware. You might try searching the kernel ring buffer if a hardware device is behaving strangely. Even if you don't understand a message you find, you could try feeding that message into a Web search engine or passing it on to a more knowledgeable colleague for advice.

Some distributions place a copy of the kernel ring buffer when the system first boots in `/var/log/dmesg` or a similar file. You can consult this file if the computer has been running for long enough for its earliest entries to be lost. If you want to create such a file on a distribution that doesn't do so by default, you can edit the `/etc/rc.d/rc.local` file and add the following line to its end:

```
dmesg > /var/log/dmesg
```

The Essentials and Beyond

An operating Linux computer can be thought of as consisting of running programs—that is, processes. Managing processes begins with managing the programs that are installed on the computer, which is a task you can perform with package management tools such as `rpm` or `dpkg`. You can learn what processes are running by using tools such as `ps` and `top`. Log files can help you learn about the actions of daemons, which may not be able to communicate error messages through the type of text-mode or GUI output that other programs can generate.

Suggested Exercises

- Is your distribution's software up to date? Locate the option in your desktop environment's menus to run a package manager and check that the system is up to date. A computer running out-of-date software can be vulnerable to bugs and security threats, so keeping your software updated is important!
- Type **`ps ax | less`** and browse through the process list. You might not recognize many of the processes, but some should be familiar. Try using `man` or a Web search to learn more about some of the processes you don't recognize.

(Continues)

7. Linux File System

Several earlier chapters in this book have alluded to the way files are organized on a Linux computer, and have even described some specifics—for instance, that `/home` is a directory that holds users' home directories. This chapter goes into more detail on this topic. First up is a description of some of the key Linux directories, including their names and purposes. I also describe some special permission bits you can set on files and directories to affect the way Linux treats them. Such attributes can enable you to hide files from view or alter the usual permissions rules in ways that can be helpful for shared user directories.

- **Understanding where things go**
- **Using special permission bits and file features**

Understanding Where Things Go

The term *filesystem* can refer to low-level data structures or to the organization of files and directories on the computer. In this chapter, the latter meaning is most common.

As described in Chapter 6, "Getting to Know the Command Line," Linux uses a unified directory tree—that is, every partition, removable disk, network file share, and other disk or disk-like storage device is accessible as a directory in a single directory tree (or *filesystem*). This filesystem is structured—certain directories have certain specific purposes, whether the directories exist as regular subdirectories on one partition or are separate devices that are mounted off the root (`/`) device. Understanding the purpose of the main directories will help you locate files and avoid making disastrous mistakes. Before delving into those specifics, you should understand the distinction between user files and system files.

User Files vs. System Files

To understand the distinction between user files and system files, you should recall that Linux is a multi-user OS. In principle, a single computer can host

thousands of users. Consider such a computer—perhaps a mainframe at a university or a large business or a cloud computing server. The vast majority of this computer's thousands of users will be unfamiliar with the details of Linux system administration—they want only to use their word processors, e-mail clients, and other user applications. Such users don't need to deal with system configuration files. Indeed, giving them access to such files—especially write access—could be disastrous. On a computer with 1,000 users, each of whom can change the system configuration, *somebody* will make a change that will bring down the computer, whether through ignorance or malice.

Of course, the issues involved in protecting the computer from its users are just a special case of more general user account issues—on that 1,000-user computer, you probably don't want users to be able to read and write each others' files except in limited ways. Thus, as described in Chapter 13, "Understanding Users and Groups" and Chapter 15, "Setting Ownership and Permissions," you can set permissions on files to prevent unauthorized access.

Certification Objective

System files are files that control how the computer operates—they include:

- System startup scripts that launch servers and other important daemons
- Program files, both binary files and scripts
- Program support files, such as fonts and icons
- Configuration files that define how the system works (network configuration settings, disk layout information, and so on)
- Configuration files for most servers and other daemons
- Data storage for system programs, such as the database that describes what programs are installed
- System log files, which record normal system activity

Chapter 9, "Using Programs and Processes," describes package management.

Obviously, non-technical users should not be able to alter system files, except perhaps indirectly. (Log files record activities such as login attempts, for instance.) Users must be able to read some types of system files, such as the fonts and icons they use, but some system files should be protected even from read access. Users should not be able to read the `/etc/shadow` file because it holds encrypted passwords, for instance.

To achieve the goal of restricting ordinary users' access to system files, they are normally owned by `root` or by another system account that has a more limited purpose. For instance, many server programs rely on their own specific system accounts. System files can then be protected from unwanted access by setting

permissions in some appropriate way, depending on their specific needs. Ordinary users are then unable to write to most system files, protecting them from harm. Because `root` can read and write any file, you must acquire `root` privileges to perform most system maintenance tasks.

Immediately after installing Linux, most of the files it contains are system files, and most of the directories and subdirectories on a fresh Linux installation are system directories. As described shortly, a few directories, such as `/home` and `/tmp`, are set aside for user files, although even these are structured or configured in such a way as to prevent problems with multi-user access.

Users' home directories traditionally reside in `/home`, whereas `/tmp` is accessible to all users and holds temporary files.

The distinction between system files and user files exists even on single-user Linux computers such as your personal laptop computer. This may seem strange or frustrating; after all, if you're the only user, and if you have `root` access, why bother with the distinction between system files and user files? The answer is that it provides a layer of protection against accidental or malicious damage. If a typo, a bug, or malware would, say, delete all the files on the computer, the damage is contained if this action is performed as a normal user rather than as `root`—a normal user *can't* delete all the files on the computer. Thus, the distinction between these two account types (and, by extension, these two classes of files) is useful even on a single-user computer.

The Filesystem Hierarchy Standard

The FHS evolved from an earlier Linux-only standard, the *Filesystem Standard* (FSSTND).

Although every Linux distribution has its own unique way of doing certain things, their developers all recognize the need for some standardization in the layout of their directories. For instance, programs should be able to consistently locate key system configuration files in the same places on all distributions. If this weren't the case, programs that rely on such features would become more complex and might not work on all distributions. In order to address this need, the *Filesystem Hierarchy Standard* (FHS) was created. Even aside from Linux, some Unix-like OSs also follow the FHS to one degree or another.

One important distinction made by the FHS is that between *shareable files* and *unshareable files*. Shareable files may be reasonably shared between computers, such as user data files and program binary files. (Of course, you don't *need* to share such files, but you *may* do so.) If files are shared, they're normally shared through a Network File System (NFS) server. Unshareable files contain system-specific information, such as configuration files. For instance, you're not likely to want to share a server's configuration file between computers.

A second important distinction made by the FHS is that between *static files* and *variable files*. The former don't normally change except through direct intervention by the system administrator. Most program executables are good

examples of static files. Variable files may be changed by users, automated scripts, servers, or the like. For instance, users' home directories and mail queues are composed of variable files.

The FHS tries to isolate each directory into one cell of this 2 × 2 (shareable/unshareable × static/variable) matrix. Figure 16.1 illustrates these relationships. Some directories contain subdirectories in multiple cells, but in these cases, the FHS tries to specify the status of particular subdirectories. For instance, `/var` is variable, and it contains some shareable and some unshareable subdirectories, as shown in Figure 16.1.

	Shareable	Unshareable
Static	`/usr` `/opt`	`/etc` `/boot`
Variable	`/home` `/var/mail`	`/var/run` `/var/lock`

FIGURE 16.1 The FHS attempts to fit each important directory in one cell of a 2 × 2 matrix.

The FHS comes in numbered versions. Version 2.3, the latest version as I write, was released in January 2004. The URL for FHS's official Web page is `http://www.pathname.com/fhs/`.

The FHS has been stable for years. Fedora is planning changes for Fedora 17 that will violate or require changes to the FHS.

Important Directories and Their Contents

The FHS defines the names and purposes of many directories and subdirectories on a Linux system. Table 16.1 summarizes the most important of these. Most of these directories are system directories, the main exceptions being `/home`, `/tmp`, `/mnt`, and `/media`.

Certification Objective

TABLE 16.1 Important Linux directories, according to the FHS

Directory	Purpose
`/`	The root directory; all files appear in this directory or subdirectories of it.
`/etc`	Holds system configuration files.
`/boot`	Holds important boot files, such as the Linux kernel, the initial RAM disk, and often boot loader configuration files.

(Continues)

TABLE 16.1 *(Continued)*

Directory	Purpose
/bin	Holds program files that are critical for normal operation and that ordinary users may run.
/sbin	Holds program files that are critical for normal operation and that ordinary users seldom run.
/lib	Holds libraries—code used by many other programs—that are critical for basic system operation.
/usr	Holds programs and data used in normal system operation but that aren't critical for a bare-bones boot of the system. This directory is split into subdirectories that mirror parts of the root organization—/usr/bin, /usr/sbin, /usr/lib, and so on.
/home	Holds users' home directories. Separating this directory into its own low-level filesystem effectively isolates most user data from the OS, which can be useful if you want to re-install the OS without losing user data.
/root	The root user's home directory. Note that this is different from /, which is pronounced *root*.
/var	Holds miscellaneous transient files, such as log files and print spool files. One subdirectory of /var, /var/tmp, deserves special mention. Like /tmp (described next), /var/tmp holds temporary files. These files should *not* be deleted when the computer reboots.
/tmp	Holds temporary files, often including temporary files created by user programs. Such files may theoretically be deleted when the computer reboots, although in practice many distributions don't do this.
/mnt	The traditional mount point for removable media; sometimes split into subdirectories for each mounted filesystem.
/media	The new mount point for removable media; typically split into subdirectories for each mounted filesystem.
/dev	Holds *device files*, which provide low-level access to hardware.

As an ordinary user, you will create most of your files in your home directory, which is normally a subdirectory of /home. You might also access removable media mounted at /media, and perhaps network resources that might be mounted

Ordinary users can't write to most system directories, such as /usr. Thus, you can't damage your installation by checking out these directories—*if* you're running as an ordinary user!

elsewhere. You can use /tmp and certain subdirectories of /var, too, although most users don't need to be explicitly aware of these locations—programs are normally hard-coded to use them for temporary files or for specific types of files, such as incoming e-mail files. As a system administrator, you might manipulate files located in any of these directories; however, for a system administrator, /etc is particularly important, since that's where most system configuration files reside. As you explore your computer with GUI or text-mode utilities, you should keep this directory structure in mind.

Of these directories, several individual directories or collections of them bear special attention:

The configuration directory The /etc directory holds most system configuration files. Previous chapters have referred to several such files, such as /etc/fstab (which defines where partitions are mounted) and /etc/passwd (which is the primary account definition file). Many more exist. Indeed, you'll find subdirectories in /etc to house multiple configuration files for complex subsystems and servers, such as /etc/X11 (for the X Window System) and /etc/samba (for the Samba file server).

See the sidebar, "Fedora 17 and the FHS," for information on changes Fedora is planning to where it stores program files.

Executable directories Program files reside mainly in /sbin, /bin, /usr/sbin, and /usr/bin. (Additional directories can house program files on some systems. Most notably, /usr/local/sbin and /usr/local/bin hold locally compiled programs.)

Certification Objective

Library directories *Libraries* are collections of programming functions that can be useful to many programs. They're stored in separate files to save disk space and RAM when programs run, and enable easy updates to library files without re-installing all the programs that rely on them. In Linux, most libraries reside in /lib and /usr/lib, although some can reside elsewhere (such as /usr/local/lib) on some systems.

If you've administered Windows computers, you should be aware of an important difference between Windows and Linux: In Windows, it's common for a program binary, its configuration files, and all its support files to reside in a single directory tree that belongs to the program, such as C:\Program Files\SomeProgram. In Linux, by contrast, most of a program's key files are likely to reside in standard locations that are shared with other programs, and to be scattered about. For instance, the program's executable might be in /usr/bin, related libraries in /usr/lib, configuration files in /etc or in users' home directories, and so on. This works well in Linux because Linux's packaging systems, described in Chapter 9, keep track of where a package's many files go, enabling you to easily delete or upgrade a package. The Linux system has the advantage of simplifying the *path*, which is the list of directories in which program files

reside. (Paths also exist for libraries and `man` pages.) If you're used to looking for files in program-specific locations, though, adjusting to the Linux system can be awkward. The key is to use your package system to identify where a package's files reside; for instance, typing **`rpm -ql someprogram`** shows where every file in the *someprogram* package resides on an RPM-based system.

The package system does not manage user configuration files, so they can linger after you delete a program. This causes no harm aside from the disk space consumed.

Fedora 17 and the FHS

Fedora's developers have announced that they intend to fold the contents of several directories together—for instance, to combine `/bin`, `/sbin`, `/usr/bin`, and `/usr/sbin` into a single directory (`/usr/bin`). Such a change is a violation of the FHS, but whether it will cause any problems remains to be seen. (Fedora 17 is not yet available as of this writing.)

Using Special Permission Bits and File Features

When you investigate the Linux directory tree, you will encounter certain file types that require special attention. Sometimes you may just want to be aware of how these files are handled, since they deviate from what you might expect based on the information presented in earlier chapters. In other cases, you may need to adjust how you use `ls` or other commands to deal with these files and directories. These special cases include the "sticky bit," hiding files from view, obtaining long listings of directories, and using special execute permissions.

Using Sticky Bits

Certification Objective

Consider the following commands, typed on a system with a few files and subdirectories laid out in a particular way:

```
$ whoami
kirk
$ ls -l
total 0
drwxrwxrwx 2 root root 80 Dec 14 17:58 subdir
$ ls -l subdir/
total 2350
-rw-r----- 1 root root 2404268 Dec 14 17:59 f1701.tif
```

These commands establish the current configuration: The effective user ID is `kirk` and the current directory has one subdirectory, called `subdir`, which `root` owns but to which `kirk`, like all the system's users, has full read/write access. This subdirectory has one file, `f1701.tif`, which is owned by `root` and to which `kirk` has no access. You can verify that `kirk` can't write to the file by attempting to do so with the `touch` command:

```
$ touch subdir/f1701.tif
touch: cannot touch `subdir/f1701.tif': Permission denied
```

This error message verifies that `kirk` could not write to `subdir/f1701.tif`. The file, you might think, is safe from tampering. Not so fast! Try this:

```
$ rm subdir/f1701.tif
$ ls -l subdir/
total 0
```

The `rm` command returned no error message, and a subsequent check of `subdir` verifies that it's now empty—in other words, `kirk` could delete the file even without write permission to it! This may seem like a bug—after all, if you can't write to a file, you might think you shouldn't be able to delete it. Recall, however, that directories are just a special type of file, one that holds other files' names and pointers to their lower-level data structures. Thus, modifying a file requires write access to the file, but creating or deleting a file requires write access to the *directory in which it resides*. In this example, `kirk` has write access to the `subdir` directory, but not to the `f1701.tif` file within that directory. Thus, `kirk` can delete the file but not modify it. This result is not a bug; it's just a counterintuitive feature.

Chapter 15 describes the basics of `chown`.

Although Linux filesystems were designed to work this way, such behavior is not always desirable. The way to create a more intuitive result is to use a *sticky bit*, which is a special filesystem flag that alters this behavior. With the sticky bit set on a directory, Linux will only permit you to delete a file if you own either it or the containing directory; write permission to the containing directory is not enough. You can set the sticky bit with `chown`, in either of two ways:

Other odd numbers will set the sticky bit, too, but will also set additional special permission bits, which are described shortly, in "Using Special Execute Permissions."

Using an octal code By prefixing the 3-digit octal code described in Chapter 15 with another digit, you can set any of three special permission bits, one of which is the sticky bit. The code for the sticky bit is 1, so you would use an octal code that begins with 1, such as 1755, to set the sticky bit. Specifying a value of 0, as in 0755, removes the sticky bit.

Using a symbolic code Pass the symbolic code `t` for the world permissions, as in **`chmod o+t subdir`**, to set the sticky bit on `subdir`. You can remove the sticky bit in a similar way by using a minus sign, as in **`chmod o-t subdir`**.

Restoring the file and setting the sticky bit enables you to see the effect:

```
$ ls -l
total 0
drwxrwxrwt 2 root root 80 Dec 14 18:25 subdir
$ ls -l subdir/
total 304
-rw-r--r-- 1 root root 2404268 Dec 14 18:25 f1701.tif
$ rm subdir/f1701.tif
rm: cannot remove `subdir/f1701.tif': Operation not permitted
```

In this example, although `kirk` still has full read/write access to `subdir`, `kirk` cannot delete another user's files in that directory.

You can identify a directory with the sticky bit set by a small change in the symbolic mode shown by `ls -l`: The world execute bit is shown as a `t` rather than an `x`. In this example, the result is that `subdir`'s permission appears as `drwxrwxrwt` rather than `drwxrwxrwx`.

The sticky bit is particularly important for directories that are shared by many users. It's a standard feature on `/tmp` and `/var/tmp`, for instance, since many users store temporary files in these directories, and you wouldn't want one user to be able to delete another's temporary files. If you want users who collaborate on a project to be able to write files into each others' home directories, you might want to consider setting the sticky bit on those home directories, or on the subdirectories in which users are sharing files.

If you delete `/tmp` or `/var/tmp` and need to re-create it, be sure to set the sticky bit on your new replacement directory!

Using Special Execute Permissions

As described in Chapter 15, the execute permission bit enables you to identify program files as such. Linux then enables you to run these programs. Such files run using your own credentials, which is generally a good thing—associating running processes with specific users is a key part of Linux's security model. Occasionally, though, programs need to run with elevated privileges. For instance, the `passwd` program, which sets users' passwords, must run as `root` to write, and in some cases to read, the configuration files it handles. Thus, if users are to change their own passwords, `passwd` must have `root` privileges even when ordinary users run it.

Certification Objective

To accomplish this task, two special permission bits exist, similar to the sticky bit described earlier:

Set user ID (SUID) The *set user ID* (SUID) option tells Linux to run the program with the permissions of whoever owns the file rather than with the permissions of the user who runs the program. For instance, if a file is owned by `root` and has its SUID bit set, the program runs with `root` privileges and can therefore read any file on the computer. Some servers and other system programs run this

way, which is often called SUID `root`. SUID programs are indicated by an `s` in the owner's execute bit position in the permission string, as in `rwsr-xr-x`.

Set group ID (SGID) The *set group ID* (SGID) option is similar to the SUID option, but it sets the group of the running program to the group of the file. It's indicated by an `s` in the group execute bit position in the permission string, as in `rwxr-sr-x`.

You can set these bits using `chmod`:

Using an octal code In the leading digit of a 4-digit octal code, set the leading value to 4 to set the SUID bit, to 2 to set the SGID bit, or to 6 to set both bits. For instance, 4755 sets the SUID bit, but not the SGID bit, on an executable file.

Using a symbolic code Use the `s` symbolic code, in conjunction with `u` to specify the SGID bit, `g` to specify the SGID bit, or both to set both bits. For instance, typing **`chmod u+s myprog`** sets the SUID bit on `myprog`, whereas **`chmod ug-s myprog`** removes both the SUID bit and the SGID bit.

Ordinarily, you don't need to set or remove these bits; when necessary, the package management program sets these bits correctly when you install or upgrade a program. You might need to alter these bits if they've been mistakenly set or removed on files. In some cases you might want or need to adjust these values on program files that you compile from source code or if you need to modify the way a program works. Be very cautious when doing so, though. If you set the SUID or SGID bit on a garden-variety program, it will run with increased privileges. If the program contains bugs, those bugs will then be able to do more damage. If you accidentally remove these permissions, the results can be just as bad—programs like `passwd`, `sudo`, and `su` all rely on their SUID bits being set, so removing this feature can cause them to stop working.

Hiding Files from View

Certification Objective

If you're used to Windows, you may be familiar with the concept of a *hidden bit*, which hides files from view in file managers, by the Windows `DIR` command, and in most programs. If you're looking for something analogous in Linux, you won't find it, at least not in the form of a dedicated filesystem feature. Instead, Linux uses a file-naming convention to hide files from view: Most tools, such as `ls`, hide files and directories from view if their names begin with a dot (`.`). Thus, `ls` shows the file `afile.txt`, but not `.afile.txt`. Most file managers and dialog boxes that deal with files also hide such *dot files*, as they're commonly called; however, this practice is not universal.

Many user programs take advantage of this feature to keep their configuration files from cluttering your display. For instance, `~/.bashrc` is a Bash user configuration file, Evolution's configuration files go in the `~/.evolution` directory, and `~/.fonts.conf` holds user-specific font configuration information.

You can view dot files in various ways depending on the program in question. Some GUI tools have a check box you can set in their configuration options to force the program to display such files. At the command line, you can use the `-a` option to `ls`:

```
$ ls -l
total 0
drwxrwxrwt 2 root root 80 Dec 14 18:25 subdir
$ ls -la
total 305
drwxr-xr-x 3 kirk users    104 Dec 14 18:44 .
drwxr-xr-x 3 kirk users    528 Dec 14 18:21 ..
-rw-r--r-- 1 kirk users 309580 Dec 14 18:44 .f1701.tif
drwxrwxrwt 2 root root      80 Dec 14 18:25 subdir
```

This example shows the hidden file, `.f1701.tif`, in the current directory. It also shows two hidden directory files. The first, `.`, refers to the current directory. The second, `..`, refers to the parent directory.

Certification Objective

Note that renaming a file so that it begins with a dot will hide it, but this action will also make the file inaccessible to any program that uses the original filename. That is, if you rename `f1701.tif` to `.f1701.tif`, and if another program or file refers to the file as `f1701.tif`, that reference will no longer work. You *must* include the leading dot in any reference to the hidden file.

Recall from Chapter 6 that `..` is a relative directory reference. This hidden entry is why it works.

Viewing Directories

Certification Objective

Chapter 6 introduced the `ls` command, including many of its options. One of these deserves elaboration at this point: `-d`. If you're working in a directory that holds many subdirectories, and if you use a wildcard with `ls` that matches one or more subdirectories, you may get an unexpected result: The output will show the files in the matched subdirectories, rather than the information on the subdirectories themselves. For instance, starting in a directory with two subdirectories, `subdir1` and `subdir2`:

```
$ ls -l subdir*
subdir1:
total 304
-rw-r--r-- 1 kirk users 309580 Dec 14 18:54 f1701.tif

subdir2:
```

```
total 84
-rw-r--r-- 1 kirk users 86016 Dec 14 18:54 106792c17.doc
```

If instead you want information on the subdirectories, rather than the contents of those subdirectories, you can include the `-d` option:

```
$ ls -ld subdir*
drwxr-xr-x 2 kirk users 80 Dec 14 18:54 subdir1
drwxr-xr-x 2 kirk users 80 Dec 14 18:54 subdir2
```

The Essentials and Beyond

Linux's directory tree places files in particular locations according to their function—program binaries, libraries, configuration files, user files, and so on. Knowing where these files reside will help you locate them—say, to find configuration files. When you go looking for files, you should be aware of some tools and filesystem features that can be useful—or confusing, if you're not aware of them. Sticky bits, hidden ("dot") files, the `-d` option to `ls`, and SUID and SGID features can all be important when you view or modify files, and especially system files.

Suggested Exercises

- Examine the names of configuration files in `/etc` and try to find matching program files in `/bin`, `/sbin`, `/usr/bin`, and `/usr/sbin`. Examine the configuration files with `less` or a text editor and read their `man` pages. (Don't do this for *all* the configuration files in `/etc`, though—that would take forever! A couple will suffice.)
- Experiment with the sticky bit on a "scratch" directory. Using your regular account and either another regular account or the `root` account, set the ownership and sticky bit on the scratch directory in different ways and test whether you can delete and modify files in the directory.

Review Questions

1. What types of files are you likely to find in `/usr/lib`, according to the FHS?

   A. Liberty files
   B. Liberated files
   C. Libra files
   D. Library files
   E. Liberal files

(Continues)

8. Managing Hardware

Although Linux is software, it relies on hardware to operate. The capabilities and limitations of your hardware will influence the capabilities and limitations of Linux running on that hardware. Therefore, you should know these features for any computer you use extensively or that you plan to buy. This chapter will help you learn about and perform basic management tasks with your hardware. I begin with low-level issues, such as the nature of your central processing unit (CPU) and motherboard. Often overlooked, the power supply can cause problems if it's undersized or misbehaves, so I describe it. Hard disks require special care in their setup, which I briefly describe. A common sticking point in Linux is display issues, since a malfunctioning display prevents any interaction with the computer. Today, most external devices attach via the Universal Serial Bus (USB), so I describe its features. Finally, this chapter covers the common issue of *drivers*, which are software components that control hardware devices.

- **Learning about your CPU**
- **Identifying motherboard capabilities**
- **Sizing your power supply**
- **Understanding disk issues**
- **Managing displays**
- **Handling USB devices**
- **Managing drivers**

Learning About Your CPU

Certification Objective

Your CPU (sometimes called the *processor*) is the "brain" of your computer—it does most of the computer's actual computing. I've mentioned CPUs in earlier

chapters in reference to distribution availability—to run on a given CPU, most software must be recompiled for that CPU. Thus, it's important that you know enough about the different CPU families to know their strengths and weaknesses and to identify what type of CPU your computer uses.

Many devices have more specialized computing circuitry. Most notably, video cards include graphics processing units (GPUs) to do specialized graphics computations.

Understanding CPU Families

CPU manufacturers tend to create product lines by making regular improvements to their products. Improvements range from minor (such as running the CPU at a faster *clock rate*, which is like running an engine at a faster speed) to moderate redesigns to improve speed (which is like shifting gears to get better speed) to radical redesigns (which are like using a bigger or more efficient engine, but one based on the original design). All of these types of changes remain within a single CPU family, so they can run the same code as their predecessors. Still more radical differences exist across CPU families; two CPUs from different families can typically *not* run each other's binary programs, although there are exceptions to this rule, including one very important one that I describe shortly.

On desktop computers, two CPU families are (or were recently) common:

x86 This CPU type originated with Intel's 8086 CPU, but the first model capable of running Linux was the 80386 (also known as the *386*). Development continued with the 80486 (also known as the *486*), the Pentium, and related Intel CPUs, such as the Celeron. AMD, Cyrix, VIA, and others have all released CPUs that are compatible with Intel's designs. Recent AMD names for its *x*86 CPUs include Athlon and Duron. The earliest *x*86 CPUs were 16-bit models, but the model line has been 32-bit since the 80386.

The upcoming section, "Identifying Your CPU," describes how to determine what CPU you've got.

x86-64 Other names for this architecture include *AMD64*, *x64*, and *EM64T*. AMD created the *x*86-64 architecture as a 64-bit extension to the *x*86 architecture. Unusually, *x*86-64 CPUs can run the earlier 32-bit *x*86 code, but when run in 64-bit mode, such CPUs have access to additional features that improve speed. Intel has created its own *x*86-64 CPUs. Both Intel and AMD have used the same product line names for their *x*86-64 CPUs as for their *x*86 CPUs. This fact can make it hard to tell whether you've got a 32-bit *x*86 CPU or a 64-bit *x*86-64 CPU, at least based on the CPU's marketing name. Most desktop and small server computers sold since about 2007 have used *x*86-64 CPUs.

CPU Bit Depth

CPUs process data in binary (base 2), meaning that numbers are represented using only two digits—0 and 1. CPUs have limits to the sizes of the numbers they can process, though, and those limits are described in terms of the number of binary digits, or *bits*, that the CPU can handle. A 32-bit CPU, for instance, can process numbers that contain up to 32 binary digits. Expressed as positive integers, this means that numbers can range in size from 0 to $2^{32} - 1$ (4,294,967,295 in the base 10 that people generally use). When dealing with larger numbers, the CPU must combine two or more numbers, which requires extra code.

CPUs with larger bit depths have an advantage when dealing with lots of memory, since memory addresses must fit into the CPU's basic unit size. In particular, a 32-bit CPU has a 4 GiB limit on memory—although some architectures, including *x*86, provide tricks to work around this limit. Greater bit depth does not improve speed per se except when dealing with very large numbers; however, the 64-bit *x*86-64 CPU architecture is faster than its 32-bit *x*86 predecessor for unrelated reasons.

In addition to *x*86 and *x*86-64 CPUs, several other model lines are available. Most of these CPUs are used in embedded applications or in very high-end servers. A couple you might encounter on desktops or small servers are:

PowerPC This CPU, created as a cooperative effort of Apple, IBM, and Motorola, was Apple's CPU of choice between 1994 and 2006, and so is found in Macintoshes of that vintage. Today, it's used in some game consoles, embedded devices, and servers. Both 32- and 64-bit versions of this architecture are available. *PPC* is a common abbreviation for this architecture.

Itanium Intel created the Itanium, or IA-64, architecture as a 64-bit replacement for the *x*86 line; however, it gained little market penetration except in the server field.

Linux can run on PowerPC and Itanium, as well as on other CPU families, such as MIPS and SPARC; however, your choice of distribution is likely to be limited. Because software has to be recompiled and tested on new architectures, it takes effort to prepare a distribution for each architecture, and many distribution maintainers are unwilling or unable to expend this effort for more than *x*86 and *x*86-64.

Debian is available on many architectures, so if you need to support Linux on a wide range of CPUs, using Debian can make sense.

Some high-end motherboards also support multiple CPUs, so you can use, say, two 4-core CPUs to get the performance of an 8-CPU system.

Many CPUs today are *multi-core* models. These CPUs package the circuitry for two or more CPUs into one unit. When plugged into a motherboard, such a CPU looks like multiple CPUs to the OS. The advantage is that Linux can run as many CPU-intensive programs as you have cores and they won't slow each other down significantly by competing for CPU resources.

Identifying Your CPU

If you've got a working Linux system, you can learn a great deal about your CPU by using three text-mode commands:

`uname -a` Typing **`uname -a`** displays basic information on the kernel and the CPU. For instance, one of my systems returns `x86_64 AMD Phenom (tm) II X3 700e Processor`, among other things, indicating the manufacturer and model number of the CPU.

`lscpu` This command returns additional information on about 20 lines of output. Much of this information is highly technical, such as the sizes of *caches* the CPU supports. Some of it's less technical, such as the architecture and the number of CPUs or cores it supports.

`cat /proc/cpuinfo` This command returns still more information compared to `lscpu`. Chances are you won't need this information yourself, but a developer or technician might want some of this information to help debug a problem.

One thing to keep in mind is that modern *x*86-64 CPUs can run software compiled for the older *x*86 architecture. Thus, you might be running a 32-bit Linux distribution on a 64-bit CPU. The output of these commands can be confusing in such cases. For instance, here's part of what `lscpu` shows on one such system:

```
Architecture:          i686
CPU op-mode(s):        64-bit
```

The `Architecture` line suggests an *x*86 CPU (i686 being a specific variant of that architecture), but the `CPU op-mode(s)` line indicates that the CPU supports 64-bit operation. If you have trouble interpreting this output, you might be able to find something by looking up the CPU's model on the manufacturer's Web site; however, manufacturers tend to bury such information in hard-to-read specification sheets, so be prepared to read carefully.

Identifying Motherboard Capabilities

Certification Objective

If the CPU is the computer's brain, the motherboard is the rest of the computer's central nervous system. The motherboard is a large circuit board inside the

computer. It's dominated by a *chipset*, which is one or more chips that provide key functionality for the computer—they handle the hard disk interfaces, the USB interfaces, the network devices, and so on. Some chipsets include video circuitry for video cards, although this functionality is sometimes separate, and sometimes it's built into the CPU.

Motherboards are sometimes referred to as *mainboards*.

In addition to the chipset, motherboards include plug-in interfaces for key components:

- One or more slots for the computer's CPU(s)
- Slots for random access memory (RAM)
- Slots for plug-in Peripheral Component Interconnect (PCI) or other cards
- Connectors for Serial Advanced Technology Attachment (SATA) disks, and sometimes for Parallel ATA (PATA) disks
- Back-panel connectors that provide external interfaces for USB devices, keyboards, monitors, and so on
- Connectors for additional external devices, such as front-panel USB plugs; such devices are attached via short internal cables

Some motherboards—typically used in larger desktop and server computers—have many connectors for many purposes. Such motherboards are highly expandable but they're physically large enough that they require bulky cases. Other motherboards are much smaller and can be used in compact computers, but such computers aren't as expandable. Laptop computers also have motherboards, which are necessarily of the small variety, with little opportunity for internal expansion.

Most of the connectors on a motherboard are managed by its primary chipset. Some high-end boards provide features beyond those of its primary chipset. Such features require a secondary chipset, such as an extra Ethernet chipset for a second network port or an extra SATA chipset for more or faster hard disk interfaces.

Whether the feature is provided by the main or by a secondary chipset, you can learn about most of the motherboard's features with the `lspci` command, which shows information on PCI devices. The output looks something like this:

```
$ lspci
00:00.0 Host bridge: Advanced Micro Devices [AMD] RS780 Host Bridge
00:11.0 SATA controller: ATI Technologies Inc SB700/SB800 SATA↵
 Controller [IDE mode]
00:12.0 USB Controller: ATI Technologies Inc SB700/SB800 USB OHCI0↵
 Controller
00:14.1 IDE interface: ATI Technologies Inc SB700/SB800 IDE Controller
00:14.2 Audio device: ATI Technologies Inc SBx00 Azalia (Intel HDA)
```

I've edited this for brevity.

```
01:05.0 VGA compatible controller: ATI Technologies Inc Radeon HD 3200↵
 Graphics
01:05.1 Audio device: ATI Technologies Inc RS780 Azalia controller
02:00.0 Ethernet controller: Realtek Semiconductor Co., Ltd.↵
 RTL8111/8168B PCI Express Gigabit Ethernet controller (rev 02)
03:06.0 Ethernet controller: Intel Corporation 82559 InBusiness 10/100↵
 (rev 08)
```

You may not understand everything in this output, but you should be able to glean some information from it. For instance, the computer has a number of ATI devices—an SATA controller, a USB controller, a graphics adapter, and so on. Two Ethernet devices are present—one made by Realtek and the other by Intel. Although it's not obvious from this output, the Realtek network adapter is built into the motherboard, whereas the Intel device resides on a plug-in card.

Sizing Your Power Supply

Certification Objective

A computer's power supply takes the alternating current (AC) power from a wall outlet and converts it to the direct current (DC) that your motherboard and everything you plug into it uses. Laptop computers and some small desktop units use power adapter "bricks" that you can put on the floor. Larger desktop computers have internal power supply units. These internal units are larger, both physically and in terms of the amount of power they can deliver.

Every power supply has limits on the amount of DC power it can deliver. This is important because every device you plug into the computer consumes a certain amount of power. If your computer manufacturer cut corners, the power supply may be barely adequate for the computer as delivered. If you add a hard disk or a power-hungry plug-in card, you could exceed the amount of power that the power supply can deliver. The result can be unreliable operation—the computer can crash or behave erratically, perhaps corrupting data or files. Such problems can be hard to distinguish from other problems, such as bad RAM or a failing hard disk.

If you need to replace your power supply, pay attention to its output in watts. You should be able to find the output of your current power supply on a sticker—but you'll need to open your computer first, at least for most desktop systems. Be sure to get a power supply that's rated for at least as many watts as the one you're replacing. Also be sure it will fit—sizes are standardized, but there are a few variants available. In the case of a laptop or a small desktop computer with an external power supply, you must ensure that a replacement provides the right type of connector to the computer. Buying a replacement from the computer's manufacturer is usually the best course of action in this case.

Understanding Disk Issues

Disks are a critical part of most Linux installations. Therefore, I describe three basic disk issues in this chapter: disk hardware interfaces, disk partitioning, and filesystems. I also describe some of the issues surrounding removable disks, including optical (CD-ROM, DVD-ROM, and Blu-ray) discs.

You can install Linux in a *diskless* configuration, in which a Linux computer boots using files stored on a network server.

Disk Interfaces

Certification Objective

Today, two disk interfaces are common, both of which have already been mentioned:

PATA This interface was very common in the past, but it's fading in popularity. It features wide 40- or 80-pin cables that transfer several bits of data simultaneously—hence the word *parallel* in the name Parallel ATA (PATA). A PATA cable can have up to three connectors—one for the motherboard or disk controller card and two more for up to two hard disks. Alternative names for PATA (or specific variants of it) include Integrated Device Electronics (IDE) or Enhanced IDE (EIDE). The ATA Packet Interface (ATAPI) standard defines a software interface that helps ATA manage devices other than hard disks. Although in some cases the differences between the technologies described by these variant terms are important, today they're often used synonymously.

SATA In 2003, a serial version of the ATA protocol was created, hence Serial ATA (SATA). SATA is more or less software compatible with PATA, but it uses thinner cables that can handle just one hard disk per cable. In 2012, SATA is the dominant disk technology on new computers. An external variant, eSATA, provides high-speed connections to external hard disks.

In addition to these technologies, others exist. The Small Computer System Interface (SCSI) is a parallel interface that was once common on servers and high-end interfaces but is less common today. The Serial Attached SCSI (SAS) is a serial variant that's quite similar to SATA. Both of these technologies are important because ATAPI is modeled after SCSI. The Universal Serial Bus (USB) interface is often used for connecting external disks.

Most modern Linux distributions treat all disks as if they were SCSI disks from a software perspective.

Partitioning a Disk

Certification Objective

You can think of a hard disk as a set of *sectors*, each of which holds a small amount of data—normally 512 bytes, although some disks have larger sectors. The disk hardware itself does little to help organize data on the disk, aside from providing a means to read and write specific sectors. On-disk data management is left up to

the OS. Disk partitions and filesystems are two levels of organization imposed on disks to help manage the data they store.

Partitions are a lot like the drawers in a filing cabinet. Think of a single disk as the main filing cabinet, which is then split up into multiple partitions, much like drawers. This analogy is good as far as it goes, but it has its limits. Unlike filing cabinet drawers, disk partitions can be created in whatever size and quantity are convenient, within the limits of the disk's size. A typical disk has between one and a dozen partitions, although you can create more.

Disk partitions exist to help subdivide the disk into pieces with broadly different purposes, such as partitions for different OSs or for different types of data within an OS. For instance, it's common to create separate partitions for swap space (which is used much like RAM in case you run out of RAM), for user data files, and for the OS itself.

Hard disks and their partitions are frequently represented in diagrams similar to Figure 5.1. This diagram displays partitions as subdivisions of the disk, with partition sizes in the diagram more or less proportional to their true sizes on the disk. Thus, in Figure 5.1 you can see that `/boot` is tiny compared to `/home`. As in the figure, partitions are uninterrupted sections of a disk—that is, `/home`, for instance, is a set of sectors with no other partition carved out of its interior.

Some partitioning tools represent their partitions in a vertical stack rather than a horizontal chain. The principle is the same either way.

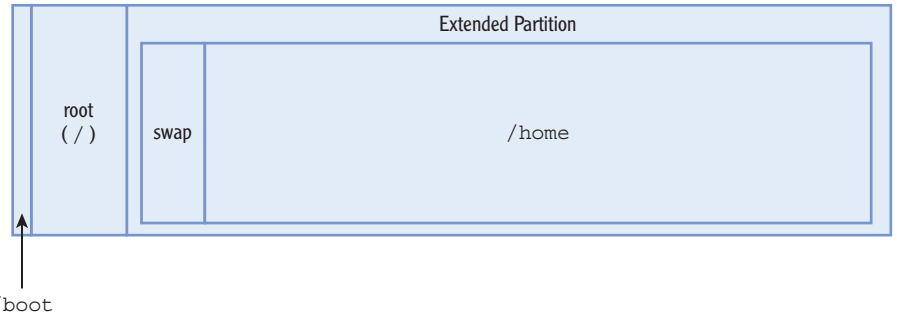


FIGURE 5.1 **Disk partitions are often visualized as boxes within a hard disk.**

The most common partitioning scheme for *x*86 and *x*86-64 computers has gone by various names over the years, including *master boot record (MBR)*, *MS-DOS*, and *BIOS partitioning*. It supports three types of partitions:

Primary This is the simplest type of partition. A disk can have zero to four primary partitions, one of which may be an extended partition.

Extended This is a special type of primary partition that serves as a placeholder for logical partitions. A disk may have at most one extended partition.

Logical These partitions are contained within an extended partition. In theory, a disk can have billions of logical partitions, thus overcoming the limit of four primary partitions, but in practice you're unlikely to see more than about a dozen of them.

MBR's use of three partition types is awkward and limiting, but inertia has kept it in place for three decades. MBR partitions have a hard limit, though: They can't support disks larger than 2 TiB (tebibytes), assuming 512-byte sectors, which are nearly universal today.

The Globally Unique Identifier (GUID) Partition Table (GPT) is the successor to MBR. GPT supports disks of up to 8 ZiB (zebibytes), which is about 4 billion times as large as MBR's limit. GPT also supports up to 128 partitions by default, with no distinction between primary, extended, and logical partitions. In these respects, GPT is a superior partitioning system to MBR; however, its support varies between OSs. Linux supports both systems quite well. Windows can boot only from MBR when the computer uses the Basic Input/Output System (BIOS), and it can boot only from GPT when the computer is based on the Unified Extensible Firmware Interface (UEFI). Thus, if you dual-boot with Windows, you may need to select your partitioning system with care.

1 TiB is 2^{40} bytes, whereas 1 ZiB is 2^{70} bytes.

Multi-Byte Units

It's common to use prefixes from the International System of Units (SI units)—*kilo* (*k*), *mega* (*M*), *giga* (G), *tera* (T), and so on—in conjunction with *byte* (B) to refer to large quantities of storage space, as in kB, MB, and so on. Technically, these units are defined as base-10 values—*kilo* means 1,000, *mega* means 1,000,000, and so on. In computers, though, base-2 values, such as 2^{10} (1024) and 2^{20} (1,048,576), are often more natural, so the SI units have often (but not always) been used to mean these base-2 values. This practice has led to confusion, since it's not always clear whether base-10 or base-2 units are being used.

To resolve this conflict, the Institute of Electrical and Electronics Engineers (IEEE) defined a new set of prefixes as IEEE-1541. Under this system, new units and prefixes describe base-2 values. The first few of these are:

- A kibibyte (KiB) is 2^{10} (1024) bytes.

(Continues)

MULTI-BYTE UNITS *(Continued)*

- A mebibyte (MiB) is 2^{20} (1,048,576) bytes.
- A gibibyte (GiB) is 2^{30} (1,073,741,824) bytes.
- A tebibyte (TiB) is 2^{40} (1,099,511,627,776) bytes.

In this book, I use IEEE-1541 units when describing features that are best expressed in this system, such as partition table size limits. Most Linux disk utilities use SI and IEEE-1541 units correctly, but which is used depends on the whim of the programs' authors. Be alert to this difference, particularly when dealing with large numbers—note that a tebibyte is almost 10 percent larger than a terabyte!

Several other partitioning systems exist, but you're unlikely to encounter most of them. One possible exception is the Apple Partition Map (APM), which Apple used on its Macintoshes prior to its switch to Intel CPUs.

When it comes to partitioning a disk, Linux supports three families of tools:

`fdisk` family The `fdisk`, `cfdisk`, and `sfdisk` tools are simple text-based partitioning utilities for MBR disks and some more exotic partition table types. These tools work well and provide the means to recover from some disk errors, but their text-based nature can be intimidating to those who are unfamiliar with disk partitioning.

`libparted`-based tools Tools based on the `libparted` library can handle MBR, GPT, and several other partition table types. Some of these tools, such as GNU Parted, are text based, but others, such as GParted, are GUI, and so are likely to be easier for new users to use. Figure 5.2 shows GParted in action. Note how its display mirrors the structure shown in Figure 5.1. Many Linux installers include `libparted`-based partitioning tools that run during system installation.

GPT fdisk family The `gdisk`, `cgdisk`, and `sgdisk` tools are modeled after the `fdisk` family but work with GPT disks. They provide more options for handling GPT than do `libparted`-based tools, but at the cost of friendliness for new users.

I am the author of the GPT fdisk family of partitioning tools.

If you're working with a pre-installed Linux system, you may not need to partition your disk; however, if you ever replace or install a new hard disk, you'll have to partition it before you can use it. You may also need to partition removable disks, although they generally come from the factory pre-partitioned with one big partition.

To partition a disk, you must know the disk's device filename. In Linux, these filenames are normally `/dev/sda`, `/dev/sdb`, and so on, with each disk taking on a new letter. Partitions are numbered starting with 1, so you might refer to `/dev/sda2`, `/dev/sdb6`, and so on. When using MBR, partitions 1 through 4 are reserved for primary or extended partitions, whereas logical partitions take numbers 5 and up.

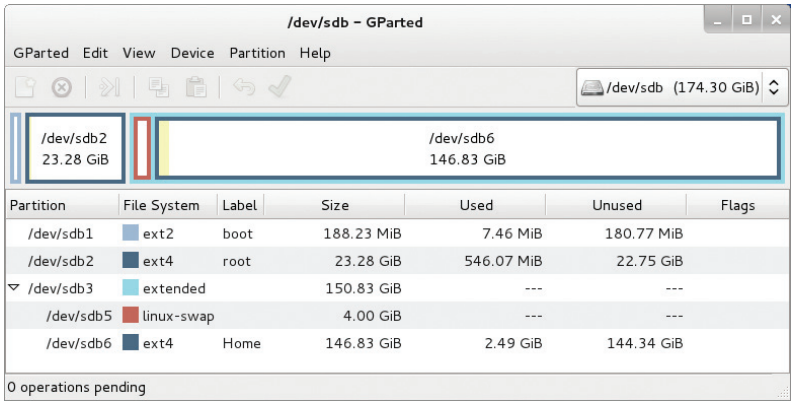


FIGURE 5.2 GParted, like other GUI disk partitioning tools, provides a graphical representation of your partitions.

Understanding Filesystem Issues

Most disk partitions contain filesystems, which are data structures that help the computer organize your directories and files. In Windows, each filesystem receives its own device letter, such as `A:` and `B:` for floppy disks, `C:` for the first hard disk partition (normally the boot partition), and so on. In Linux, by contrast, all filesystems are part of a single directory tree. The main filesystem is referred to as the root (`/`) filesystem. If a disk has multiple filesystem partitions, each is *mounted* at a *mount point* in the root (`/`) filesystem—that is, the contents of the additional filesystems are made available at specific directories, such as at `/home` (which holds users' data files) or `/boot` (which holds boot files). Several Linux filesystems exist, each with its own unique features:

Ext2fs The Second Extended Filesystem (ext2fs) was popular in the 1990s but is rarely used today because it lacks a *journal*, which is a filesystem feature that speeds filesystem checks after power outages or system crashes. A journal consumes disk space, though, so ext2fs is still useful on small disks. You might want to use it for a separate `/boot` partition, for instance, since such partitions are rather small. Its Linux filesystem type code is `ext2`.

The word *filesystem* is sometimes applied to the directory structure as a whole, even if it contains multiple low-level filesystems. Which meaning is intended is usually clear from the context.

The original Extended Filesystem (extfs) was used in the early 1990s but was quickly eclipsed by ext2fs. Extfs is no longer supported.

Ext3fs The Third Extended Filesystem (ext3fs) is essentially ext2fs with a journal. Until around 2010, it was a very popular filesystem, but ext4fs has taken its place. It supports files of up to 2 TiB and filesystems of up to 16 TiB (ext2fs imposes the same limits). Its Linux type code is `ext3`.

Ext4fs The Fourth Extended Filesystem (ext4fs) is a further development of the ext filesystem line. It adds speed improvements and the ability to handle larger files and disks—files may be up to 16 TiB in size, and filesystems may be up to 1 EiB (2^{60} bytes). Linux utilities refer to it as `ext4`.

Current partitioning tools limit ext4 filesystem sizes to 16 TiB; however, this limit will be raised sooner or later.

ReiserFS This filesystem, referred to as `reiserfs` by Linux tools, is similar to ext3fs in features, with an 8 TiB file-size limit and 16 TiB filesystem-size limit. Its best feature is its capacity to make efficient use of disk space with small files—those with sizes measured in the low kibibyte range. ReiserFS development has slowed, but it remains usable.

JFS IBM developed its Journaled File System (JFS) for its AIX OS, and its code eventually worked its way into Linux. JFS supports maximum file and filesystem sizes of 4 PiB and 32 PiB, respectively (1 PiB is 1024 TiB). JFS is not as popular as many other Linux filesystems. Linux tools use `jfs` as its type code.

XFS Silicon Graphics developed the Extents File System (XFS; Linux type code `xfs`) for its IRIX OS and later donated its code to Linux. XFS supports files of up to 8 EiB and filesystems of up to 16 EiB, making it the choice for *very* big disk arrays. XFS works well with large multimedia and backup files.

Btrfs This new filesystem (pronounced "butter-eff-ess" or "better-eff-ess") is intended as the next-generation Linux filesystem. It supports files of up to 16 EiB and filesystems of the same size. It also provides a host of advanced features, such as the ability to combine multiple physical disks into a single filesystem. As of early 2012, Btrfs is still experimental, but it may provide the best overall feature mix once it's finished. Its Linux type code is `btrfs`.

It's possible to use several filesystems in a single Linux installation, to take advantage of the benefits of each filesystem for different sets of files.

If you're planning a new Linux installation, you should consider ext4fs, ReiserFS, or XFS as your filesystems. Currently, ext4fs provides the best overall features and performance, while ReiserFS and XFS are worth considering for volumes that will hold particularly small and large files, respectively. Ext4fs is a good choice for volumes that hold large files, though, so you could use ext4fs for everything and not go far wrong, particularly on a general-purpose computer.

In addition to Linux's native filesystems, the OS supports several other filesystems, some of which are important in certain situations:

FAT The File Allocation Table (FAT) filesystem was the standard with DOS and Windows through Windows Me. Just about all OSs support it. Its compatibility also makes it a good choice for exchanging data between two OSs that dual-boot on a single computer. Unlike most filesystems, FAT has two Linux type codes: `msdos` and `vfat`. Using `msdos` causes Linux to use the filesystem as DOS did, with short filenames with at most 8 characters plus a 3-character extension (*8.3 filenames*); when you use `vfat`, Linux supports long filenames on FAT.

FAT's simplicity and widespread support make it a popular filesystem on floppy disks, USB flash drives, cell phones, e-book readers, digital camera media, and so on.

NTFS Microsoft developed the New Technology File System (NTFS) for Windows NT, and it is the default filesystem for recent versions of Windows. Linux provides a limited read/write NTFS driver, and a full read/write driver is available in the NTFS-3g software (`http://www.tuxera.com`). You're most likely to encounter it on a Windows boot partition in a dual-boot configuration or on larger removable or external hard disks. Under Linux, the standard kernel driver is known as `ntfs`, whereas the NTFS-3g driver is called `ntfs-3g`.

Linux's `ntfs` driver is based in the kernel, which makes it fast. The `ntfs-3g` driver, unlike most filesystem drivers, is *not* kernel-based, so it's not as fast.

HFS Apple used its Hierarchical File System (HFS) in Mac OS through version 9.*x* and still supports it in Mac OS X. You might encounter HFS on some removable media, and particularly on older disks created under pre-X versions of Mac OS. Linux provides full read/write HFS support using its `hfs` driver.

HFS+ Apple's HFS+, also known as *Mac OS Extended*, is the current filesystem for Mac OS X; you're likely to encounter it on dual-boot Macs and on some removable media created for use with Macs. Linux provides read/write HFS+ support with its `hfsplus` driver; however, write support is disabled by default on versions of the filesystem that include a journal.

Mac users often use FAT on removable media for compatibility reasons.

ISO-9660 This filesystem is used on optical media, and particularly on CD-ROMs and CD-Rs. It comes in several levels with differing capabilities. Two extensions, *Joliet* and *Rock Ridge*, provide support for long filenames using Windows and Unix standards, respectively. Linux supports all these variants. You should use the `iso9660` type code to mount an ISO-9660 disc.

UDF The Universal Disk Format (UDF) is a filesystem that's intended to replace ISO-9660. It's most commonly found on DVD and Blu-ray media, although it's sometimes used on CD-Rs as well. Its Linux type code is, naturally, `udf`.

Most non-Linux filesystems lack support for the Unix-style ownership and permissions that Linux uses. Thus, you may need to use special mount options to set ownership and permissions as you want them. Exceptions to this rule include HFS+ and ISO-9660 when Rock Ridge extensions are in use. Rock Ridge discs are generally created with ownership and permissions that enable normal use of the disc, but if you're faced with an HFS+ disk, you may find that the user ID (UID) values don't match those of your Linux users. Thus, you may need to copy data as `root`, create an account with a matching UID value, or change the ownership of files on the HFS+ disk. (This last option is likely to be undesirable if you plan to use the disk again under Mac OS X.)

To access a filesystem, you must mount it with the `mount` command. For instance, to mount the filesystem on `/dev/sda5` at `/shared`, you would type the following command:

```
# mount /dev/sda5 /shared
```

The `umount` command's name has just one `n`.

Alternatively, you can create an entry for the filesystem in the `/etc/fstab` file, which stores information such as the device file, filesystem type, and mount point. When you're done using a filesystem, you can unmount it with the `umount` command, as in **`umount /shared`**.

Using Removable and Optical Disks

Certification Objective

If you insert a removable disk into a computer that's running most modern Linux distributions, the computer will probably detect that fact, mount the disk in a subdirectory of `/media`, and launch a file manager on the disk. This behavior makes the system work in a way that's familiar to users of Windows or Mac OS.

Some devices, such as optical disc drives, can lock their eject mechanisms to prevent forced removal of the media.

When you're done using the disk, you *must* unmount it before you can safely remove it. Most file managers enable you to do this by right-clicking the entry for the disk in the left-hand pane and selecting an option called Unmount, Eject Volume, or Safely Remove, as shown in Figure 5.3. If you fail to do this, the filesystem may suffer damage.

Most removable disks are either unpartitioned or have a single partition. They frequently use FAT, which is a good choice for cross-platform compatibility. If you need to, you can partition USB flash drives and most other removable media.

Optical discs are unusual in that they require their own special filesystems (ISO-9660 or UDF). Although you can mount and unmount these discs just like other disks, you can only read them, not write to them. If you want to create an optical disc on blank media, you must use special software, such as the text-mode `mkisofs`, `cdrecord`, or `growisofs`, or the GUI K3B or X-CD-Roast. These tools create an ISO-9660 filesystem from the files you specify and then burn it to the blank disc. Thereafter, you can mount the disc in the usual way.

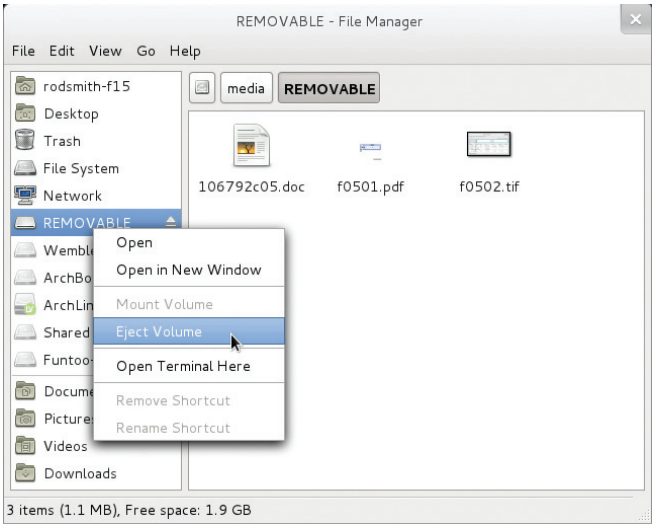


FIGURE 5.3 **Linux file managers enable you to unmount removable media.**

Managing Displays

Certification Objective

Linux provides two display modes: text-mode and GUI. A text-mode display is fairly straightforward and requires little or no management. GUI displays, on the other hand, are more complex. In Linux, the X Window System (or X for short) manages the GUI display. In the next few pages, I describe what X is and how X interacts with common display hardware.

Understanding the Role of X

Most people don't give much thought to the software behind their computers' displays; it all just works. Of course, behind the scenes the task of managing the display is fairly complex. Just some of the things that the software must do, on any platform, includes:

1. Initialize the video card, including set its resolution
2. Allocate sections of the display to hold windows that belong to particular applications
3. Manage windows that overlap so that only the "topmost" window's contents are displayed

4. Manage a pointer that the user controls via a mouse or similar device
5. Direct user input from a keyboard to whatever application is active
6. Display text and simple shapes within windows, at the request of user programs
7. Provide user interface elements to move and resize windows
8. Manage the interiors of windows, such as displaying menus and scroll bars

Desktop environments include window managers, but window managers without desktop environments are also available.

In Linux, tasks 1 through 6 are handled by X, but task 7 is handled by a program called a *window manager* and task 8 is handled by GUI libraries known as *widget sets*. The font display element of task 6 can be handled by X, but in recent years many programs have begun using a library called Xft for this task. Thus, the overall job of handling the display is broken up across several programs, although X handles most of the low-level tasks.

Modern Linux distributions use a version of X that can detect your hardware—including the video card, monitor, keyboard, and mouse—and configure itself automatically. The result is that the software normally works properly without any explicit configuration. Sometimes, though, this auto-configuration fails. When this happens, you must manually edit the X configuration file, `/etc/X11/xorg.conf`. If this file is missing, you can generate a sample file by typing the following command (with X *not* running) as `root`:

Chapter 11, "Editing Files," describes the `pico`, `nano`, and Vi text-mode text editors.

```
# Xorg -configure
```

The result is normally a file called `/root/xorg.conf.new`. You can copy this file to `/etc/X11/xorg.conf` and begin editing it to suit your needs. This task is complex and is beyond the scope of this book, but knowing the name of the file can help you get started—you can examine the file and locate additional documentation by searching on that name.

Using Common Display Hardware

Certification Objective

Much of the challenge in dealing with video devices is in managing drivers for the video chipsets involved. Most modern computers use one of a handful of video drivers:

- `nv`, `nouveau`, and `nvidia` work with NVIDIA video hardware.
- `ati` and `fglrx` work with AMD/ATI video hardware.
- `intel` works with Intel video hardware.

- The `fbdev` and `vesa` drivers are generic drivers that work with a wide variety of hardware, but they produce suboptimal performance.
- Many older video cards use more obscure drivers.

The `nvidia` and `fglrx` drivers are proprietary drivers provided by their manufacturers. Check the manufacturers' Web sites for details, or look for packages that install these drivers. These proprietary drivers provide features that aren't available in their open source counterparts, although the `nouveau` driver implements some of these features. In this context, video driver "features" translate into improved performance, particularly with respect to 3D graphics and real-time displays (as in playing back video files).

In the past, most video cards connected to monitors using a 15-pin Video Graphics Array (VGA) cable. Today, 29-pin Digital Visual Interface (DVI) cables are quite common. (Figure 5.4 shows both types.) DVI has the advantage of being a digital interface, which can produce a cleaner display on modern liquid crystal display (LCD) monitors.

High Definition Multimedia Interface (HDMI) is another cable type. HDMI and DVI are similar, but HDMI is more common on televisions, whereas DVI is more common on computers.



FIGURE 5.4 VGA connectors (left) were common in years past and are still available today; DVI connectors (right) are common on newer monitors and video cards.

Monitor resolutions are typically measured in terms of horizontal and vertical number of pixels. In the past, resolutions of as low as 640×480 have been common, but today it's rare to use a monitor that has an optimum resolution of lower than 1280×1024 or 1366×768, and resolutions of 1920×1080 or higher are commonplace. You should consult your monitor's manual to determine its optimum resolution. Typically, physically larger monitors have higher resolutions; however, this isn't always true.

Most monitors use an *aspect ratio* of either 4:3 or 16:9, referring to the ratio of the length to the height of the display.

In the best of all possible worlds, Linux will auto-detect your monitor's optimum resolution and set itself to that value whenever you boot the computer. Unfortunately, this sometimes doesn't work. Keyboard/video/mouse (KVM) switch boxes and extension cables can sometimes interfere with this auto-detection, and

older monitors might not support the necessary computer/monitor communication. Thus, you may need to crack open your monitor's manual to learn what its optimum resolution is. Look for this information in the features or specifications section; it will probably be called *optimum resolution*, *maximum resolution*, or something similar. It may also include a refresh rate value, as in *1280×1024 @ 60 Hz* .

In most cases, you can set the resolution using a GUI tool such as the Displays item in GNOME System Settings panel, shown in Figure 5.5. In Figure 5.5, the Resolution button enables you to set the resolution to any desired value. If you can't find the optimum resolution in the list, you may need to perform more advanced adjustments involving the `/etc/X11/xorg.conf` file—a topic that's beyond the scope of this book. On rare occasion, you may need to upgrade your video card; some cards aren't able to handle the optimum resolutions used by some monitors.

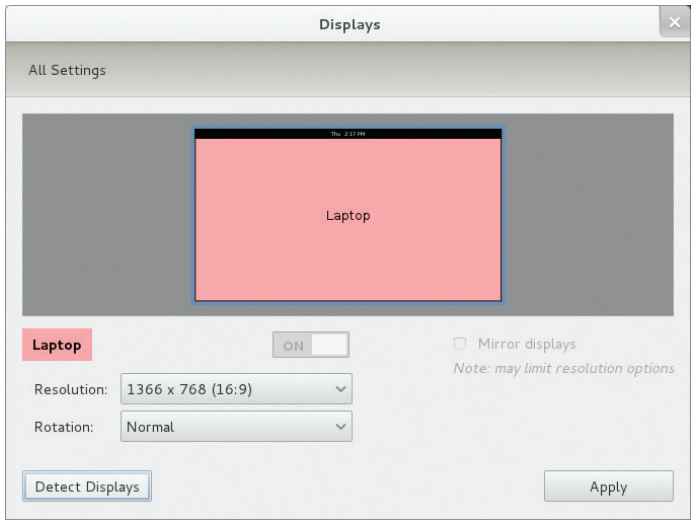


FIGURE 5.5 **Most desktop environments provide GUI tools to help you set your display's resolution.**

Handling USB Devices

Certification Objective

Most modern computers use USB as the primary interface for external peripherals. Keyboards, mice, cameras, flash storage, hard disks, network adapters, scanners, printers, and more can all connect via USB. For the most part, USB devices

work in a plug-and-play manner—you plug them in and they work. Some specific caveats include the following:

Human interface devices X usually takes over keyboards, mice, tablets, and similar devices when you plug them in. If you have problems, you may need to adjust your X configuration by editing `/etc/X11/xorg.conf`, but this is an advanced topic that's beyond the scope of this book.

Disk storage I include USB flash drives, external hard disks, and other storage devices in this category. As described earlier, in "Using Removable and Optical Disks," it's critical that you unmount the disk before you unplug its USB cable. Failure to do so can result in data corruption.

Cell phones, cameras, e-book readers, and music players You can often use these devices like disk devices to transfer photos, music, or other files. You may need to activate an option on the device to make it look like a disk device to the computer, though. When you're done, unmount the device and deactivate its interface mode.

Scanners Linux uses the Scanner Access Now Easy (SANE; `http://www.sane-project.org`) software to handle most scanners. A few require obscure or proprietary software, though.

Printers Most distributions automatically configure suitable printer queues when you plug in a USB printer. If you need to tweak the configuration, try entering **`http://localhost:631`** in a Web browser on the computer in question. Doing so opens a Web-based printer configuration utility. Some distributions provide distribution-specific printer configuration tools, as well.

Managing Drivers

Certification Objective

Most hardware devices require the presence of special software components to be useful. A piece of software that "talks" to hardware is known as a *driver*, so you should know how drivers work in Linux. Several broad classes of drivers exist, so I begin by describing those. Whatever the driver type, you should know how to locate and install drivers for your hardware.

Understanding Types of Drivers

Linux requires drivers because different hardware—even two devices that serve very similar purposes, such as two network adapters—can function in very different

In fact, several layers exist between the network hardware and a program like Firefox; the driver is just one of these layers.

ways. That is, the methods required to initialize and use two network adapters may be entirely different. To provide generalized interfaces so that programs like the Firefox Web browser can use any network adapter, the Linux kernel uses drivers as a bridge between the hardware-agnostic kernel interfaces and the hardware itself.

Broadly speaking, drivers can exist in one of two locations:

- The kernel
- A library or application

Most drivers are kernel based, and in fact a large chunk of the Linux kernel consists of drivers. Kernel drivers handle most of the devices that are internal to the computer's box, such as the hard disks, network interfaces, and USB interfaces. The kernel hosts most drivers because drivers typically require privileged access to hardware, and that's the purpose of the kernel.

Some drivers reside in a library or application. Many of these devices are external to the computer itself. Examples include:

- SANE, which handles scanners
- Ghostscript, which converts printed output into a form that particular printers can understand
- X, which manages the display

X is unusual in that it's a non-kernel element that communicates more or less directly with the video hardware. SANE and Ghostscript, by contrast, both communicate with external hardware devices via interfaces (such as a USB port) that are handled by the kernel. That is, you need at least two drivers to handle such devices. To print to a USB printer, for instance, you use the kernel's USB driver and a Ghostscript printer driver. Ideally, most users will be unaware of this complexity, but you may need to understand it if problems arise.

Locating and Installing Drivers

Most drivers come with the Linux kernel itself, or with the library or application that handles the type of hardware. For instance, most X installations include a set of video drivers so that you can use most video cards. For this reason, it's seldom necessary to track down and install additional drivers for common hardware. There are exceptions; for instance:

New hardware If your hardware is very new (meaning the model is new, not just that you purchased it recently), it might need drivers that haven't yet made their way into whatever distribution you're using.

Unusual hardware If you're using very exotic hardware, such as a specialized scientific data-acquisition board, you may need to track down drivers for it.

Proprietary drivers Some manufacturers provide proprietary drivers for their hardware. For instance, the `nvidia` and `fglrx` video drivers (referred to earlier, in "Using Common Display Hardware") can improve the performance of video displays based on NVIDIA or ATI/AMD chipsets, respectively. Some hardware requires proprietary drivers. This is particularly common for some exotic hardware.

Bug fixes Drivers, like other software, can be buggy. If you run into such a problem, you might want to track down a more recent driver to obtain a bug fix.

One way to obtain a new kernel-based driver is to upgrade the kernel. Note that a kernel upgrade can provide both bug fixes to existing drivers and entirely new drivers. Similarly, upgrading software such as SANE, Ghostscript, or X can upgrade or add new drivers for the devices that such packages handle.

If you're using exotic hardware or need some other hard-to-find driver, your task can be more difficult. You can check with the manufacturer or perform a Web search to try to find drivers.

If you obtain a driver that's not part of the kernel (or software package to handle the device), you should read the instructions that come with the driver. Installation procedures vary quite a bit from one driver to another, so it's impossible to provide a simple step-by-step installation procedure that works in all cases. Some drivers come with installation utilities, but others require you to follow a procedure that involves typing assorted commands. If you're very unlucky, the driver will come in the form of a *kernel patch*. This is a way to add or change files in the main kernel source code package. You must then recompile the kernel—a task that's well beyond the scope of this book.

The Essentials and Beyond

Software and hardware interact in numerous ways to determine a computer's capabilities. Your CPU is one determinant of your computer's speed, and it also determines what version of Linux you can run. CPUs are mounted on motherboards, which contain other critical circuitry for managing hard disks, displays, and other devices. Your hard disk must be partitioned and prepared with one or more filesystems before it's useful. Video hardware—both the monitor and the video circuitry inside the computer—determine how your desktop environment looks and how fast the computer can move windows and display videos. USB manages most external devices, such as keyboards, mice, and external disks. Software known as drivers manages all these hardware devices.

(Continues)

9. Managing Network

Networking is a critical part of modern computing. Fortunately, most Linux distributions can often create a network connection automatically; however, sometimes this process doesn't work, or you may need to tweak the connection or debug problems when they occur. This chapter therefore covers network configuration.

I begin by describing some basic network features that you must understand to properly manage your network configuration. I then describe the process of configuring a network connection. A few tests will help you track down and correct problems, so I explain them in case you run into difficulties. Finally, I explore the dark side to networking: the undesirables of the Internet and how to keep them out of your computer.

- **Understanding network features**
- **Configuring a network connection**
- **Testing your network connection**
- **Protecting your system from the bad guys**

Understanding Network Features

Networking involves numerous protocols and technologies that interact in complex ways. Although modern networks and OSs make it easy for users to set up their computers, at least compared to 10 or 20 years ago, doing anything but the simplest configuration requires understanding the key networking protocols and technologies. To begin, I present the following mini-glossary of networking terms:

Certification Objective

DNS The *Domain Name System (DNS)* is a global network of servers that translate between *hostnames* and *IP addresses* (both described shortly). Most

Other means of name resolution exist but are impractical for handling more than a few dozen computers.

computers are configured to use a *DNS server*, which is a server that performs DNS lookups.

DHCP The *Dynamic Host Configuration Protocol (DHCP)* is a way for most computers on a network to obtain configuration information from another computer (the *DHCP server*) on the network. DHCP is the key to today's easy network configuration. It's described in more detail shortly, in the section "Deciding Whether to Use DHCP."

Other methods of wired networking exist, but most are either obsolete or are used in very high-speed, long-distance, or other capacities.

Ethernet This is the wired network hardware that's most often used today. Ethernet comes in several varieties, the most common of which use cables that resemble telephone wires but with broader plugs. Common speeds range from 10 megabits per second (Mbps) to 1 gigabit per second (Gbps, or *gigabit Ethernet*), although faster speeds are available.

Hostname This is a name that a computer uses for the benefit of humans. Hostnames consist of a computer portion and a network portion. For instance, in `lunokhod.luna.edu`, `lunokhod` is the machine name and `luna.edu` is the network name.

Internet Used generically (with a lowercase *i*), the word *internet* refers to a network of networks, connected via *routers* (described shortly). With a capital *I*, the word *Internet* refers to the globe-spanning network with which you're no doubt familiar. The Internet is the largest internet ever created.

Certification Objective

IP address An *Internet Protocol (IP) address* is a number that's assigned to a computer for network addressing purposes. You can think of an IP address as being like a street address or telephone number. Computers communicate with each other via IP addresses. In the past, four-byte *IPv4* addresses have dominated; however, the pool of IPv4 addresses has recently been depleted, so many new installations now use 16-byte *IPv6* addresses. IP addresses are broken down into two parts: a machine portion and a network portion. The network portion may be used in routing (see *router*), whereas the machine portion identifies a computer on a given network. This breakdown into machine and network addresses is accomplished via a *network mask*, as described next.

Network mask A *network mask* (*mask* or *netmask* for short) is a way to distinguish between the network and machine portions of an IP address. The netmask defines the individual bits of an IP address that belong to each part of the address. This can be done by giving each bit of the machine address a binary 1 value and expressing the result as a number (such as 255.255.0.0 or 255.255.255.0) for an IPv4 address or by specifying the number of bits in the machine portion of the address (as in 16 or 24).

Router A *router* (also known as a *gateway*) connects two or more networks together. Your desktop or server computer is likely to connect directly to a handful of other computers, including a router. The router links to another network, which in turn links to another, and so on. To contact a computer on another network, your computer communicates through the router. Homes and small businesses often use small *broadband routers* to connect to the Internet. Such devices perform routing and have built-in DNS, DHCP, and other useful servers.

Certification Objective

TCP/IP The *Transmission Control Protocol/Internet Protocol (TCP/IP)* is a set of standards that underlie most modern network communications at the software level.

Several alternatives to TCP/IP exist, such as AppleTalk, NetBEUI, and IPX/SPX. TCP/IP, however, is the most common network protocol, and the one that underlies the Internet.

Wi-Fi This term applies to the most common forms of wireless networking, more properly known as *IEEE 802.11*. Several variants are available that differ in speed. The section "Creating a Wi-Fi Connection," later in this chapter, describes Wi-Fi configuration in more detail.

The process of creating a network connection is one of assigning the computer an IP address and an associated netmask. Using the IP address, the computer can communicate with other computers on the local network—that is, anything that doesn't require a router as an intermediary. In most cases, you'll want to tell the computer about the network's router so that it can communicate with computers on other network segments (often including the entire Internet). Giving your computer the IP address of a DNS server will enable you to specify computer hostnames rather than IP addresses, which is a highly desirable feature. Thus, these four features—the IP address, the netmask, the router's IP address, and the DNS server's IP address—constitute the basic network configuration features.

In most cases, DHCP can dish out these four features. This can make network configuration fairly automatic, as you'll see in this chapter. If your network doesn't support DHCP, though, you'll need to contact your network's administrator to obtain the critical configuration information. In some cases, you'll need still more information. For instance, if you're using Wi-Fi, you'll need to know the Wi-Fi network's name and perhaps its password.

Configuring a Network Connection

In most cases, your network connection will come up automatically (or very nearly so). Sometimes, though, you'll have to manually adjust the configuration, or simply activate it. If available, automatic configuration is handled via DHCP, so I begin by describing it in more detail. I then cover a topic that's applicable to only some systems: configuring Wi-Fi. If you use it, you must configure Wi-Fi before setting other network options. Once Wi-Fi is configured, the rest of the network setup

procedure is much like that for wired networks. I then provide an example of using GUI tools for network configuration; these tools are easy to use and can handle most situations. Finally, I cover text-based configuration tools, which provide a great deal of power that you might need to use in certain settings.

Deciding Whether to Use DHCP

As noted earlier, a computer requires, at a minimum, two pieces of information to connect to a typical network: an IP address and a netmask. IP addresses for a router and a DNS server are practical necessities on most networks, although in some cases you can do without one or both. Some networks require additional configuration details.

Configuring every computer on a large network with all of this information can be time-consuming. Worse, it can lead to problems caused by human errors, such as typos in IP addresses. For this reason, most networks provide a DHCP server, which can provide this information to other computers.

DHCP can deliver information even to computers without an IP address by using lower-level addressing methods.

Depending on the configuration of the server, DHCP can be used to deliver IP addresses in one of two ways:

- In a fixed way so that each computer receives the same IP address every time it boots
- Dynamically so that a single computer might receive different IP addresses on different boots

Which is used is a policy detail for your network administrator. You needn't be concerned with it, except to know that a computer configured via DHCP isn't necessarily guaranteed to receive the same IP address on each boot. This fact can have important implications. For instance, it's generally easiest and most reliable to configure a server computer if it has a fixed IP address. Thus, as a matter of policy, server computers are often configured *without* using DHCP, even if workstations on the same network use DHCP. This isn't always the case, though—as already noted, DHCP can deliver a fixed address to some or all computers, so the DHCP server's administrator can configure a network's servers in this way.

If you're configuring a home or small office network with a broadband router, the router almost certainly includes a DHCP server that you can use. Consult its documentation for details.

As a practical matter, unless you're configuring your own small private network, you should ask your network administrator whether to use DHCP for a new computer. If the answer is *no*, you should ask for an IP address, the netmask, the router's IP address, and the DNS server's IP address.

Creating a Wi-Fi Connection

Wi-Fi connections are most often used on laptop and smaller portable computers, since using such devices without a physical network cable is often desirable. Some desktop computers, though, have Wi-Fi capabilities, and you may want to use them.

Before proceeding further, be aware that you must have access to a Wi-Fi network. If you're configuring a computer to connect to a broadband router that you've purchased and set up, consult the router's documentation to learn how to locate the router's critical information, such as the *service set identifier* (SSID; that is, the Wi-Fi network's name, which may be distinct from the TCP/IP network's name) and the password it uses. If you're trying to connect to a business network or a public network, consult the network administrator to obtain this information.

Whatever the computer type, the easiest method of configuring your Wi-Fi connection is to use the GUI. Details of how to do this vary from one distribution to another. As an example, consider doing the task in Fedora:

1. Open the System Settings tool by clicking your name in the upper-right corner of the desktop and selecting System Settings from the resulting menu or by typing **`gnome-control-center`** at a shell prompt.
2. Click the Network item in the System Settings Hardware section.
3. Click Wireless in the list on the left side of the Network window. The window will change to resemble Figure 17.1.

The procedure described here is equivalent to plugging an Ethernet cable into an Ethernet port. This procedure might or might not set up TCP/IP options.

If you don't see a Wireless option, you may lack Linux drivers for your Wi-Fi hardware. Consult the sidebar "Obtaining Wi-Fi Drivers" for advice on resolving this problem.

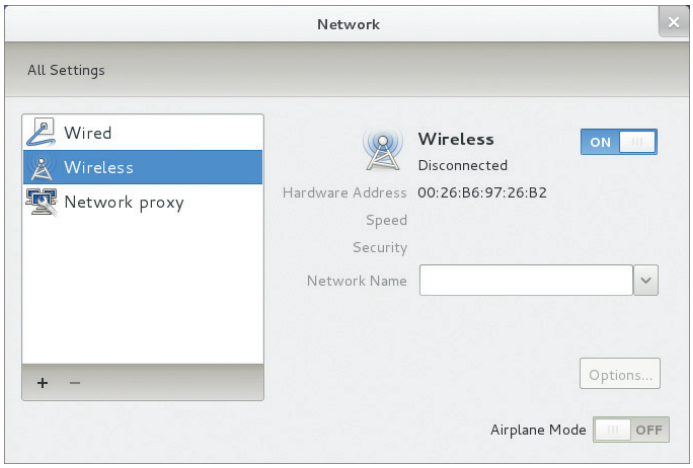


FIGURE 17.1 You can adjust the most common network settings from the Network item in the System Settings menu.

4. Click the inverted caret to the right of the Network Name field to access a pull-down menu with a list of networks.
5. Select your network from the list. If the network requires authentication, a dialog box like the one shown in Figure 17.2 will appear.
6. Type your network's password in the Password field, and then click Connect.



FIGURE 17.2 **If your Wi-Fi network requires authentication, you must enter a password before you can begin using it.**

The word Disconnected in the Network window (Figure 17.1) should change to read Connecting, followed by Connected. Depending on your network's configuration, you may also see an IP address, the default route, and the DNS server's IP address appear in the window. If so, the computer is now completely configured for network access—the utility has automatically performed the tasks described in the section "Using a Network Configuration GUI," later in this chapter. If not, though, you may need to perform additional configuration tasks, such as manually setting an IP address. It's also possible that the Wi-Fi connection has failed.

Wi-Fi Security

Because Wi-Fi data are carried over radio waves, it's easy for the transmissions to be intercepted. Somebody sitting in a car outside of a building can often gain access to a Wi-Fi network, which poses three problems:

- An intruder can intercept data passing over the network, possibly gaining access to sensitive files, acquiring credit card numbers, and so on.

(Continues)

Wi-Fi Security *(Continued)*

- An intruder may be able to launch attacks on your computers from behind any firewall you maintain to protect yourself from outsiders.
- An intruder can use your network access to attack other computers or to conduct illegal activities such as pirating movies, music, or software. This not only consumes your network bandwidth, but if the activities are detected, *you* will fall under suspicion!

For these reasons, you should secure any Wi-Fi network you manage. Three security levels are available (four if you count none at all):

WEP The Wired Equivalent Privacy (WEP) is a very weak encryption protocol. Methods of breaking into a WEP-encrypted network are well known, making this method essentially worthless.

WPA Wi-Fi Protected Access (WPA) security provides improved encryption and authentication tools compared to WEP.

WPA2 WPA2 is WPA's replacement, and it provides still more improvements in encryption and authentication.

Several variants on each of these encryption methods are available. For instance, WPA and WPA2 provide both *personal* and *enterprise* options, the latter requiring a more complex setup with a dedicated authentication server. Typically, you set the encryption method in a broadband router or wireless access point (WAP). Clients then detect the method in use and present appropriate options, such as the password prompt shown earlier in Figure 17.2.

Public Wi-Fi connections might not use any of these protocols. You can use such connections, but be aware that any data you send or receive might be visible to others.

The GUI method of establishing a Wi-Fi connection is almost always the easiest method to use. If you need to fine-tune the connection, though, you can use various tools to probe for and manage Wi-Fi links. Here are the two most important of these tools:

`iwlist` This command can identify nearby Wi-Fi networks. Type **`iwlist scan`** or **`iwlist scanning`** as `root` to obtain a list of nearby networks. This command produces a great deal of technical data, though. You're probably most interested in the network names, which appear in the output following the string `ESSID`, so typing **`iwlist scan | grep ESSID`** will pare down the output.

You might use `iwlist scan` to locate public Wi-Fi networks in a coffee shop or hotel, followed by an `iwconfig` command to connect to a network you've found.

iwconfig This utility connects and disconnects you from specific networks. It uses many options, and you may need to study its `man` page to fully understand it. In most cases, though, typing something like **`iwconfig wlan0 essid *NoWires* channel *1* mode *Managed* key *s:N1mP7mHNw***` will do the trick. This example connects to the *`NoWires`* network running on channel *`1`* using the password *`N1mP7mHNw`*.

Obtaining Wi-Fi Drivers

Unfortunately, Linux driver support for Wi-Fi hardware is fairly weak. If you don't see your Wi-Fi hardware when you try to configure it, you may need to track down suitable drivers. You can begin this task with a tool called `lspci`, which is described in Chapter 5, "Managing Hardware." Type this command with no options to see a list of available hardware and search that list for a wireless network adapter. For instance, my own laptop's `lspci` output includes the following line:

```
03:00.0 Network controller: Realtek Semiconductor Co., Ltd.↵
 RTL8191SEvB Wireless LAN Controller (rev 10)
```

This line identifies the Wi-Fi adapter as a Realtek RTL8191SEvB. A search on Realtek's Web site turns up a driver; however, this driver has to be compiled locally, which is a topic that's beyond the scope of this book. You also might not be lucky enough to find a driver in this way.

An alternative to using a native Linux driver is to use a Windows driver. This unusual option is possible using a package called `ndiswrapper` (`http://ndiswrapper.sourceforge.net`), which enables you to install Windows Wi-Fi drivers in Linux. Unfortunately, not all distributions provide `ndiswrapper` in their standard package sets, but you can usually find a binary package in an add-on repository. Similar functionality is provided by the Linuxant DriverLoader software (`http://www.linuxant.com/driverloader/`), which is less popular than `ndiswrapper` but is still worth considering.

If all other options fail, you may need to buy new networking hardware. Many USB Wi-Fi adapters are available, but you should research them to find one that has good Linux support. You can also replace the built-in adapters on some laptops.

Using a Network Configuration GUI

If you've configured a Wi-Fi link but it hasn't obtained an IP address, or if you need to set up a wired network connection, you'll have to set up the TCP/IP networking options. You can do this via text-mode tools, as described later, in "Using Text-Based Tools," or you can use GUI tools. The latter option is easier for new users, but the GUI tools vary from one distribution to another. As an example, the procedure for configuring Fedora using GUI tools is as follows:

1. Open the System Settings tool by clicking your name in the upper-right corner of the desktop and selecting System Settings from the resulting menu or by typing **`gnome-control-center`** at a shell prompt.
2. Click the Network item in the System Settings Hardware section.
3. Select the network device from the list on the left side of the window. The result should resemble Figure 17.3; however, this figure shows a properly configured network link that has no obvious problems. If your network link isn't working, it might read Disconnected rather than Connected or it might otherwise show significantly different information.
4. Click Options to adjust your network settings.

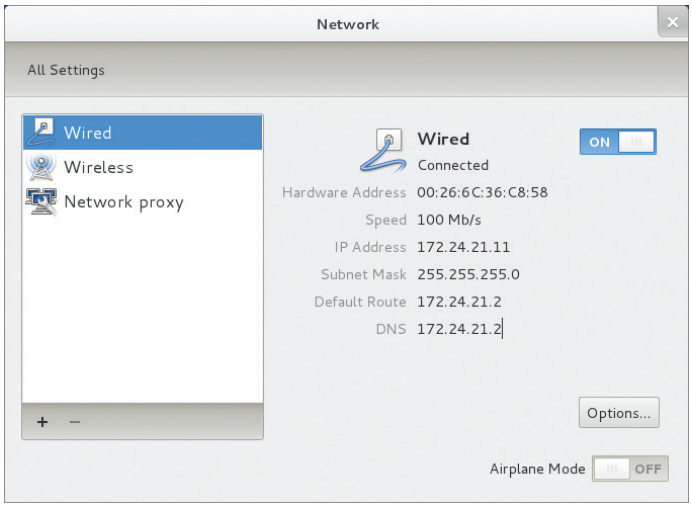


FIGURE 17.3 Network connections usually set themselves up automatically, but sometimes they don't.

You can configure a computer for both IPv4 and IPv6. To do so manually, however, you must have both IPv4 and IPv6 addresses.

5. Select the IPv4 Settings or IPv6 Settings tab, depending on whether your network uses IPv4 or IPv6. The result resembles Figure 17.4, which shows a partially configured manual IPv4 setup.

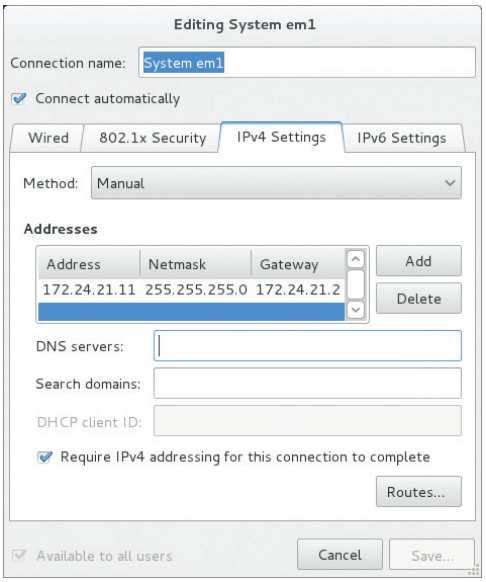


FIGURE 17.4 You can set key TCP/IP features after clicking Options in the Network window.

6. Select the connection method using the Method button. The Automatic (DHCP) and Manual options are the most common, but a few more obscure options are available, too. You can disable a network connection by selecting Disabled.
7. If you're using DHCP, you can type your computer's DHCP client ID in the DHCP Client ID field. Some networks use this information to assign the computer a hostname or a fixed IP address. If your network administrator hasn't given you this information, you can leave this field blank.
8. To perform manual configuration, select Manual in the Method button and enter the remaining information. You enter an IP address, netmask, and router address by clicking the Add button, whereupon suitable fields in the Addresses section of the dialog box become available.
9. When you're done making changes, click Save.

If all goes well, your Network dialog box should change to show an IP address, subnet mask, default route, and DNS server address—although some networks omit some of these items.

Using Text-Based Tools

Although the GUI network configuration tools are easy to use and work well most of the time, sometimes you must use a more flexible set of text-based tools. These tools include the following:

Certification Objective

`ifconfig` This program brings up or shuts down a network connection by associating an IP address and network mask to a piece of network hardware.

Certification Objective

`route` This program adjusts the computer's *routing table*, which determines through which network device specific network packets are sent.

Certification Objective

`/etc/resolv.conf` This file contains the IP addresses of up to three DNS servers as well as the name of the computer's Internet domain and of other domains that should be searched when the user omits a domain name from a hostname.

A DHCP client A DHCP client program, such as `dhclient` or `dhcpcd`, can often configure a network connection automatically. You simply type the program's name, perhaps followed by a network device name.

Don't confuse `dhcpcd`, a DHCP client, with `dhcpd`, a DHCP server.

Distribution-specific network scripts The `ifconfig`, `route`, and DHCP client programs produce temporary changes to the computer's network configuration. If you want to make permanent changes, you must store your settings in a configuration file. The name and format of this file varies from one distribution to another. For instance, in Fedora it's `/etc/sysconfig/network-scripts/ifcfg-`*`netname`*, where *`netname`* is the name of the network device; in Debian or Ubuntu, it's `/etc/network/interfaces`.

The `ifconfig` and `route` commands are both quite complex; however, their basic use is fairly straightforward. Suppose you want to assign the IP address 192.168.29.39/24 to the `eth0` network device and tell it to use 192.168.29.1 as the router. The following two commands would do the trick:

```
# ifconfig eth0 up 192.168.29.39 netmask 255.255.255.0
# route add default gw 192.168.29.1
```

You can instead take down a connection by using `down` rather than `up` with `ifconfig`, delete routes by using `del` rather than `add` with `route`, and so on. Consult these programs' `man` pages for details.

Some distributions use `dhclient` by default, but others use `dhcpcd`. If you don't know which to use, try both.

If your network uses DHCP, you can type **`dhclient` *`netname`*** or **`dhcpcd` *`netname`*** to bring up the *`netname`* interface and configure it using DHCP. Manually typing this command can be handy if you're trying to debug a network problem related to DHCP or if your network's DHCP configuration has changed and temporarily broken connectivity for some computers.

Network Device Names

Traditionally, Linux has given the first Ethernet device a name of `eth0` and subsequent devices names of `eth1` and so on. Similarly, Wi-Fi devices have acquired names of the form `wlan0` and up. Fedora, however, has begun deviating from this scheme, starting with version 15. This makes it harder to predict the device names of Ethernet devices from one computer to another. The goal is to make the device names more stable between reboots on computers with multiple network interfaces. Such systems are often routers or servers with multiple network links, and they typically require specific configurations on specific interfaces. If the device name changes from one boot to another—which can happen if network cards happen to be discovered in different order on different boots—problems can ensue.

You can learn your network device names by typing `ifconfig` with no options. (You can even do this as a normal user.) The result includes one or more blocks of output like the following:

```
em1   Link encap:Ethernet  HWaddr 00:26:6C:36:C8:58
      inet addr:172.24.21.11  Bcast:172.24.21.255 ↵
Mask:255.255.255.0
      inet6 addr: fe80::226:6cff:fe36:c858/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:15962 errors:0 dropped:0 overruns:0 ↵
frame:0
      TX packets:9150 errors:0 dropped:0 overruns:0 ↵
carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:22295082 (21.2 MiB)  TX bytes:2050687 (1.9 ↵
MiB)
      Interrupt:45
```

(Continues)

Network Device Names *(Continued)*

In this example, `em1` is the device name—it prominently begins this block of output. The name `lo` refers to a *loopback device*, which is used for certain local access procedures, so you can ignore it if you want to find your network hardware's device name.

If you're configuring your network settings manually, you should enter your DNS name server data and your network name into `/etc/resolv.conf`. A sample of this file appears in Listing 17.1.

Listing 17.1: A sample `/etc/resolv.conf` configuration file

```
domain luna.edu
search example.com example.org
nameserver 192.168.1.2
nameserver 10.78.102.1
nameserver 10.78.221.1
```

Three keywords identify DNS resolution features in `/etc/resolv.conf`:

`domain` You identify your computer's default domain name on this line. The primary effect is that the computer searches for hostnames without domain names in this domain. For example, in Listing 17.1, if you specify a hostname of `lunokhod`, the computer will find the IP address of `lunokhod.luna.edu`.

`search` You can have the computer search for hostnames in additional domains by specifying them on a `search` line, with domains separated by spaces or tabs. You can specify up to six domains in this way. Be aware, however, that adding domains to the search path is likely to slow down hostname lookup.

`nameserver` This keyword identifies DNS server computers by IP address. You can specify multiple servers by including multiple `nameserver` lines, up to the maximum of three.

If you want to manually adjust your permanent network configuration, you can do so by editing the network configuration file, such as `/etc/sysconfig/network-scripts/ifcfg-`*`netname`* on a Fedora system. Listing 17.2 shows a sample of this file.

◀

`/etc/resolv.conf` supports additional keywords, but they're seldom used. Consult its `man` page for details.

Listing 17.2: A sample network configuration file

```
DEVICE="em1"
BOOTPROTO="static"
IPADDR=192.168.29.39
NETMASK=255.255.255.0
NETWORK=192.168.29.0
BROADCAST=192.168.29.255
GATEWAY=192.168.29.1
ONBOOT=yes
NM_CONTROLLED="yes"
HWADDR=00:26:6C:36:C8:58
TYPE=Ethernet
DEFROUTE=yes
PEERDNS=yes
PEERROUTES=yes
IPV4_FAILURE_FATAL=yes
IPV6INIT=no
NAME="System em1"
UUID=1dad842d-1912-ef5a-a43a-bc238fb267e7
```

Some of these options have purposes that you can figure out based on their names, and you can adjust as necessary. If you don't understand an option, it's best to leave it alone. Some specific features you may want to attend to include the following:

- If you want to use DHCP, change the `BOOTPROTO` line to read `BOOTPROTO="dhcp"` rather than `BOOTPROTO="static"` and remove the lines from `IPADDR` through `GATEWAY`.
- If you want to keep the network link from starting up automatically when the computer boots, change `ONBOOT` to `no`. Users will then have to manually activate the network whenever they want to use it.
- The `NETWORK` and `BROADCAST` options are both derived from the IP address (`IPADDR`) and netmask (`NETMASK`). The `NETWORK` address is the network portion of the IP address but with binary 0 values substituted for the host-specific part of the IP address. The `BROADCAST` address is similar, but it substitutes binary 1 values for the host-specific part of the address.



Forcing users to start their network connections can limit the computer's use of network resources, such as IP addresses if your network uses DHCP.

If you're running Debian, Ubuntu, or some other distribution that uses a different network configuration file, you'll have to track down that file and modify it. The format may vary from the file shown in Listing 17.2, too, although the entries you must change are usually fairly obvious. If in doubt, consult distribution-specific documentation.

The `ifup` and `ifdown` commands bring up or take down a network connection based on the settings in your configuration file. Thus, after you change these settings, you may want to type **`ifdown`** ***`netname`*** followed by **`ifup`** ***`netname`***. The result should be a network link that reflects the new settings. Be aware, however, that this procedure may disconnect network clients and servers.

Testing Your Network Connection

In most cases, your network connection will work fine from the moment you start it. Sometimes, though, you may need to diagnose problems, either because you can't get the connection to work at all or because a formerly working connection has stopped working. In the following pages, I describe several types of network tests: checking your routing table, testing basic connectivity, testing to locate where a break in connectivity has occurred, testing DNS functioning, and testing the network's status.

Checking Your Routing Table

Certification Objective

Earlier, I described using the `route` command to set the default route on a computer. You can check that your route is sensible by using the same command. In most cases, typing **`route`** alone will do the job:

```
$ route
Kernel IP routing table
Destination  Gateway      Genmask       Flags Metric Ref  Use Iface
192.168.29.0 *            255.255.255.0 U     0      0      0 eth0
127.0.0.0    *            255.0.0.0     U     0      0      0 lo
default      192.168.29.1 0.0.0.0       UG    0      0      0 eth0
```

This shows that data destined for 192.168.29.0 (that is, any computer with an IP address between 192.168.29.1 and 192.168.29.254) goes directly over `eth0`. The 127.0.0.0 network is a special interface that "loops back" to the originating computer. Linux uses this for some internal networking purposes. The last line shows the default route, which describes what to do with everything that doesn't match any other entry in the routing table. This line identifies the default route's gateway system as 192.168.29.1. If it's missing or misconfigured, some or all traffic destined for external networks, such as the Internet, won't make it beyond your local network segment.

The routing table of a typical workstation or even server is quite simple and can be automatically configured. Network routers often require complex routing tables, though.

Testing Basic Connectivity

Certification Objective

The most basic network test is `ping`, which sends a simple network packet to the system you name (via IP address or hostname) and waits for a reply. In Linux,

`ping` continues sending packets once every second or so until you interrupt it with a Ctrl+C keystroke. You can instead specify a limited number of tests via the `-c` *num* option. Here's an example of its output:

```
$ ping -c 4 nessus
PING nessus.example.com (192.168.1.2) 56(84) bytes of data.
64 bytes from nessus.example.com (192.168.1.2): icmp_req=1 ttl=64↵
 time=0.607 ms
64 bytes from nessus.example.com (192.168.1.2): icmp_req=2 ttl=64↵
 time=0.147 ms
64 bytes from nessus.example.com (192.168.1.2): icmp_req=3 ttl=64↵
 time=0.145 ms
64 bytes from nessus.example.com (192.168.1.2): icmp_req=4 ttl=64↵
 time=0.283 ms

--- nessus.example.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 0.145/0.295/0.607/0.189 ms
```

This command sent four packets and waited for their return, which occurred quite quickly (in an average of 0.295 ms) because the target system was on the local network. A few common problem patterns can help you diagnose where a network problem resides:

- If you can ping local systems but not remote systems, the problem is probably in your router or in an improper router specification.
- If you can ping by IP address but not by name, the problem is most likely with your DNS server or DNS configuration.
- If you can't ping at all, even by IP address, then you probably have a fundamental configuration problem.

Finding Breaks in Connectivity

A step up from `ping` is the `traceroute` command, which sends a series of three test packets to each computer between your system and a specified target system. Listing 17.3 shows a sample of `traceroute`'s output.

Listing 17.3: A sample `traceroute` output

```
$ traceroute -n 10.1.0.43
traceroute to 10.1.0.43 (10.1.0.43), 30 hops max, 52 byte packets
 1  192.168.1.2  1.021 ms  36.519 ms  0.971 ms
 2  10.10.88.1  17.250 ms  9.959 ms  9.637 ms
 3  10.9.8.173  8.799 ms  19.501 ms  10.884 ms
 4  10.9.8.133  21.059 ms  9.231 ms  103.068 ms
```

```
5  10.9.14.9  8.554 ms  12.982 ms  10.029 ms
6  10.1.0.44  10.273 ms  9.987 ms  11.215 ms
7  10.1.0.43  16.360 ms *  8.102 ms
```

The `-n` option to this command tells it to display target computers' IP addresses rather than their hostnames. This can speed up the process a bit, particularly if you're having DNS problems, and it can sometimes make the output easier to read—but you may want to know the hostnames of problem systems because they can help you pinpoint who's responsible for a problem.

The output of `traceroute` can be diagnostic in several ways:

- Typically, times increase with increasing hop numbers. This is normal, and aside from some variability in the example output, you can see this pattern in Listing 17.3.
- Variable times on a single hop can indicate that a router is overloaded or that some other intermittent issue is causing variability in the times. You can see this factor at play in several hops in Listing 17.3, and particularly in hops 1 and 4. Such issues may result in variable performance but should not cause network sessions to time out or otherwise fail completely.
- Sometimes a response to a packet is never received. This condition is indicated by an asterisk (*) in a time field, as in the second packet for hop 7 in Listing 17.3.
- A dramatic jump in times can indicate a great physical distance between routers. Ignoring the middle time of 36.519 ms in the first hop of Listing 17.3, you can see such a jump in the change from hop 1 to hop 2. In the case of Listing 17.3, this indicates the link from the local network to the Internet service provider's network. Even bigger jumps in time are common on intercontinental links. Such jumps are not necessarily signs of misconfiguration that needs to be fixed, but they can influence network performance, particularly for highly interactive protocols such as remote login tools or network-enabled gaming.

Some routers block all `traceroute` data. If your `traceroute` output contains nothing but asterisks past a certain point, you may have run into such a system.

Among other things, `traceroute` is helpful in determining whether a problem in network connectivity exists in a network for which you're responsible. For instance, the variability in the first hop of Listing 17.3 could indicate a problem on the local network, but the lost packet associated with the final destination most likely is not a local problem. If the trouble link is within your jurisdiction, you can check the status of the problem system, nearby systems, and the network segment in general.

Testing DNS

DNS problems can cause networking to fail almost as badly as can a physically cut cable. Because both people and many network tools rely on hostnames, if DNS resolution doesn't work, the network becomes nearly useless.

Certification Objective

You can test your network's DNS server by using a number of tools, such as `host`, `dig`, and `nslookup`. These tools are similar in many ways, and they all enable you to look up a hostname by typing the command's name followed by a hostname:

The `nslookup` program is *deprecated*, meaning that developers plan to eventually remove it from common use, but it's available at the moment.

```
$ host www.sybex.com
www.sybex.com has address 208.215.179.220
```

This example shows normal name resolution. In this case, the hostname links directly to just one IP address. In some cases, you may find notices about aliases or multiple IP addresses. This is also normal, at least for some sites. If the command takes a long time to respond or complains about a timeout or an inability to reach servers, though, it means that your computer's DNS configuration is wrong.

Recall that `/etc/resolv.conf` contains the IP addresses of your network's DNS servers. They're identified on `nameserver` lines.

You can add the IP address of a specific DNS server, as in **`host www.whitehouse.gov 192.168.39.7`** to test the `192.168.39.7` server. If your computer is configured to use multiple DNS servers, you can test each one individually in this way. Perhaps one is unreliable or incorrect, in which case you might want to remove it from your configuration. Be aware, though, that if you remove a DNS server from `/etc/resolv.conf` on a computer that's configured via DHCP, your change will eventually be undone by DHCP. If this is a recurring problem, you should probably consult your network administrator.

Although `host` is a useful tool for performing basic hostname lookups, the `dig` utility (or the deprecated `nslookup`) can perform more complex queries and can return more information about a hostname, domain, or IP address. This makes `dig` a useful tool for advanced network diagnostics, but at the cost of ease of use; it's harder for a novice to interpret `dig`'s output.

Checking Your Network Status

Certification Objective

Another useful diagnostic tool is `netstat`. This is something of a Swiss Army knife of network tools because it can be used in place of several others, depending on the parameters it's passed. It can also return information that's not easily obtained in other ways. Some examples include the following:

Interface information Pass `netstat` the `--interface` or `-i` parameter to obtain information about your network interfaces similar to what `ifconfig` returns. (Some versions of `netstat` return information in the same format, but others lay out the information differently.)

Routing information You can use the `--route` or `-r` parameter to obtain a routing table listing similar to what the `route` command displays.

Masquerade information Pass `netstat` the `--masquerade` or `-M` parameter to obtain information about connections mediated by Linux's Network Address Translation (NAT) features, which often go by the name *IP masquerading*. NAT enables a Linux router to "hide" a network behind a single IP address. This can be a good way to stretch limited IPv4 addresses.

Small broadband routers use NAT. If you have such a device, chances are your desktop and laptop systems needn't be configured to do so themselves.

Program use Some versions of `netstat` support the `--program` (or `-p`) parameter, which attempts to provide information about the programs that are using network connections. This attempt isn't always successful, but it often is, so you can see what programs are making outside connections.

Open ports When used with various other parameters, or without any parameters at all, `netstat` returns information about open ports and the systems to which they connect.

All connections The `--all` or `-a` option is used in conjunction with others. It causes `netstat` to display information about the ports that server programs open to listen for network connections, in addition to already-open connections.

Keep in mind that `netstat` is a powerful tool, and its options and output aren't entirely consistent from one distribution to another. You may want to peruse its `man` page and experiment with `netstat` to learn what it can do.

Protecting Your System from the Bad Guys

Networks provide many useful features for their users. Unfortunately, they also provide many risks. Network security is a complex topic, and I can provide only a few basic tips to help you prevent your system from being compromised. Here are some of the most important things you can do:

Shut down unused servers. Linux's major security vulnerabilities aren't in worms and viruses, as they are with Windows; in Linux, the risks center around outsiders breaking into your computer by abusing server programs that you run. Therefore, it's important that you don't run servers unnecessarily. Some distributions automatically install and run servers, such as the Secure Shell (SSH), the Apache Web server, or a mail server such as sendmail or Postfix. The most thorough way to remove a server is to uninstall it using your package system (described in Chapter 9, "Using Programs and Processes")—but you must issue the command to *uninstall* the relevant package rather than to install it.

Firewalls can run on one computer to protect just it or on a router to protect a whole network.

Enable a firewall. A *firewall* is a program or system setting that polices network transactions, allowing or disallowing them based on programmed criteria. Most Linux distributions enable firewalls by default, but you may want to adjust their settings for your specific needs.

Use good passwords. Chapter 14, "Creating Users and Groups," describes how to select a good password. If your computer runs a login server of any type, setting a good password can minimize the risk of an outsider breaking in by guessing your password.

Be suspicious. You should be suspicious of untrusted sources of data. *Phishing* (attempting to extract sensitive data from users by posing as some trusted individual or organization) and similar attacks can dupe individuals into giving up passwords, financial data, and so on. Although malicious software (or *malware*) for Linux is rare, it would be easy to produce. You're best off sticking to official software sources and remembering that e-mails, Web sites, and other communications can be faked.

Keep your software up to date. Chapter 9 describes the package management tools you can use to keep your system up to date. It's best to regularly check your system for software updates, since many such updates fix security bugs—problems that can be exploited by outsiders or by legitimate but malicious users to take over your computer.

The Essentials and Beyond

Networking is integral to today's computers, and Linux provides excellent networking capabilities. In most cases, your computer will detect your network and configure itself automatically; however, sometimes you must manually create an advanced configuration or override an incorrect automatic configuration. You may also need to test your network configuration to determine where a problem lies—in an IP address setting, in a DNS configuration, or elsewhere. Unfortunately, networks can be abused as well as used, so you must remain vigilant against those who would break into your computer and use it for their own purposes. A few steps, such as keeping your software updated, can greatly minimize the risks.

Suggested Exercises

- On a small private network that you control or in a lab environment under the supervision of an instructor, reconfigure a computer that normally uses DHCP to use its assigned IP address in a static fashion. Test the connection, and then switch back to DHCP.

(Continues)